MIVA OPEN UNIVERSITY, LAGOS, NIGERIA
DEPARTMENT OF INFORMATION TECHNOLOGY, SCHOOL OF COMPUTING
MASTER OF INFORMATION TECHNOLOGY (MIT)

DESIGN AND IMPLEMENTATION OF A SECURE AWS-GCP MULTI-CLOUD ARCHITECTURE FOR ENTERPRISE WORKLOADS: A CASE STUDY OF VERDAD SOLUTIONS

BY
UGWULEBO LESILE NGOZI
(2025/A/MIT/0719)

A Professional Master's Project Report Submitted to the Department of Information Technology, School of Computing, MIVA Open University, Lagos, Nigeria, in Partial Fulfilment of the Requirements for the Award of the Professional Master of Information Technology (MIT) Degree.

JULY 2026

DECLARATION

I, Ugwulebo Lesile Ngozi, hereby declare that this Professional Master's Project Report is my own original work, prepared under the supervision of my assigned project supervisor, and has not been submitted, in whole or in part, for the award of any other degree or qualification at this or any other institution. All sources of information used have been duly acknowledged by means of complete references.

Signature: _____ Date: _____
Ugwulebo Lesile Ngozi (2025/A/MIT/0719)

CERTIFICATION / APPROVAL PAGE

This is to certify that this Professional Master's Project Report titled Design and Implementation of a Secure AWS-GCP Multi-Cloud Architecture for Enterprise Workloads: A Case Study of Verdad Solutions was prepared by Ugwulebo Lesile Ngozi (2025/A/MIT/0719) and is approved as meeting the requirements of the Department of Information Technology, School of Computing, MIVA Open University, Lagos, Nigeria, for the Professional Master's Project Assessment.

_____ _____
Project Supervisor (Name & Signature) Date

_____ _____
Head of Department (Name & Signature) Date

ACKNOWLEDGEMENTS

I acknowledge the guidance of my project supervisor, the faculty and staff of the Department of Information Technology, MIVA Open University, and the professional and academic communities whose standards, documentation and research informed this project. I also acknowledge the support of colleagues, family and friends during the planning, implementation and documentation of this Professional Master's Project.

ABSTRACT

Enterprise adoption of hybrid and multi-cloud computing can improve service flexibility and resilience, but it also introduces heterogeneous networking, identity, security-policy, observability and governance challenges. This Professional Master’s Project designs and implements a secure Amazon Web Services (AWS)-Google Cloud

Platform (GCP) multi-cloud architecture for Verdad Solutions, a fictitious Nigerian enterprise with its headquarters in Lagos and branch offices in Abuja, Imo and Port Harcourt. The organisation operates a hub-and-spoke site-to-site network, uses on-premises Active Directory synchronised to Microsoft Entra ID, and maintains an existing inventory-management workload in GCP using Compute Engine and Cloud SQL. The project extends this environment to AWS to provide a controlled secondary cloud and disaster-recovery capability without assuming that the presence of two providers automatically creates resilience.

Design Science Research Methodology is used to translate the enterprise problem into an implemented and evaluable technical artefact. The architecture combines segmented GCP Virtual Private Cloud and Amazon VPC environments, route-based IPsec connectivity, Border Gateway Protocol routing, Microsoft Entra-based workforce identity, least-privilege network controls, encryption, provider-native logging and security monitoring, and Terraform Infrastructure as Code. The literature review critically evaluates recent multi-cloud, Zero Trust and Infrastructure-as-Code research, compares provider-native and alternative approaches, and integrates Nigerian requirements under the Nigeria Data Protection Act 2023 and National Cloud Policy 2025 throughout the design rationale.

The evaluation framework measures functional correctness, authorised and prohibited traffic flows, identity controls, security posture, inter-cloud latency, packet loss, throughput and recovery behaviour during controlled tunnel or route failure. Actual numerical findings are intentionally not fabricated; Chapter Five provides controlled result fields that must be completed with evidence from the deployed laboratory environment. The project's professional contribution is a reproducible AWS-GCP reference implementation, supporting Terraform repository, architecture diagrams and evidence-driven evaluation method that Nigerian practitioners can adapt to comparable hybrid multi-cloud environments while recognising the operational and regulatory limits of a laboratory proof of concept.

TABLE OF CONTENTS

Right-click and update field to generate the Table of Contents.

LIST OF FIGURES

LIST OF TABLES

Tables are numbered within the chapters and include requirements, architecture comparisons, traffic-flow matrices, control mappings, implementation evidence and test-result registers.

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AD DS	Active Directory Domain Services
BGP	Border Gateway Protocol
CSPM	Cloud Security Posture Management
DSRM	Design Science Research Methodology
GCP	Google Cloud Platform
HA	High Availability
IAM	Identity and Access Management
IaC	Infrastructure as Code
IPsec	Internet Protocol Security
MFA	Multi-Factor Authentication
NCC	Network Connectivity Center
NDPA	Nigeria Data Protection Act 2023

NITDA	National Information Technology Development Agency
NSG	Network Security Group
RPO	Recovery Point Objective
RTO	Recovery Time Objective
SCC	Security Command Center
SIEM	Security Information and Event Management
SLA	Service Level Agreement
VNet	Amazon VPC
VPC	Virtual Private Cloud
VPN	Virtual Private Network
ZTA	Zero Trust Architecture

Chapter One
INTRODUCTION

1.0 Introduction

Cloud computing is now an operational platform for enterprise applications, continuity services, remote access and security operations. The practical challenge addressed by this project is not simply whether an organisation can consume two cloud providers, but whether it can connect distributed sites, cloud workloads and identities without creating excessive trust, fragmented monitoring or an untested recovery design. Verdad Solutions is used as the case organisation so that the architecture is tied to a defined enterprise problem rather than presented as an abstract provider comparison.

1.1 Verdad Solutions Enterprise Context

Verdad Solutions is a fictitious Nigerian enterprise with its headquarters in Lagos and offices in Abuja, Imo and Port Harcourt. The offices are connected through site-to-site links using a hub-and-spoke pattern in which Lagos is the principal enterprise hub. Shared directory, DNS, management and application services are coordinated from the headquarters environment. The organisation uses an off-the-shelf inventory-management application that has been customised for its operational processes and is accessed by authorised personnel across the four locations.

The organisation already has a Google Cloud presence. Compute Engine provides selected online application capacity and Cloud SQL provides a managed relational database for the inventory service and continuity use cases. Microsoft Entra Connect synchronises the organisation's on-premises Active Directory identities to Microsoft Entra ID, allowing users to retain a common corporate identity for supported cloud services. The proposed project extends this hybrid architecture to Amazon Web Services (AWS) so that selected recovery, monitoring and supporting services can operate in a second public cloud while workforce identity remains centrally governed.

Figure 1.1. Current-state Verdad Solutions hybrid architecture.

Figure 1.2. Proposed AWS-GCP multi-cloud target architecture.

The target architecture deliberately separates provider diversity from verified resilience. AWS is not introduced merely to claim multi-cloud status. It provides an independently controlled environment through which the project can test encrypted cross-cloud transport, routing, identity governance, segmentation, observability and recovery behaviour. This distinction is central to the research position developed in Chapter Two and evaluated in Chapter Five.

1.1.1 Evolution of enterprise cloud computing

Cloud computing represents a model through which computing resources can be provisioned and accessed as services over a network. The National Institute of Standards and Technology defines cloud computing through characteristics such as on-demand self- service, broad network access, resource pooling, rapid elasticity and measured service.

Cloud services are commonly delivered through infrastructure, platform and software service models and may be deployed through public, private, community or hybrid arrangements (Mell & Grance, 2011).

The attraction of cloud computing lies partly in its ability to separate the use of computing capacity from the ownership of the underlying physical infrastructure. An enterprise can provision virtual machines, storage, databases, networks and security services without constructing and maintaining every component within its own

data centre. Capacity can be increased or reduced in response to workload demand, and infrastructure can be deployed through automated interfaces rather than lengthy physical procurement processes.

For modern organisations, these capabilities have strategic as well as technical value. Cloud platforms can shorten the time required to launch new services, support geographically distributed users, facilitate experimentation, and provide access to specialised technologies that may be difficult to develop internally. Cloud adoption can also improve infrastructure standardisation because computing resources can be defined through templates and deployed consistently across development, testing and production environments.

Nevertheless, migration to the cloud does not remove organisational responsibility for security and governance. It changes the distribution of that responsibility. Cloud providers secure the underlying facilities and managed-service infrastructure within the boundaries of their service models, while customers remain responsible for matters such as user access, workload configuration, data classification, network rules and application security.

Misunderstanding this allocation can result in serious control weaknesses even where the underlying cloud platform is technically secure.

The complexity increases where the organisation uses more than one provider. Each cloud platform has its own naming conventions, administrative interfaces, identity structures, network objects and security services. A technology team that understands one platform may incorrectly assume that equivalent terminology or configuration behaviour exists on another. A secure multi-cloud architecture therefore requires more than connecting two networks. It requires an integrated design that reconciles different technical models while maintaining consistent security outcomes.

1.1.2 Emergence of multi-cloud enterprise environments

A multi-cloud environment exists where an organisation deliberately consumes services from more than one cloud provider. This may involve a relatively simple arrangement, such as hosting a production application in one cloud and maintaining backup resources in another. It may also involve a tightly integrated architecture in which identity, application, database, analytics and disaster recovery services are distributed across multiple providers and communicate continuously.

Several factors may motivate this model. One is the desire to avoid excessive dependence on a single provider. Although complete portability between major cloud platforms is difficult because managed services are designed differently, distributing selected workloads can reduce the impact of provider-specific service disruption or commercial dependency. A second motivation is access to specialised capabilities. An organisation may prefer one platform for infrastructure hosting but use another for identity integration, analytics, productivity services or disaster recovery.

A third motivation is resilience. A properly engineered multi-cloud arrangement can provide an additional recovery option where a serious regional or provider-level disruption affects the primary environment. However, multi-cloud adoption does not automatically create resilience. An architecture that depends on one identity provider, one Domain Name System service, one inter-cloud gateway or one manually maintained VPN configuration may still contain a critical single point of failure. Resilience must therefore be designed and tested rather than assumed.

Regulatory, contractual and customer requirements may also influence cloud distribution. Organisations processing sensitive information may need to control where data is stored, how it is transferred, which parties can access it and how security incidents are detected.

These obligations can lead to hybrid or multi-cloud designs in which particular data sets or processing functions remain within designated jurisdictions or controlled environments.

The Nigerian National Cloud Policy 2025 illustrates the increasing policy importance of secure and sovereign cloud adoption. The policy promotes a Cloud First direction while also emphasising data classification, sovereignty, secure cloud use and local data- residency considerations. It supersedes the earlier Nigeria Cloud Computing Policy and provides a more current basis for evaluating the national context of cloud architecture (National Information Technology Development Agency [NITDA], 2025).

Nigeria's data-protection framework is similarly relevant. The Nigeria Data Protection Act 2023 governs the processing of personal data and includes requirements affecting accountability, security, data-subject rights and cross-border transfers. Consequently, an enterprise cannot treat cloud location and inter-cloud data movement as purely technical matters. The organisation must be able to establish what data is processed, where it is transmitted, who can access it, what safeguards apply and whether the processing complies with applicable legal requirements (Federal Republic of Nigeria, 2023).

1.1.3 Google Cloud Platform and Amazon Web Services (AWS) as enterprise cloud platforms

Google Cloud Platform and Amazon Web Services (AWS) are broad public-cloud platforms that provide infrastructure, networking, storage, database, security, observability and application services. Both platforms can support enterprise workloads, but their resource models and administrative structures di er.

Within GCP, an enterprise workload may be hosted in an Google Cloud Virtual Private Cloud. The VPC provides a logically isolated networking environment in which the organisation defines address ranges, subnets, route tables, network gateways and security controls. Security groups provide stateful filtering around network interfaces and

workloads, while network access control lists can provide subnet-level stateless filtering. Identity and Access Management controls permissions to Google Cloud resources, and services such as GCP Key Management Service, Cloud Monitoring, Cloud Audit Logs and Security Command Center support encryption, operational monitoring, audit logging and threat detection.

Google Cloud Network Connectivity Center can function as a regional network transit hub connecting multiple VPCs, VPNs and external networks. Instead of maintaining a large number of individual point-to-point connections, the enterprise can route traffic through a central transit component. Google Cloud HA VPN can terminate on a transit gateway and provide IPsec connectivity between GCP and an external network, including a network hosted in another cloud. GCP also provides two VPN tunnels for a standard HA VPN connection and recommends configuring both tunnels for redundancy.

In Amazon Web Services (AWS), enterprise workloads are commonly deployed within Amazon VPCs. A virtual network contains subnets, routing components, private-address spaces and network-security controls. Network VPC Firewall Rules apply traffic-filtering rules to network interfaces and subnets, while AWS Network Firewall or other network virtual appliances can provide centralised inspection where required. Microsoft Entra ID supports identity and access management, and services such as AWS KMS and Secrets Manager, Amazon CloudWatch, Log Analytics and Microsoft AWS Security Hub and GuardDuty support secrets management, logging, security-posture assessment and threat protection.

AWS Site-to-Site VPN can provide encrypted connectivity between an AWS virtual network and an external network over the public Internet. AWS Virtual WAN offers a more centralised model that combines networking, routing, VPN and security capabilities through managed virtual hubs. The appropriate AWS-side design for this project will depend on lab cost, complexity and the required implementation depth, but both approaches provide credible foundations for establishing IPsec connectivity with GCP.

The technical richness of both platforms is beneficial, but it also creates integration challenges. An GCP VPC firewall rule is not identical to an AWS Network VPC Firewall Rule. Google Cloud IAM and Microsoft Entra ID implement identity and authorisation through different structures. Google Cloud Network Connectivity Center and AWS Virtual WAN perform related hub functions but are not configured in the same manner. Logs are generated in different formats and stored through different services. A secure architecture must therefore achieve functional consistency without pretending that both platforms are technically identical.

1.1.4 Enterprise workload context

This project uses a representative inventory-management enterprise workload to demonstrate and evaluate the architecture. Three-tier application design separates a system into presentation, application-processing and data-storage layers. In a conventional deployment, the web tier receives user requests, the application tier processes business logic, and the database tier stores persistent information.

The separation of these tiers creates a practical basis for security segmentation. External users should normally communicate only with an approved public endpoint or web service. The web tier may communicate with the application tier through specific ports, while the application tier may access the database through a narrowly defined database connection.

The database should not be directly reachable from the public Internet or from unauthorised network segments.

For the proposed architecture, selected tiers or supporting services will be distributed between Google Cloud and AWS. For example, a web and application component may operate within GCP while a supporting application, monitoring service, identity service or recovery component operates in AWS. This distribution generates genuine inter-cloud traffic and allows the implementation to demonstrate routing, encryption, access control, logging and failure recovery.

The workload is not intended to reproduce every feature of a large bank or telecommunications platform. It serves as a controlled proof of concept through which the security and operational behaviour of the architecture can be observed. Synthetic or non- sensitive data will be used so that the implementation does not create unnecessary privacy or confidentiality exposure.

1.1.5 Security implications of multi-cloud connectivity

The central security difficulty in a multi-cloud environment is that connectivity can create trust more quickly than governance can control it. Once Google Cloud and AWS networks are joined through a VPN or routing appliance, resources in one environment may become reachable from the other. Where routing tables, firewall rules or identity policies are overly permissive, the connection can become a path for lateral movement.

A malicious actor who compromises one workload may attempt to discover adjacent systems, access management services or transfer data through the inter-cloud link. Similarly, a configuration error may expose a database subnet, allow unrestricted administrative access or route Internet-bound traffic through an unintended path. The presence of encryption does not by itself prevent these outcomes. IPsec can protect traffic from interception in transit while still carrying unauthorised traffic between poorly segmented networks.

Identity inconsistency is another source of risk. Where separate user accounts are maintained in both Google Cloud and AWS, inactive accounts, excessive permissions and inconsistent authentication controls can develop. Shared

administrative credentials create greater danger because activity becomes difficult to attribute to an individual. A mature design should favour federated identity, role-based permissions, multifactor authentication and short-lived credentials where technically feasible.

Monitoring also becomes more difficult when security information is dispersed across platforms. Google Cloud and AWS generate logs through different services and schemas. If those records are reviewed separately, an investigator may see only part of an event. An authentication anomaly in one cloud may be connected to network activity in the other.

Centralised or correlated monitoring is therefore necessary to reconstruct cross-cloud events and support timely detection. These challenges support the adoption of Zero Trust principles. NIST explains Zero Trust as a security model that removes automatic trust based solely on network location and instead focuses on protecting users, assets and resources. Access decisions should consider identity, device or workload context, requested resource, policy and current security information. In a multi-cloud environment, this means that the existence of an encrypted network path must not be interpreted as permission for unrestricted communication (Rose et al., 2020).

NIST SP 800-207A extends this reasoning to cloud-native applications operating across multiple locations. It focuses on granular application-level policies and the runtime requirements of Zero Trust in multi-cloud and hybrid environments. Although the present project includes substantial network-level controls, its design will also reflect the broader principle that access should be explicitly authorised and continuously subject to policy rather than inherited from a trusted network boundary (Chandramouli & Butcher, 2023).

The Cloud Security Alliance Cloud Controls Matrix provides another relevant control perspective. The current CCM v4.1 contains 207 controls across 17 cloud-security domains and addresses areas such as identity and access management, infrastructure and virtualisation security, cryptography, logging, threat management, data security and business continuity. It can therefore support the project's control mapping and help translate broad security requirements into implementable and testable measures (Cloud Security Alliance, 2026).

1.1.6 Infrastructure as Code and reproducibility

Manual configuration presents a particular problem in multi-cloud environments. Where engineers create networks, subnets, gateways, route tables and security rules through separate web portals, it becomes difficult to reproduce the environment exactly.

Configuration steps may not be documented, naming standards may vary, and changes may be applied without review.

Infrastructure as Code addresses this problem by defining infrastructure in machine- readable configuration files. Terraform is suitable for this project because it can manage Google Cloud and AWS resources through provider plugins within a unified workflow. The configuration can be version-controlled, reviewed and applied repeatedly. This does not remove the need for sound architectural judgement, but it creates a clearer relationship between the approved design and the deployed resources.

The Terraform codebase will be organised into logical components such as Google Cloud networking, AWS networking, VPN connectivity, security controls, compute resources and monitoring. Variables will be used for parameters such as address ranges, regions and resource names. Outputs will expose information required by connected modules.

Sensitive values will not be embedded directly in source files.

Infrastructure as Code also strengthens evaluation. The deployed architecture can be inspected through its code, compared with the documented design and recreated where necessary. Where a test reveals an inappropriate rule, the correction can be recorded as a controlled code change rather than an undocumented portal adjustment. This supports the professional requirement for a reproducible technical artefact.

1.1.7 Need for an implemented and evaluated reference architecture

Considerable guidance exists on cloud security, Zero Trust, Google Cloud networking and AWS networking. However, these sources often address individual technologies or provide high- level architectural principles. An enterprise architect must still decide how to combine them into a functioning design.

Vendor documentation is essential for accurate implementation, but it is usually organised around the provider's own platform. GCP documentation explains how Network Connectivity Center and HA VPN operate, while Microsoft documentation explains AWS Site-to-Site VPN and Virtual WAN. Neither provider is responsible for producing an independent academic evaluation of the complete AWS-GCP architecture proposed in this project.

Similarly, Zero Trust publications explain principles and logical models but do not prescribe a single provider-specific deployment. This is appropriate because Zero Trust must be adapted to organisational context. It nevertheless leaves a practical gap between conceptual guidance and tested implementation.

The approved project proposal was developed to address this gap through a secure hub- and-spoke architecture, Terraform implementation, identity and segmentation controls, centralised observability, and measurable security, performance and resilience evaluation.

The proposal also limits the implementation to a lab-scale three-tier workload and excludes a full penetration test, formal compliance audit and production-scale deployment.

The project therefore occupies a professional middle ground. It is more concrete than a purely conceptual architecture because it produces and tests a working artefact. At the same time, it does not claim to represent a complete production deployment for every enterprise. Its value lies in demonstrating a reproducible design approach, identifying implementation challenges and generating evidence about the behaviour of the resulting architecture.

1.2 Conceptual Foundation of the Project

The conceptual foundation of the project connects five elements: enterprise requirements, security and regulatory drivers, architecture design, automated implementation and empirical evaluation. The project begins with the practical difficulties created when enterprise workloads are distributed between Google Cloud and AWS. Those difficulties are translated into requirements for secure connectivity, segmentation, identity control, observability, resilience and data governance.

The requirements then shape the architecture. Google Cloud and AWS networks are structured around hub-and-spoke principles, with controlled routes between workload segments and an encrypted inter-cloud connection. Zero Trust and defence-in-depth principles influence how access is authorised, how traffic is filtered and how security events are monitored.

Terraform converts the design into a deployable artefact. The implemented environment is then tested using configuration-scanning tools and network-testing utilities. Findings from the tests are compared with the project objectives and predefined success criteria. Where weaknesses are identified, the design can be refined and redeployed.

Figure 1.1: Conceptual framework for the secure AWS-GCP multi-cloud project

The feedback path in the diagram is important. The project does not assume that the first design will be perfect. Testing may identify a route that is too permissive, a security control that is not operating as expected, or a failover process that takes longer than the defined target. Such results will lead to design refinement. This reflects the iterative nature of Design Science Research and the professional reality of cloud architecture, where solutions are progressively validated rather than accepted solely because they appear correct on paper.

1.3 Statement of the Problem

Enterprises that adopt Google Cloud Platform and Amazon Web Services (AWS) concurrently often need to connect workloads, users and shared services across the two platforms. In practice, such connectivity may begin as a limited operational requirement. A team may create a VPN to enable one application to communicate with another, permit an administrator to access a remote environment or transfer backup data between platforms. Over time, additional routes, accounts and firewall exceptions are added.

Where this growth occurs without an integrated architecture, the result is frequently an ad hoc multi-cloud environment. Network connectivity may depend on isolated point-to-point tunnels. Route tables may be configured independently. Security rules may use inconsistent naming and access standards. Logging may remain separated between provider portals. Identity administration may be duplicated, with users holding unrelated credentials and permissions in both clouds.

This arrangement creates several related problems. First, it makes security-policy enforcement inconsistent. A communication path that is restricted in GCP may be allowed more broadly in AWS, or vice versa. Because the platforms use different security constructs, superficially similar rules may produce different results.

Second, fragmented identity management can result in excessive privileges and weak accountability. An administrator may have broad permissions in one cloud and narrower permissions in the other. Dormant accounts may remain active, and emergency credentials may become part of normal operations. Without federation or coordinated governance, the organisation cannot easily establish a consistent access-control model.

Third, uncoordinated inter-cloud routing can increase the attack surface. An encrypted VPN connects networks, but it does not determine which applications should communicate. If broad address ranges are permitted, a compromised system in one cloud may scan or access resources in the other. The organisation may therefore create a trusted network extension that conflicts with the principle of explicit and least-privilege access.

Fourth, fragmented monitoring reduces visibility. Google Cloud and AWS security events may be collected independently without correlation. An incident that begins with a suspicious identity event in AWS and continues with network activity in GCP may not be recognised as one sequence. Delayed detection increases the likelihood that an attacker can maintain access or transfer sensitive information.

Fifth, resilience may be assumed rather than verified. A design may contain two cloud providers but still depend on one VPN tunnel, one gateway, one route or one manually executed recovery process. Without controlled failure tests and a measured Recovery Time Objective, the organisation cannot demonstrate that its inter-cloud architecture will continue to operate or recover within an acceptable period.

These technical weaknesses are particularly important where the architecture processes personal, financial or operationally sensitive information. The Nigeria Data Protection Act requires organisations to apply appropriate safeguards to personal-data processing and to remain accountable for how data is handled, including where it is transferred across borders or processed by service providers. The National Cloud Policy 2025 also places greater

emphasis on secure, sovereign and properly classified cloud use. A poorly governed multi-cloud environment can make it difficult to demonstrate compliance because the organisation may not maintain clear visibility of data location, access paths and security controls.

The problem is not an absence of individual cloud-security technologies. Google Cloud and AWS both provide mature networking, encryption, access-control and monitoring services. The deeper problem is the absence, in many practical implementations, of a coherent and independently evaluated method for combining those technologies into a secure cross- cloud architecture.

Existing standards and frameworks provide valuable direction, but they remain technology- neutral or control-oriented. NIST SP 800-207 establishes Zero Trust principles, and NIST SP 800-207A addresses application-level access control across multiple cloud locations. The CSA Cloud Controls Matrix provides a broad catalogue of cloud-security controls. These sources do not, however, substitute for the detailed design, implementation and testing of a specific AWS-GCP environment.

The specific problem addressed by this project is therefore the lack of a practical, reproducible and empirically evaluated AWS-GCP reference architecture that combines encrypted inter-cloud networking, workload segmentation, controlled identity, centralised observability and Infrastructure as Code for a representative enterprise workload.

Without such an artefact, organisations and practitioners may understand the principles of multi-cloud security but still lack evidence about how the controls interact in an implemented environment. They may also be unable to determine whether the architecture meets measurable security, performance and resilience criteria.

The project responds to this problem by developing a lab-scale architecture and evaluating it through automated configuration assessment, access-control tests, latency and throughput measurements, and controlled connectivity-failure scenarios. The evaluation is intended to show not merely that the environment can be deployed, but whether it performs its intended security and operational functions.

The following questions guide the project:

1. What limitations exist in current multi-cloud architectures, security frameworks and regulatory requirements relevant to AWS-GCP enterprise deployments?

2. How can a secure hub-and-spoke AWS-GCP architecture be designed for a representative inventory-management enterprise workload?

3. How can Terraform be used to implement the proposed architecture consistently across Google Cloud and AWS?

4. How can Zero Trust and defence-in-depth controls be configured across the two cloud platforms?

5. How effective is the implemented architecture when evaluated against predefined security, performance and resilience criteria?

1.4 Aim and Objectives of the Project

1.4.1 Aim

The aim of this project is to design, implement and evaluate a secure, resilient and compliant AWS-GCP multi-cloud architecture for a representative enterprise workload using Zero Trust principles, defence-in-depth security and Infrastructure as Code.

The aim contains three connected professional activities. The first is architecture design, through which requirements are converted into network, identity, monitoring and security- control models. The second is implementation, through which the design is deployed in Google Cloud and AWS using Terraform and native cloud services. The third is evaluation, through which the architecture is tested against measurable criteria.

The use of the term compliant does not mean that the project will conduct a formal legal or regulatory certification. Rather, the architecture will be designed with reference to relevant requirements and recognised control frameworks. Formal compliance ultimately depends on the organisation's complete operations, contracts, policies, applications, data processing and governance arrangements, which extend beyond the scope of a lab implementation.

1.4.2 Objectives

To achieve the stated aim, the project will pursue the following objectives:

1. To review relevant multi-cloud architectures, interconnection approaches, Zero Trust publications, cloud-security frameworks and Nigerian regulatory requirements, and to identify the limitations that justify the proposed solution.

This objective establishes the knowledge base for the project. The review will cover Google Cloud and AWS networking models, native and third-party interconnection approaches, identity federation, logging, encryption, NIST SP 800-207, NIST SP 800- 207A, the Cloud Controls Matrix, the Nigeria Data Protection Act and the National Cloud Policy. It will distinguish general guidance from solutions that have been implemented and independently evaluated.

2. To design a secure hub-and-spoke AWS-GCP architecture for a representative inventory-management enterprise workload, incorporating network segmentation, encrypted inter-cloud connectivity, identity controls and centralised observability.

The design will define address spaces, network segments, routing relationships, security boundaries, authorised traffic flows, gateway components, logging sources and management access. It will also define the relationship between the workload tiers and the controls that prevent unauthorised lateral movement.

3. To implement the architecture across Google Cloud and AWS using Terraform Infrastructure as Code.

Terraform will be used to provision and configure the principal cloud resources. The implementation will be modular, parameterised and version-controlled. Manual actions that cannot reasonably be automated will be clearly documented.

4. To configure and validate security controls based on Zero Trust and defence-in- depth principles.

The controls will include least-privilege network rules, encryption in transit, identity and access restrictions, micro-segmentation, secure administrative access, central logging, threat detection and configuration assessment. The project will verify whether authorised communication succeeds and unauthorised communication is blocked.

5. To test and evaluate the architecture using predefined security, performance and resilience criteria.

The security evaluation will use tools such as Prowler and ScoutSuite, supplemented by controlled connectivity and access tests. Performance testing will measure latency and throughput using tools such as ping and iperf3. Resilience testing will simulate a tunnel or connectivity failure and measure recovery behaviour against a defined Recovery Time Objective.

The objectives are deliberately measurable. The project will not judge success merely by the presence of cloud resources. Success will depend on whether the deployed environment satisfies the intended control, communication, monitoring and recovery requirements.

1.5 Scope of the Project

1.5.1 Technical scope

The project covers the design and implementation of secure network connectivity between Google Cloud Platform and Amazon Web Services (AWS). It focuses on the cloud infrastructure and security controls required to support a representative enterprise workload across both providers.

On the GCP side, the project will include a VPC-based environment containing segmented subnets for selected workload tiers. Depending on the final detailed design, Google Cloud Network Connectivity Center or an equivalent central routing component will provide transit connectivity, while Google Cloud HA VPN will establish the encrypted link to AWS. Security groups, route tables and network access controls will restrict traffic between tiers and across the inter- cloud boundary.

On the AWS side, the project will include one or more virtual networks with equivalent workload segmentation. AWS Site-to-Site VPN or a suitable Virtual WAN configuration will provide the AWS endpoint for the IPsec connection. Network VPC Firewall Rules, route tables and related controls will regulate traffic within AWS and between AWS and Google Cloud.

The implementation will include an encrypted VPN-based inter-cloud connection. A private dedicated interconnect, such as GCP Cloud Interconnect combined with an appropriate AWS connectivity service, will be considered as an enterprise-scale design alternative but will not form the primary lab implementation because of cost and provisioning constraints.

The project will also include identity and access-management controls. These will cover administrative access, least-privilege roles, multifactor-authentication requirements where available, and federation between Microsoft Entra ID and GCP where technically feasible within the lab environment. Where complete federation cannot be implemented without production licences or organisational prerequisites, the limitation and an appropriate enterprise pattern will be documented.

Monitoring will be implemented using relevant native services. Google Cloud logs may include Cloud Audit Logs, Cloud Monitoring, VPC Flow Logs and Security Command Center findings. AWS sources may include AWS CloudTrail, Network VPC Firewall Rule flow information where available, Amazon CloudWatch, Log Analytics and Microsoft AWS Security Hub and GuardDuty. A centralised or correlated view will be developed to the extent permitted by the lab environment and available service tiers.

The architecture will be implemented primarily through Terraform. Supporting scripts may use GCP Command Line Interface, AWS CLI, PowerShell or Linux shell commands for validation and operational tasks. The source code will be maintained through version control.

1.5.2 Workload scope

The workload will represent a inventory-management application organised into presentation, application and data tiers consisting of web, application and database functions. It will operate at proof-of-concept scale and use small virtual machines or lightweight services appropriate to the available cloud budget.

The workload will generate representative inter-cloud communication so that routing, encryption, segmentation and monitoring can be tested. It will not contain actual banking, customer or confidential enterprise data. Synthetic records will be used to avoid privacy risks.

The project is concerned primarily with infrastructure and network security. It will configure sufficient operating-system and service settings to support testing, but it will not undertake a full secure-software-development assessment of the application code.

1.5.3 Security-testing scope

Security testing will focus on cloud configuration, network access and visibility. Prowler and ScoutSuite will be used where technically applicable to identify insecure settings, excessive permissions, missing logging, weak encryption configurations and other cloud- posture issues.

Network tests will verify that intended communication paths are available and that prohibited paths are blocked. For example, the web tier may be permitted to reach a specified application port, while direct web-to-database access should be denied.

Administrative services will be restricted to approved sources or secure management paths.

The project will not perform an unrestricted penetration test against cloud-provider infrastructure or third-party systems. It will not conduct destructive exploitation, social engineering or testing beyond resources owned and authorised for the project.

1.5.4 Performance and resilience scope

Performance evaluation will measure inter-cloud round-trip latency and throughput under controlled lab conditions. Ping or an equivalent tool will provide latency measurements, while iperf3 will measure achievable throughput between selected endpoints.

The results will be interpreted within the limitations of Internet-based VPN connectivity, virtual-machine capacity, cloud-region selection, encryption overhead and the chosen service tiers. They will not be presented as universal benchmarks for all AWS-GCP deployments.

Resilience testing will examine the architecture's response to a controlled tunnel or route failure. The test will observe whether traffic moves to an alternative tunnel or recovers after corrective action and will record the recovery time. Because the project is a lab implementation, it may evaluate both automatic and documented manual failover, depending on the final gateway configuration.

1.5.5 Regulatory and governance scope

The project will consider the Nigeria Data Protection Act 2023, the National Cloud Policy 2025 and relevant cloud-security control frameworks. This consideration will influence data classification, logging, access control, encryption, data-location awareness and documentation.

However, the project will not constitute a formal legal opinion, data-protection audit or regulatory certification. It will not assess every obligation applicable to a specific bank, telecommunications operator or public institution. Regulatory requirements will be treated as design inputs and evaluation considerations rather than as a claim of complete organisational compliance.

1.5.6 Exclusions and limitations

The project is restricted to Google Cloud Platform and Amazon Web Services (AWS). Integration with Google Cloud Platform, Oracle Cloud, private cloud platforms or an on-premises data centre is outside the implementation scope.

Production-scale availability testing, dedicated carrier connectivity, full disaster-recovery orchestration, enterprise Security Information and Event Management integration, advanced Security Orchestration, Automation and Response, full application penetration testing and detailed cost optimisation are excluded.

The project will also be affected by cloud-service costs, student or trial-account limitations, regional service availability and the time available within the academic schedule. Certain premium services may therefore be represented through architectural design rather than fully deployed.

These boundaries are consistent with the approved proposal, which defines the artefact as a lab-scale implementation and excludes a full third-party penetration test, formal compliance audit, extensive application-layer assessment and detailed production cost optimisation.

1.6 Significance of the Project

1.6.1 Significance to enterprise organisations

The project will provide a structured reference for enterprises considering or operating AWS-GCP environments. Its value lies in demonstrating how separate cloud services can be combined into one controlled architecture rather than treated as unrelated technical deployments.

An enterprise may adapt the resulting network segmentation model, routing principles, security-rule structure and logging approach to its own environment. The Terraform implementation will also illustrate how cloud infrastructure can be codified and reviewed.

The project is particularly relevant to organisations whose digital services depend on multiple technology providers. Banks, payment-service companies, telecommunications operators and government agencies may need to combine cloud services while retaining visibility and control over data flows. A tested architecture can assist such organisations in identifying decisions that must be made before inter-cloud connectivity is enabled.

1.6.2 Significance to the Nigerian context

Nigeria's digital economy increasingly depends on reliable and secure infrastructure. Cloud platforms can support service availability, innovation and rapid deployment, but poorly governed cloud adoption can create privacy, sovereignty and cybersecurity risks.

The National Cloud Policy 2025 promotes cloud adoption while emphasising secure, sovereign and economically beneficial use. The policy's focus on classification and residency means that cloud architecture must be accompanied by a clear understanding of the information being processed and the jurisdictions in which it moves.

The Nigeria Data Protection Act 2023 also creates accountability obligations for organisations processing personal data. A multi-cloud environment must therefore support controlled access, appropriate safeguards, auditability and defensible cross-border- transfer decisions. The project's emphasis on encryption, identity, logging and explicit traffic control reflects these concerns.

Although a lab architecture cannot solve every national cloud-governance issue, it can demonstrate how policy requirements influence technical design. This is professionally valuable because compliance is often discussed at policy level without showing how it affects route tables, identity roles, network controls, logs and deployment code.

1.6.3 Significance to cybersecurity practice

Multi-cloud security requires controls to operate consistently across different platforms. This project will demonstrate how Zero Trust principles can be translated into practical rules and configurations. Rather than treating the VPN as a trusted extension of both networks, the design will restrict each communication path according to workload need.

The project will also show how defence in depth can be applied. A traffic flow may be controlled by route design, security-group rules, Network VPC Firewall Rule rules, host configuration and identity permissions. If one control is incorrectly configured, another control may still reduce exposure.

The use of security-scanning tools will support evidence-based improvement. Findings will be recorded, assessed and remediated rather than merely mentioned as theoretical possibilities. This is important for professional cybersecurity practice, where control effectiveness must be verified.

1.6.4 Significance to cloud architecture practice

The project contributes a provider-specific design that brings Google Cloud and AWS constructs into one architectural model. It will document how address planning, route propagation, VPN parameters, security rules, naming standards and monitoring sources interact.

This documentation can help practitioners recognise that equivalent outcomes may require different implementation methods. For instance, the objective of limiting application-to-database traffic applies equally in both clouds, but the relevant objects and rule-processing behaviour di er.

The project will also compare the lab VPN design with enterprise alternatives. Organisations requiring predictable private connectivity may consider dedicated circuits and managed routing services. By distinguishing proof-of-concept choices from production recommendations, the project will remain commercially and professionally realistic.

1.6.5 Significance to Infrastructure as Code practice

The Terraform codebase will provide a reproducible expression of the architecture. This supports repeatability, peer review and controlled modification.

Where infrastructure is deployed manually, an architecture diagram may not accurately represent the running environment. With Infrastructure as Code, the relationship can be tested more directly. The declared resources, variables and dependencies provide evidence of how the system was intended to operate.

The code will also support learning and reuse. Practitioners can examine the modules, identify parameters requiring adaptation and understand the dependencies between Google Cloud and AWS resources. Sensitive credentials and environment-specific values will remain separated from reusable configuration.

1.6.6 Significance to professional and academic knowledge

The project contributes to professional knowledge by connecting conceptual frameworks with implementation evidence. NIST and CSA publications provide recognised principles and control categories, while GCP and Microsoft provide accurate service documentation.

The project applies these sources to one defined problem and evaluates the resulting artefact.

Its contribution is therefore not the invention of a new encryption protocol or networking algorithm. Rather, it is the design, implementation and systematic evaluation of a coherent solution using established technologies. This form of contribution is appropriate to a Professional Master's Project because it demonstrates advanced application of existing knowledge to a real-world information technology problem. The MIVA regulations specifically position the project as practice-oriented and solution-driven and assess implementation quality, technical depth, testing, evaluation and professional documentation.

1.6.7 Expected professional contribution

At completion, the project is expected to produce a reusable AWS-GCP architecture model, a version-controlled Terraform codebase, configuration guidance, diagrams, test procedures and evaluation results.

The professional contribution will include evidence concerning:

how Google Cloud and AWS can be connected securely through an IPsec-based hub-and-spoke model;

how segmentation rules can limit inter-tier and inter-cloud communication;

how identity and monitoring controls can be coordinated across both platforms;

how cloud-security scanning can identify and support remediation of configuration weaknesses;

how latency, throughput and recovery behaviour can be measured in a lab-scale multi-cloud deployment; and

how Nigerian data-protection and cloud-policy considerations can be reflected in technical architecture decisions.

These outputs will make the project useful beyond its academic assessment. They can serve as a foundation for technical workshops, architecture reviews, proof-of-concept deployments and further research.

1.7 Organisation of the Remaining Project Report

Following this introductory chapter, Chapter Two will review the literature and technology context relevant to multi-cloud architecture, cloud interconnection, Zero Trust, identity federation, Infrastructure as Code, security monitoring and Nigerian regulatory requirements. It will critically compare existing approaches and establish the specific solution gap addressed by the project.

Chapter Three Will Present The Methodology And System Design. It Will Justify The Use Of Design Science Research Methodology, define functional and non-functional requirements, present the Google Cloud and AWS network designs, explain the Terraform structure and document the security, monitoring and evaluation models.

Chapter Four Will Describe The Implementation. It Will Explain The Development environment, Terraform deployment process, Google Cloud and AWS configurations, VPN setup, routing, identity controls, monitoring components, security hardening measures and implementation challenges. Relevant code listings, command-line instructions and screenshots will be included.

Chapter Five Will Present The Testing Strategy, Results And Evaluation. It Will Document security-scanning results, authorised and unauthorised traffic tests, latency measurements, throughput tests, tunnel-failure scenarios and the evaluation of each project objective.

Chapter Six Will Summarise The Completed Project, Present The Conclusion, Identify Its professional contributions and recommend future enhancements.

1.8 Chapter Summary

This chapter established the practical and professional context for the design and implementation of a secure AWS-GCP multi-cloud architecture. It explained that enterprises increasingly distribute workloads across multiple cloud providers to obtain flexibility, specialised capabilities, resilience and reduced provider dependence. It also showed that multi-cloud adoption introduces additional complexity in network design, identity, policy enforcement, monitoring, data governance and recovery.

The chapter identified the central problem as the absence of a coherent, reproducible and independently evaluated approach for combining Google Cloud and AWS networking and security services for an enterprise workload. Ad hoc VPN connections and inconsistent cloud configurations may create broad trust relationships, fragmented visibility and unverified resilience.

To address this problem, the project will design a segmented hub-and-spoke architecture, implement it through Terraform, configure security and monitoring controls, and evaluate it using security, performance and resilience tests. The project is limited to a lab-scale three- tier workload and will not claim to represent a full production deployment, penetration test or compliance audit.

The chapter also explained the significance of the project to enterprise organisations, Nigerian cloud adoption, cybersecurity practice, Infrastructure as Code and professional information technology knowledge. Its main

contribution will be an implemented and evaluated technical artefact supported by reusable code, diagrams, configurations and test evidence.

Transition to Chapter Two

Chapter Two Will Critically Examine The Concepts, Technologies, Frameworks And Existing solutions that inform the proposed architecture. It will review multi-cloud computing, AWS-GCP connectivity, hub-and-spoke networking, Zero Trust, defence in depth, identity federation, cloud monitoring, Infrastructure as Code and relevant Nigerian regulatory requirements. The review will distinguish conceptual guidance from implemented solutions and develop the gap analysis upon which the project design is based.

Chapter Two
LITERATURE REVIEW AND TECHNOLOGY CONTEXT

Revision prepared to address supervisor comments on critical analysis, recent peer-reviewed evidence, AWS–GCP implementation comparison, Nigerian context, diagrams, section synthesis, and repetition.

2.0 Introduction

2.0.1 Critical review emphasis and recent evidence

This revised literature review treats vendor documentation as implementation evidence rather than as the principal basis for scholarly claims. Alonso et al. (2023), in a systematic literature review of multi-cloud-native applications, show that provider diversity addresses some concentration and lock-in concerns while simultaneously increasing heterogeneity, interoperability and operational complexity. Diningrat, Suhardi and Rahardjo (2025) similarly identify authentication, authorisation, cryptography, data protection and availability as recurring multi-cloud security problem areas. These findings challenge simplistic arguments that multi-cloud adoption is inherently more secure: security depends on the integration and verification of controls across provider boundaries.

The same critical position applies to Infrastructure as Code. Verdet et al. (2023) analysed 812 open-source Terraform projects spanning GCP, AWS and Google Cloud and found uneven adoption of security practices, illustrating that reproducibility does not guarantee secure configuration. More recent Zero Trust scholarship also warns against reducing Zero Trust to a network product. Gambo and Almulhem (2026) synthesise a decade of Zero Trust research and identify continuous authentication, conditional access, least privilege and dynamic evaluation as recurring principles while emphasising adoption complexity. Consequently, this project treats Terraform, IPsec and segmentation as mechanisms that require independent validation rather than as evidence of security in themselves.

The Nigerian context changes the architecture questions rather than merely adding a compliance paragraph. The Nigeria Data Protection Act 2023 makes security safeguards, accountability and cross-border processing relevant to data-flow design. The National Cloud Policy 2025 and its Technical Document add explicit emphasis on cloud interoperability, portability, security operations and sovereign governance. These requirements support the project's focus on documented data paths, attributable identity, controlled cross-cloud routing, auditable logs and an explicit distinction between laboratory feasibility and production regulatory assurance.

Figure 2.1. Literature-derived integrated trust and control model.

This chapter critically reviews the contemporary literature and technology context relevant to the design, implementation and evaluation of a secure AWS–GCP multi-cloud architecture for enterprise workloads. The review is deliberately analytical rather than product-descriptive. It evaluates what existing research demonstrates, where findings converge or conflict, what remains insufficiently tested, and how those limitations translate into requirements for the proposed artefact.

The chapter retains foundational sources where they establish enduring concepts, but places greater weight on recent peer-reviewed evidence. Alonso et al. (2023), for example, show through a systematic literature review that multi-cloud adoption can improve flexibility and reduce provider dependence while simultaneously increasing architectural heterogeneity, interoperability difficulty, monitoring complexity and operational burden. More recent empirical Infrastructure as Code research similarly demonstrates that automation improves reproducibility but does not itself guarantee secure configuration: Verdet et al. (2025) found uneven adoption of security policies across 812 Terraform projects spanning GCP, AWS and Google Cloud.

The Nigerian context is treated as a design constraint throughout rather than as a separate legal afterthought. The Nigeria Data Protection Act (NDPA) 2023 affects accountability, security safeguards and cross-border processing, while the National Cloud Policy 2025 places additional emphasis on secure cloud adoption, data classification and sovereignty. Consequently, decisions about inter-cloud routing, identity, logging, encryption and data placement have both technical and governance consequences.

The chapter is organised around six connected analytical questions: why enterprises adopt multi-cloud; how Google Cloud and AWS can be interconnected without creating excessive trust; how identity, segmentation and Zero Trust should operate as one control system; how Terraform affects reproducibility and configuration risk; how observability and resilience should be evaluated; and what prior work has not yet demonstrated in an integrated AWS–GCP and Nigerian enterprise setting.

Figure 2.1. Literature-derived conceptual model for the proposed AWS–GCP artefact.

2.1 Multi-Cloud Enterprise Architecture: Concepts and Contemporary Evidence

2.1.1 From cloud adoption to multi-cloud architecture

Cloud computing is defined by NIST through on-demand self-service, broad network access, resource pooling, rapid elasticity and measured service (Mell & Grance, 2011). These characteristics remain useful, but they do not explain the principal challenge addressed in this project: the coordination of security and operations across heterogeneous providers. Multi-cloud architecture is therefore treated here not simply as the use of two cloud accounts, but as an integrated arrangement of workloads, networks, identities, data flows, policy controls and operational processes spanning more than one provider.

Alonso et al. (2023) identify heterogeneity as a defining characteristic of multi-cloud-native systems. Their review is important because it challenges a common strategic assumption: adding providers can reduce concentration risk, but it also creates additional interfaces, deployment models, observability paths and governance responsibilities. Thus, multi-cloud is not inherently more resilient or more secure than single-cloud. Its value depends on whether the organisation can control the complexity it introduces.

For the proposed artefact, this distinction is material. Google Cloud and AWS provide comparable categories of service, but their constructs are not interchangeable. An GCP VPC firewall rule and an AWS Network VPC Firewall Rule both filter traffic, yet their attachment models, rule semantics and surrounding routing context differ. Likewise, Google Cloud IAM and Microsoft Entra ID support identity and authorisation through different administrative structures. A defensible architecture therefore requires outcome consistency rather than superficial service equivalence.

2.1.2 Enterprise drivers and their limitations

The literature repeatedly identifies resilience, access to differentiated services, vendor-risk reduction and workload optimisation as drivers of multi-cloud adoption. These benefits are conditional. Provider diversity does not remove dependency if both environments rely on one identity provider, one DNS path, one untested VPN design or one administrative team. Similarly, vendor lock-in may be reduced at the infrastructure layer while remaining strong at the managed-service, data-model or identity layer.

This has two implications for the project. First, the artefact should not claim resilience merely because resources exist in Google Cloud and AWS; resilience must be demonstrated through controlled failure and recovery measurement. Second, the design should distinguish portability of provisioning workflow from portability of cloud services. Terraform can standardise how infrastructure is declared, but it cannot make Google Cloud Network Connectivity Center and AWS Virtual WAN technically identical.

2.1.3 Nigerian enterprise relevance

For Nigerian enterprises, multi-cloud decisions also interact with data governance, connectivity economics, regulatory accountability and skills availability. The NDPA 2023 requires appropriate safeguards and accountability for personal-data processing, including circumstances involving cross-border transfer. The National Cloud Policy 2025 further elevates data classification, sovereignty and secure cloud adoption. These requirements do not prescribe a unique AWS–GCP topology, but they increase the importance of knowing which data crosses the inter-cloud boundary, which identities can access it, which logs evidence the activity, and where security responsibility resides.

Accordingly, the proposed architecture should avoid treating the AWS–GCP link as a generic trusted backbone. Traffic crossing the link should correspond to documented application need, and the evaluation should produce evidence that unauthorised flows are denied. This makes network design directly relevant to governance and auditability.

Section synthesis and implication for the artefact. The literature supports multi-cloud as a potentially valuable enterprise strategy but rejects the assumption that provider diversity automatically produces resilience, portability or security. The artefact must therefore demonstrate controlled interoperability rather than merely dual-cloud deployment. This informs Objective 2 (architecture design) and Objective 5 (measurable security, performance and resilience evaluation).

2.2 AWS–GCP Interoperability and Cross-Cloud Connectivity

2.2.1 Native networking models

Google Cloud Virtual Private Cloud and Amazon VPC both provide logically isolated address spaces, subnets, routing and workload-level controls. Their conceptual similarity can obscure operational differences. Google Cloud Network Connectivity Center centralises routing among VPCs, VPNs and external networks, whereas AWS can centralise connectivity through Virtual WAN or a hub virtual network. The literature and vendor guidance therefore support a hub-and-spoke pattern, but the critical issue is not the label 'hub'; it is whether route propagation, inspection and segmentation remain explicit and testable.

A hub can reduce full-mesh complexity, yet it can also amplify configuration error. A broadly propagated route can make multiple spokes reachable even when application requirements do not justify that access. Centralisation also creates dependency on gateway health and route policy. The proposed design therefore uses the hub as a policy concentration point, not as evidence that all connected networks should trust one another.

2.2.2 VPN, BGP and resilience as one design problem

Site-to-site IPsec is appropriate for the project because it provides encrypted inter-cloud transport without the provisioning delay and cost of dedicated carrier connectivity. However, encryption solves only the confidentiality and integrity of packets in transit. It does not determine whether a web tier should reach a database tier or whether an administrator should reach a management endpoint. Treating the VPN as a security boundary would therefore recreate the perimeter trust model that Zero Trust seeks to reduce.

Route-based VPN with BGP is preferable to a purely static design because it separates encryption from reachability and can support route withdrawal during path failure. Yet dynamic routing also introduces risk: incorrect prefix advertisement can expand reachability. The architecture should consequently restrict advertised prefixes, document expected routes and test failover behaviour. Multiple tunnels are useful only where health detection and routing actually move traffic to an alternative path.

Dedicated connectivity such as GCP Cloud Interconnect combined with AWS ExpressRoute through a provider may offer more predictable latency and throughput for production workloads, but it is less suitable for a reproducible academic proof of concept because of cost, provisioning lead time and dependence on carrier arrangements. The literature review therefore distinguishes the project's lab-optimal design from an enterprise production recommendation.

2.2.3 Nigerian operational implications

An Internet-based AWS–GCP VPN is especially important to evaluate rather than assume in a Nigerian professional context because end-to-end performance depends on cloud-region selection, public Internet routing, local access networks and encryption overhead. The project should therefore report latency and throughput as measured properties of the selected deployment rather than generalise them as universal AWS–GCP performance. Where regulated or latency-sensitive production workloads require stronger predictability, dedicated connectivity should be presented as an architectural alternative.

Section synthesis and implication for the artefact. Cross-cloud connectivity must be evaluated as a combined routing, encryption, policy and resilience problem. The artefact will therefore use route-based IPsec with BGP and controlled prefixes, while network and identity controls determine authorisation after tunnel establishment. This directly informs the connectivity design in Objective 2 and the failover, latency and throughput tests in Objective 5.

2.3 Integrated Security Model: Zero Trust, Identity and Segmentation

2.3.1 Zero Trust as an architectural constraint, not a product

NIST SP 800-207 defines Zero Trust around explicit, resource-focused access decisions rather than implicit trust derived from network location. NIST SP 800-207A extends this reasoning to cloud-native applications operating across multiple locations. Recent scholarship reinforces the need to distinguish the principle from its implementation. Gambo and Almulhem (2026), in a systematic review covering research from 2016–2025, identify continuous authentication, conditional access, dynamic trust evaluation and least privilege as recurring elements, while also highlighting implementation complexity and the need for adaptable enforcement.

For this project, Zero Trust should not be repeated as a separate justification for every control. It is better treated as the governing logic connecting identity, segmentation, routing and telemetry. The VPN creates a transport path; identity establishes who or what is requesting access; network and workload policy limit reachable resources; and telemetry provides evidence for continuing evaluation. No single component is sufficient on its own.

2.3.2 Identity federation and least privilege

Maintaining unrelated permanent administrator identities in both providers increases lifecycle and accountability risk. Federation can reduce credential duplication by allowing a central identity authority to issue or broker

temporary access to provider-specific roles. In the proposed design, Microsoft Entra ID integration with Google Cloud Workforce Identity Federation or a suitable SAML role-federation pattern provides a practical workforce-identity model.

Federation nevertheless does not eliminate authorisation risk. A federated user can still receive an excessively privileged Google Cloud IAM role or AWS IAM assignment. The design must therefore combine federation with role scoping, MFA, separation of duties and periodic review. This is particularly relevant to Nigerian accountability requirements because the architecture should make privileged activity attributable to identifiable users rather than shared administrative credentials.

2.3.3 Segmentation and authorised-flow design

Segmentation is effective only when it reflects authorised application flows. Creating separate subnets while permitting unrestricted east-west communication produces administrative separation without meaningful security isolation. The proposed three-tier workload therefore requires a traffic-flow matrix defining source, destination, protocol, port and business justification. For example, a public-facing web component may communicate with an application service on a defined port, while direct web-to-database communication remains denied.

Micro-segmentation strengthens this principle by reducing the scope of implicit trust around individual workloads or services. However, a purely IP-based implementation has limits in dynamic cloud-native systems because addresses can change and network identity may not equal application identity. The project should acknowledge this limitation and position workload identity and service-mesh enforcement as a future extension aligned with SP 800-207A rather than overclaiming that subnet controls alone constitute complete Zero Trust.

Figure 2.2. Integrated cross-cloud trust, connectivity, segmentation and observability model.

Section synthesis and implication for the artefact. The literature indicates that Zero Trust, federation and segmentation should be evaluated as one access-control system. The artefact therefore operationalises Zero Trust through federated human identity, temporary roles, explicit network flows, restricted routes, workload isolation and telemetry. This synthesis informs Objective 4 and prevents repetitive treatment of VPN, segmentation and Zero Trust as unrelated topics.

2.4 Infrastructure as Code, Configuration Security and Reproducibility

2.4.1 Terraform as a cross-provider engineering mechanism

Infrastructure as Code converts architectural intent into machine-readable configuration that can be version-controlled, reviewed and redeployed. Terraform is appropriate because one workflow can manage Google Cloud and AWS providers while preserving provider-specific resource definitions. Its principal contribution is therefore reproducibility of process and configuration history, not automatic cross-cloud portability.

The security literature provides an important corrective to overly optimistic IaC claims. Verdet et al. (2025) empirically examined 812 open-source Terraform projects across GCP, AWS and Google Cloud using static analysis. They found that access-control policies were among the more widely adopted practices, while encryption-at-rest controls were comparatively neglected. The study demonstrates that codification can reproduce insecure defaults just as efficiently as secure ones.

Consequently, the project's Terraform implementation should be evaluated as part of the security surface. State files require protection; secrets should not be embedded in source; modules should expose constrained variables; plans should be reviewed; and static configuration assessment should complement runtime cloud-posture tools. A reproducible artefact is academically stronger when another practitioner can inspect not only the architecture diagram but also the code that created it.

2.4.2 Cross-provider dependency and drift

Cross-cloud IaC introduces dependencies that single-provider examples often avoid. VPN configuration requires values generated on both sides; route and gateway parameters must remain compatible; identity integration may depend on tenant-level prerequisites; and provider APIs evolve independently. Terraform can model these dependencies, but it does not remove them. The implementation should therefore use modular configuration, explicit outputs, version constraints and documented manual prerequisites.

Configuration drift also matters. Portal-based changes can cause the deployed environment to diverge from the reviewed code. The project should use Terraform plan output and cloud configuration checks to identify divergence where practical. This connects maintainability directly to security: an architecture that cannot be reproduced or reconciled is difficult to audit.

Section synthesis and implication for the artefact. Recent empirical evidence shows that IaC improves repeatability without guaranteeing secure configuration. The artefact will therefore combine modular Terraform with version control, state protection, secret separation, plan review and security assessment. This informs Objective 3 and provides an evidence trail linking design intent to deployed resources.

2.5 Cross-Cloud Observability, Threat Detection and Resilience

2.5.1 The observability problem

Google Cloud and AWS produce rich security telemetry, but they do so through different services, schemas and administrative boundaries. Cloud Audit Logs, VPC Flow Logs, Cloud Monitoring and Security Command Center can provide GCP evidence; AWS CloudTrail, Amazon CloudWatch, Log Analytics and AWS Security Hub and GuardDuty provide related capabilities on AWS. The challenge is not the absence of logs but correlation across platforms.

Fragmented monitoring can hide attack sequences. A suspicious sign-in may be visible in the identity platform while subsequent network activity appears in another cloud. The architecture should therefore define which events must be collected and how they can be correlated, even if the proof of concept does not deploy a full production SIEM. This is also a governance issue: accountability under the NDPA depends partly on an organisation's ability to reconstruct access and security events.

2.5.2 Security assessment and evidence

Cloud posture tools such as Prowler and ScoutSuite can identify configuration weaknesses, but their findings should not be treated as a binary certification. Scanner coverage differs by service and rule set, and false positives or context-dependent findings require interpretation. The evaluation should therefore triangulate scanner results with direct access tests, route inspection, identity evidence and logging verification.

This evidence-based approach is stronger than claiming that the presence of Security Command Center, AWS Security Hub and GuardDuty or encryption automatically proves security. Controls must be configured, observable and tested against intended outcomes.

2.5.3 Resilience must be measured

Multi-cloud resilience is frequently asserted at an architectural level, but the project requires empirical confirmation. A redundant topology can still fail to recover if route withdrawal is slow, health detection is misconfigured or a single dependency remains outside the redundant path. The proposed test therefore disables a selected tunnel or route while continuous probes measure interruption and stable restoration.

The resulting Recovery Time Objective measurement should be reported with the test conditions, selected regions, gateway tiers and workload size. This avoids presenting a lab result as a universal cloud benchmark. The same principle applies to throughput and latency.

Section synthesis and implication for the artefact. Observability and resilience are properties to be evidenced, not inferred from service selection. The artefact will combine provider-native telemetry with correlated review, posture scanning, direct access tests and controlled failure experiments. This informs Objective 5 and supports Nigerian requirements for accountability and defensible operational controls.

2.6 Nigerian Regulatory and Operational Context Integrated with Architecture

2.6.1 Nigeria Data Protection Act 2023

The NDPA 2023 is relevant wherever the representative architecture is used as a model for workloads processing personal data. Its significance to this project lies less in prescribing specific cloud products than in requiring accountable processing and appropriate safeguards. Architecture decisions should therefore make data flows, access paths, encryption controls and logging responsibilities visible.

Cross-border processing is particularly relevant to public cloud because cloud regions and supporting services may exist outside Nigeria. A technically successful AWS–GCP connection can therefore create governance exposure if data classification and transfer conditions are ignored. The artefact should document what information is intended to cross the inter-cloud link and use synthetic data during testing.

2.6.2 National Cloud Policy 2025

The National Cloud Policy 2025 strengthens the policy basis for cloud adoption while emphasising secure, sovereign and properly classified use. For the literature review, its value is analytical: it changes the criteria by which an architecture is judged. A design should not be evaluated only for connectivity and throughput, but also for visibility of data location, security ownership, recoverability and auditability.

This reinforces the project's decision to maintain explicit route boundaries, federated identity, centralised or correlated logs and documented IaC. These mechanisms do not by themselves establish legal compliance, but they provide technical evidence that can support organisational compliance processes.

2.6.3 Context-specific design implications

Table 2.1. Nigerian-context requirements translated into technical design and evaluation evidence.

Section synthesis and implication for the artefact. Nigeria does not require a unique cryptographic or routing protocol, but local law and policy change the governance criteria applied to established Google Cloud and AWS

technologies. The artefact therefore treats data-flow visibility, accountability, region awareness, logging and recoverability as design requirements rather than post-implementation documentation.

2.7 Critical Comparison of Existing Multi-Cloud and AWS–GCP Approaches

A stronger research gap requires comparison of what previous work actually contributes. The literature can be grouped into five categories: systematic multi-cloud research; Zero Trust research; IaC empirical research; provider-specific reference architectures; and integrated implementation studies. Each category contributes part of the required solution, but the degree of implementation and evaluation differs.

Table 2.2. Critical comparison of representative literature and the proposed artefact.

The comparison demonstrates that the gap is not an absence of multi-cloud concepts or security technologies. The literature is mature enough to explain why multi-cloud is adopted, how Zero Trust should shape access, and why IaC improves reproducibility. What is less common is the integration of these domains into one provider-specific artefact followed by independent testing of security policy, network performance and recovery behaviour.

This distinction is important academically. A claim that 'no AWS–GCP implementations exist' would be too broad and difficult to defend. The more precise claim is that existing sources do not sufficiently combine the project's selected dimensions—native AWS–GCP networking, encrypted connectivity, federated identity, least-privilege segmentation, correlated observability, Terraform reproducibility, measurable failure testing and Nigerian governance considerations—in one independently evaluated proof of concept.

Section synthesis and implication for the artefact. Previous work provides strong partial solutions but limited integrated evidence. The proposed artefact is therefore justified by integration and evaluation rather than by novelty of individual technologies.

2.8 Critical Analysis of Architectural Alternatives

Table 2.3. Critical comparison of cross-cloud connectivity and control approaches.

The native hub-and-spoke approach provides the strongest fit for a professional master's artefact because it exposes provider-specific security decisions, supports Terraform automation and can be evaluated without a proprietary abstraction layer. It is not necessarily the best production design for every enterprise. Dedicated connectivity may be preferable for predictable performance, while a service mesh may provide stronger application-level identity for containerised systems. The project therefore treats its selected architecture as a justified proof-of-concept choice rather than a universal optimum.

2.9 Consolidated Research Gap

Figure 2.4. Consolidated research-gap model.

The literature reveals a practical integration-and-evaluation gap. Contemporary research explains multi-cloud architecture, Zero Trust, identity, IaC and cloud security in substantial depth, while GCP and Microsoft provide accurate implementation guidance for their own platforms. However, these knowledge streams are frequently separated by research domain or provider boundary.

First, there is an integration gap: networking, identity, segmentation, observability and IaC are often examined independently even though security failure can arise from their interaction. Second, there is an implementation gap: conceptual frameworks and vendor reference patterns do not necessarily provide a reproducible cross-provider artefact. Third, there is an evaluation gap: resilience is often inferred from redundancy, while security is inferred from control presence rather than demonstrated through denied-flow tests, posture assessment and failure measurement. Fourth, there is a contextual gap: Nigerian data-protection and cloud-policy requirements are rarely translated into provider-specific implementation evidence.

The research gap is therefore defined as the limited availability of a documented, reproducible and independently evaluated AWS–GCP reference implementation that integrates native networking, encrypted connectivity, federated identity, least-privilege segmentation, correlated observability and Infrastructure as Code, and then evaluates the resulting system against measurable security, performance and resilience criteria while explicitly considering Nigerian enterprise governance requirements.

The proposed project addresses this gap at proof-of-concept scale. Its contribution is not a new cryptographic protocol or cloud service; it is the systematic integration, implementation and empirical evaluation of established technologies against a clearly defined enterprise problem.

2.10 Literature-to-Artefact Traceability

Figure 2.3. Literature-to-artefact traceability and evaluation loop.

Table 2.4. Traceability from literature findings to artefact requirements and objectives.

2.11 Chapter Synthesis

This chapter has moved from descriptive technology review to a critical argument for the proposed artefact. Recent multi-cloud literature shows that provider diversity can improve flexibility and resilience options, but only when heterogeneity, routing, identity and operational complexity are deliberately controlled. Zero Trust literature shows

that encrypted connectivity cannot be equated with trusted access; identity, least privilege, segmentation and telemetry must operate together. Recent empirical IaC research demonstrates that Terraform improves reproducibility while still permitting systematic misconfiguration, justifying code review and security assessment.

The Nigerian context strengthens these requirements. The NDPA 2023 and National Cloud Policy 2025 make data-flow visibility, accountability, secure processing, region awareness and auditability relevant to architecture design. The project therefore incorporates these concerns into networking, identity, logging and evaluation rather than confining them to a standalone compliance discussion.

The critical comparison of existing work shows that the project's contribution lies in integration and evaluation. The proposed artefact combines native Google Cloud and AWS hub-and-spoke networking, route-based IPsec/BGP connectivity, federated identity, explicit segmentation, correlated observability and Terraform. It will then be evaluated using cloud-posture assessment, authorised and unauthorised traffic tests, latency and throughput measurement, and controlled failure with recovery-time measurement. These literature-derived requirements provide the basis for the methodology and detailed system design presented in Chapter Three.

Chapter Three
METHODOLOGY AND SYSTEM DESIGN

METHODOLOGY AND SYSTEM DESIGN

3.0 Introduction

This chapter presents the methodology and detailed system design for the secure GCP- AWS multi-cloud architecture proposed in this project. It translates the problems, concepts and gaps identified in Chapters One and Two into a structured technical solution that can be implemented and evaluated.

The chapter adopts Design Science Research Methodology because the project is concerned with the creation and evaluation of a practical information technology artefact. The artefact comprises the Google Cloud and AWS network environments, encrypted inter-cloud connectivity, Terraform configuration, identity controls, security policies, logging arrangements, a representative inventory-management workload and the supporting testing procedures.

The MIVA Open University Professional Master’s Project Guide requires the methodology and system-design chapter to explain the selected project methodology, present the requirements analysis, describe the system architecture, include appropriate design models and identify the tools and technologies used. The guide also emphasises that the project must be applied, solution-driven and capable of demonstrating technical implementation and evaluation.

Accordingly, this chapter covers the following areas:
justification and application of Design Science Research Methodology;
functional and non-functional requirements;
actors and system use cases;
assumptions and design constraints;
logical and physical architecture;
Google Cloud and AWS network designs;
IP addressing and routing;
VPN and BGP design;
three-tier workload and database design;
identity federation;
Zero Trust and defence-in-depth security;
logging and monitoring;
Terraform module and state design;
system interaction and deployment diagrams;
evaluation traceability; and
design risks and mitigation measures.

The design remains consistent with the approved proposal. It uses Google Cloud Platform and Amazon Web Services (AWS), a hub-and-spoke network model, Terraform Infrastructure as Code, encrypted VPN connectivity, segmentation, identity controls, centralised observability and measurable security, performance and resilience evaluation.

3.1 Project Methodology

3.1.1 Meaning of methodology in this project

A project methodology provides the structured process through which the practical problem is investigated, the proposed solution is developed and the effectiveness of the solution is assessed. In a conventional experimental study, the principal output may be an explanation of a relationship between variables. In this Professional Master's Project, the principal output is a working technical artefact that addresses an identified enterprise problem.

The methodology must therefore support both creation and evaluation. It must provide a logical basis for moving from the problem of fragmented AWS-GCP security to a designed architecture, an implemented environment and defensible test results.

Design Science Research Methodology was selected as the primary methodology. An iterative implementation approach is embedded within its design-and-development stage. The iterative approach is not treated as a separate research methodology; it is a practical delivery mechanism through which the artefact will be built, tested and refined.

3.1.2 Design Science Research Methodology

Figure 3.1. Design Science Research Methodology applied to the AWS-GCP artefact.

Design Science Research is concerned with the construction and evaluation of artefacts intended to solve identified problems. In information-systems research, an artefact may be a model, method, architecture, software system or implemented technical environment.

Pe ers et al. developed Design Science Research Methodology as a structured process consisting of problem identification and motivation, definition of objectives for a solution, design and development, demonstration, evaluation and communication. Their methodology is intended to support research in disciplines where knowledge is advanced through the creation of useful artefacts.

The six DSRM activities are applied to this project as follows:

DSRM activity Application to the project Principal output

Problem Examine the security, networking, Problem statement and identification and identity and monitoring problems project justification motivation arising from ad hoc AWS-GCP interconnection

Define objectives of Establish requirements for secure Project objectives and a solution connectivity, segmentation, measurable automation, visibility, performance and requirements resilience

Design and Design Google Cloud and AWS networks, VPN, Architecture diagrams, development routing, security controls, workload configuration design and Terraform modules and source code

Demonstration Deploy the environment and run a Functioning technical representative inventory-management workload artefact across both clouds

Evaluation Conduct security scans, access tests, Quantitative and latency tests, throughput tests and qualitative results tunnel-failure tests

Communication Document the design, Chapters One to Six and implementation, results, professional defence materials contribution and limitations

3.1.3 Why DSRM was selected

DSRM was selected because the problem addressed by this project is practical, technical and solution-oriented. The project does not merely ask whether multi-cloud environments contain security risks. Existing research has already established that identity inconsistency, misconfiguration, fragmented visibility and insecure interfaces are significant cloud concerns. The professional need is to design and test a solution that addresses those risks within a specific AWS-GCP context.

The methodology is suitable for six principal reasons.

Artefact orientation

The required output is an implemented architecture rather than a purely conceptual argument. DSRM explicitly recognises the creation of artefacts as a legitimate research contribution. The artefact in this project includes cloud resources, Terraform code, security controls, network policies, monitoring configurations and test procedures.

Problem-to-solution traceability

DSRM requires the designed artefact to respond to an identified problem. Each major component of the architecture is linked to a problem established in Chapter One. For example, fragmented networking is addressed through a controlled hub-and-spoke design;

inconsistent access is addressed through federation and least privilege; weak visibility is addressed through logging and centralised analysis.

Iterative refinement

Cloud architectures rarely operate perfectly after the first deployment. VPN parameters may be incompatible, route propagation may be incomplete, or a security rule may block legitimate communication. DSRM permits the artefact

to be refined through evaluation feedback rather than treating implementation difficulties as failures outside the research process.

Measurable evaluation

The methodology requires the artefact to be evaluated against its objectives. The project will therefore measure whether unauthorised traffic is blocked, whether the VPN is encrypted, what inter-cloud latency and throughput are achieved, and how the system behaves during a controlled failure.

Professional relevance

The MIVA project framework requires a practice-oriented and solution-driven project. DSRM is consistent with this orientation because it combines professional problem solving with systematic documentation and evaluation.

Knowledge contribution

Although the project uses existing technologies, its contribution arises from their structured integration and evaluation. The project produces reusable knowledge about AWS-GCP interconnection, Terraform dependencies, identity integration, segmentation and measured operational behaviour.

3.1.4 DSRM process for the project

Figure 3.1: Application of DSRM to the project The feedback path between evaluation and design reflects the iterative nature of the project. A failed test will result in documented investigation, modification of the design or code, redeployment and retesting.

3.1.5 Iterative development cycles

The design-and-development activity will be divided into manageable technical cycles:

Iteration Primary objective Verification gate

1 Establish Terraform providers, naming, state Successful authentication and and base resource groups validated plans

2 Create Google Cloud VPC, subnets and Transit Internal GCP routing and Gateway components access tests

3 Create AWS VPC, subnets and VPN Internal AWS routing and Gateway components access tests

4 Establish AWS-GCP IPsec and BGP Tunnel status and learned- connectivity route validation

5 Deploy three-tier workload Approved application flows succeed

6 Apply segmentation and Zero Trust controls Prohibited paths fail

7 Configure federation and privileged access Federated sign-in and role validation

8 Enable logging and security services Logs and findings visible

9 Conduct security and performance Metrics collected and findings evaluation remediated

10 Conduct resilience testing and final Recovery time measured validation

This staged approach reduces troubleshooting complexity. The project will not attempt to deploy every component before checking foundational connectivity.

3.2 Requirements Analysis

3.2.1 Requirements elicitation

The requirements were derived from four sources:

1. the problem statement and approved proposal;
2. NIST Zero Trust principles;
3. GCP and Microsoft platform capabilities;
4. cloud-security and Nigerian governance considerations.

The requirements distinguish what the system must do from the quality and control standards under which it must operate. Functional requirements describe expected capabilities. Non-functional requirements describe characteristics such as security, availability, performance, auditability and maintainability.

Requirements are assigned unique identifiers so that they can be traced to design components and test cases in Chapter Five.

3.3 Functional Requirements

3.3.1 Functional requirement catalogue

ID Functional requirement Acceptance basis

FR-01 The system shall provision an isolated GCP virtual network with segmented web, application, database and management subnets Required Google Cloud resources exist and are associated with the correct route tables

FR- The system shall provision an isolated AWS Required AWS resources exist 02 virtual network with application, shared- and are correctly associated services, monitoring and gateway subnets

FR- The system shall establish encrypted site-to- VPN tunnels report operational 03 site connectivity between Google Cloud and AWS status and private traffic traverses the tunnel

FR- The system shall exchange authorised network Expected BGP routes appear on 04 prefixes between Google Cloud and AWS both sides

FR- The system shall restrict inter-tier and inter- Approved traffic succeeds and 05 cloud communication according to a defined prohibited traffic fails traffic-flow matrix FR- The system shall deploy a representative three- Web, application and database 06 tier enterprise workload components function according to design

FR- The system shall provide secure administrative Administration occurs through 07 access to cloud resources approved identities and protected paths

FR- The system shall support federated workforce A federated user receives 08 identity between Microsoft Entra ID and GCP temporary GCP access mapped where feasible to an approved role

FR- The system shall avoid long-lived credentials Source and configuration review 09 within application code confirms no embedded cloud keys

FR- The system shall collect GCP management, Selected Cloud Audit Logs, 10 security and network events Cloud Monitoring, flow and security records are generated

FR- The system shall collect AWS management, Selected AWS logs and security 11 identity, security and network events findings are generated

FR- The system shall support security-posture Scan reports are generated for 12 assessment using Prowler and ScoutSuite applicable environments

FR- The system shall support inter-cloud latency Ping and iperf3 results are 13 and throughput measurement collected

FR- The system shall support a controlled VPN or Tunnel failure can be simulated 14 route-failure test without unauthorised resource impact

FR- The system shall measure the recovery time Failure and recovery 15 following a controlled failure timestamps are recorded

FR- The system shall provision principal Terraform state corresponds to 16 infrastructure through Terraform deployed resources

FR- The system shall expose required outputs for Outputs include approved 17 configuration and validation resource IDs, addresses and gateway information FR- The system shall support controlled destruction Terraform destroy or 18 of chargeable lab resources documented teardown removes designated resources

3.3.2 Network connectivity requirements

The network must support private communication between authorised Google Cloud and AWS workloads. Public-IP communication will not be used for normal application-tier interaction.

The VPN will be route based and will support BGP where technically feasible. GCP Site-to- Site VPN can terminate on a Network Connectivity Center, and GCP supports dynamic or static routing for such VPN attachments. Google Cloud HA VPN normally provides two tunnels between the GCP gateway and the external gateway.

AWS Site-to-Site VPN will be configured in an active-active, BGP-enabled pattern where service tier and project budget permit. Microsoft's current guidance includes a specific BGP-enabled AWS-GCP connection pattern using an active-active AWS Site-to-Site VPN and GCP site-to-site VPN connections.

3.3.3 Workload communication requirements

The expected traffic paths are defined before security rules are created.

Flow Source Destination Protocol/port Purpose Default ID outcome

TF- Authorised GCP web tier HTTPS/443 Access Permit 01 user application

TF- GCP web tier GCP TCP/8080 Forward Permit 02 application application tier request

TF- GCP AWS service HTTPS/443 Cross-cloud Permit 03 application tier service request tier

TF- GCP GCP TCP/5432 Database Permit 04 application database tier query tier TF- AWS Approved HTTPS/443 Log or Permit 05 monitoring Google Cloud log telemetry service endpoint transfer

TF- Management Approved SSH/22 Controlled Permit from 06 host Linux administration management workloads source only

TF- Web tier Database tier Any Direct Deny 07 database access

TF- AWS general GCP Any Unapproved Deny 08 subnet database tier cross-cloud access
TF- Internet GCP Any Direct public Deny 09 database tier access
TF- Internet AWS private Any Direct public Deny 10 service tier access
TF- Unauthorised GCP or AWS Any Privileged Deny 11 administrator management management plane
TF- Scanner/test Approved test ICMP/TCP test Controlled Permit during 12 host endpoints ports evaluation test window
The traffic-flow matrix will be converted into GCP security-group rules, AWS Network VPC Firewall Rule rules, route controls and host firewall policies.

3.4 Non-functional Requirements

3.4.1 Security requirements

ID Security requirement Target
NFR- Inter-cloud traffic shall be encrypted in IPsec tunnel established with S01 transit approved cryptographic settings NFR- Network access shall follow deny-by- Only documented flows are allowed S02 default principles
NFR- Privileged access shall use named No routine shared administrator S03 identities credentials
NFR- Workforce access shall use multifactor MFA enforced for privileged identities S04 authentication where supported
NFR- Permissions shall follow least privilege Roles contain only required actions S05
NFR- Long-lived credentials shall not be No secrets committed to repository S06 stored in Terraform or source files
NFR- Security-relevant actions shall be Management and sign-in activity S07 logged recorded
NFR- Workload tiers shall be segmented Direct web-to-database access S08 blocked
NFR- Resources shall not be publicly Public exposure limited to approved S09 accessible unless required entry points
NFR- High and critical scanner findings shall Zero unresolved high or critical S10 be remediated or formally explained findings at final assessment, where feasible

3.4.2 Performance requirements

The approved proposal establishes an inter-cloud latency target below 100 milliseconds for representative cross-cloud calls. This is retained as the project acceptance threshold, although the measured outcome will depend on region selection, Internet conditions, virtual-machine size and encryption overhead.

ID Performance requirement Target
NFR- Inter-cloud round-trip latency shall Average below 100 ms under normal P01 remain suitable for the lab workload test conditions
NFR- Throughput shall be measured and iperf3 result compared with P02 reported observed link and VM limitations NFR- Security controls shall not prevent All approved flows complete P03 authorised transactions successfully
NFR- Test measurements shall be repeatable Minimum of three test runs or P04 defined sampling period
NFR- Performance results shall include Region, VM size, time, tunnel status P05 context and test settings documented

3.4.3 Availability and resilience requirements

ID Availability requirement Target
NFR- The VPN design shall include more than Redundant tunnel configuration A01 one tunnel or path
NFR- A single tunnel failure shall not cause Recovery through alternate route or A02 permanent loss of connectivity documented intervention
NFR- Failure and recovery shall be RTO calculated from test timestamps A03 measurable
NFR- The design shall avoid unnecessary Gateway and route dependencies A04 single points of failure documented
NFR- Recovery procedures shall be Runbook created and tested A05 repeatable

3.4.4 Maintainability requirements

ID Maintainability requirement Target

NFR- Infrastructure shall be modular Separate logical Terraform M01 modules
NFR- Environment-specific values shall be CIDRs, regions and names M02 parameterised defined as variables
NFR- Resource naming shall be consistent Naming convention applied to M03 both providers
NFR- Code shall be version controlled Git history maintained M04 NFR- Terraform provider versions shall be Required version blocks declared M05 constrained
NFR- Changes shall be visible before Terraform plan reviewed M06 deployment
NFR- Manual configuration shall be minimised Manual-step register maintained M07 and documented

Terraform uses state to map configuration objects to deployed infrastructure and to determine required changes. State protection is therefore part of maintainability and security rather than an incidental technical matter.

3.4.5 Scalability requirements

The project is implemented at lab scale, but the design must allow extension.

ID Scalability requirement Target
NFR- New GCP workload VPCs shall be Network Connectivity Center pattern supports SC01 attachable to the transit design additional attachments
NFR- New AWS spokes shall be supportable Hub VNet or Virtual WAN pattern SC02 documented
NFR- Address spaces shall not overlap CIDR allocation plan reserves SC03 expansion space
NFR- Security rules shall be role or tier based Avoid one-o broad exceptions SC04
NFR- Terraform modules shall support Inputs and outputs designed for SC05 repeated instances reuse

3.4.6 Auditability and compliance requirements

ID Auditability requirement Target
NFR- Infrastructure changes shall be Version history and cloud activity C01 traceable logs retained
NFR- Data flows shall be documented Tra ic-flow matrix and diagrams C02 maintained NFR- Security controls shall map to NIST and CSA control mapping C03 recognised frameworks
NFR- Sensitive data shall not be used Synthetic data only C04
NFR- Cross-cloud data movement shall be Logs and route documentation C05 visible available

3.5 Use-Case Analysis

3.5.1 System actors

The architecture contains both human and non-human actors.

Actor Description
Enterprise user Accesses the web application through the approved endpoint
Cloud administrator Deploys and manages Google Cloud and AWS resources
Security administrator Reviews policies, findings and logs
Network administrator Manages CIDRs, VPN, BGP and routing
Terraform operator Executes approved infrastructure plans
Microsoft Entra ID Authenticates workforce identities
Google Cloud IAM Identity Maps federated identities to GCP permissions Center/IAM
Web service Receives user requests
Application service Processes business logic
Database service Stores application records
Monitoring platform Collects operational and security events
Security scanner Assesses configuration posture
Test engineer Performs latency, throughput, access and failure tests

3.5.2 Use-case diagram

Figure 3.2: Principal system use cases

3.5.3 Detailed use cases

Use Case UC-01: Access the enterprise application
Element Description

Primary actor Enterprise user
Preconditions Application endpoint is available; web service is running
Trigger User submits an HTTPS request
Main flow Request reaches approved public entry point; web tier processes request; application tier performs business logic; database is queried; response is returned
Alternative flow Cross-cloud supporting service is unavailable and application returns a controlled error Security TLS required; direct database access prohibited conditions
Postcondition Transaction and relevant logs are recorded
Use Case UC-02: Federated administrator authentication
Element Description
Primary actor Cloud administrator
Supporting Microsoft Entra ID and Google Cloud Workforce Identity Federation systems
Preconditions User is assigned to an authorised group; federation is configured
Trigger Administrator attempts Google Cloud console access
Main flow GCP redirects user to Entra; Entra authenticates user and applies MFA; signed federation assertion is returned; GCP maps user to an approved permission set; temporary session is issued
Exception User lacks approved group membership or authentication fails
Postcondition Successful or failed sign-in is logged
Use Case UC-03: Deploy infrastructure
Element Description
Primary actor Terraform operator
Preconditions Credentials are available through secure authentication; code has been reviewed
Trigger Operator executes Terraform workflow
Main flow Initialise providers; validate files; format code; generate plan; review plan; apply approved changes; record outputs
Exception Provider error, dependency failure or policy violation
Postcondition State reflects successfully managed resources Use Case UC-04: Establish VPN connectivity
Element Description
Primary actor Network administrator
Preconditions Google Cloud and AWS base networks exist; CIDRs do not overlap
Trigger VPN components are deployed
Main flow Public gateway addresses are exchanged; customer/local network gateways are created; IPsec connections are established; BGP sessions form; routes are learned
Exception IKE mismatch, ASN conflict, prefix error or route-policy failure
Postcondition Approved private addresses can communicate
Use Case UC-05: Run security assessment
Element Description
Primary actor Security administrator
Preconditions Scanner has read-only audit permissions
Trigger Approved scan begins
Main flow Tool enumerates cloud configurations; checks are evaluated; findings are generated; findings are classified and remediated
Exception API access is insufficient or service is unsupported
Postcondition Dated report and remediation record exist
Use Case UC-06: Perform failover test
Element Description
Primary actor Test engineer
Preconditions Redundant connectivity is operational; maintenance window approved
Trigger Selected tunnel or route is disabled Main flow Continuous test traffic is initiated; failure is introduced; route withdrawal or tunnel transition occurs; connectivity resumes; timestamps are captured
Exception Connectivity does not recover
Postcondition RTO and failure observations are recorded

3.6 Design Assumptions and Constraints

3.6.1 Assumptions

The design assumes that:
valid Google Cloud and AWS accounts are available;
the selected regions support the required services;

the Google Cloud and AWS CIDR blocks do not overlap;
Internet connectivity is available for VPN endpoints;
Terraform providers can authenticate to both platforms;
synthetic application data is sufficient for evaluation;
the selected virtual-machine sizes support the test tools;
the user has authority to deploy and test the lab resources;
service quotas are adequate; and
tests are performed only against project-owned resources.

3.6.2 Constraints

The principal constraints are cost, project duration, cloud-account limitations and service availability. Premium services such as Cloud Interconnect and ExpressRoute are excluded from the implementation. AWS Virtual WAN may also be represented as an enterprise alternative if the cost of a fully managed secured hub is disproportionate to the lab.

The architecture must remain sufficiently realistic for enterprise analysis while small enough to deploy, test and remove within the available academic resources.

3.7 Overall System Architecture

3.7.1 Architectural style

The proposed system uses a federated hub-and-spoke architecture. Google Cloud and AWS each retain their own native administrative boundaries, but they are joined through encrypted, policy-controlled connectivity.

The architecture consists of five logical planes:

1. access plane, through which users and administrators reach approved services;
2. application plane, containing web, application and data components;
3. connectivity plane, containing transit, gateways, VPN tunnels and routing;
4. security and management plane, containing identity, logging, threat detection and policy controls;
5. automation plane, containing Terraform, source control, state and deployment workflows.

3.7.2 Complete architecture diagram

Figure 3.3: Secure AWS-GCP multi-cloud architecture

3.7.3 Architectural rationale

GCP hosts the principal three-tier workload because the approved proposal identifies GCP as a primary application-hosting environment. AWS hosts a supporting application, monitoring component or recovery service so that the project generates meaningful inter- cloud communication.

The database remains private and is not directly exposed across the inter-cloud boundary. This design reduces the number of systems that can communicate with the data tier. The AWS component communicates with the GCP application tier through an approved service interface rather than querying the database directly.

Google Cloud Network Connectivity Center provides the GCP-side transit hub. GCP describes Network Connectivity Center as a managed regional hub that interconnects VPCs and external networks and supports VPN attachments.

AWS Site-to-Site VPN terminates the AWS side of the connection. The lab uses a hub VNet to contain the gateway, shared services and controlled routing. AWS Virtual WAN remains an alternative for a larger multi-region enterprise environment.

3.8 Network Design

Figure 3.2. Proposed network addressing and routing model.

3.8.1 Addressing principles

The address plan follows four principles:
Google Cloud and AWS address spaces must not overlap;
each workload tier receives a distinct subnet;
address space must remain available for future expansion;
CIDR allocations must be easy to identify during troubleshooting.
The proposed laboratory allocation is:

Environment	Network	CIDR
GCP enterprise VPC (gcp_miva_final_project)	Main GCP workload	10.181.0.0/16 network
GCP public ingress subnet A	Public entry components	10.181.10.0/24 GCP public ingress subnet B
Redundant public entry	10.181.11.0/24 components	
GCP web subnet A	Web workloads	10.181.20.0/24

GCP web subnet B Redundant web workloads 10.181.21.0/24
GCP application subnet A Application workloads 10.181.30.0/24
GCP application subnet B Redundant application 10.181.31.0/24 workloads
GCP database subnet A Database workloads 10.181.40.0/24
GCP database subnet B Database redundancy 10.181.41.0/24
GCP management subnet Test and controlled 10.181.50.0/24 administration
Google Cloud Network Connectivity Center Transit attachment 10.181.60.0/28 and attachment subnets 10.181.60.16/28
AWS hub VPC (aws_miva_final_project) AWS transit and shared 10.121.0.0/16 services
AWS GatewaySubnet VPN Gateway 10.121.0.0/27
AWS application subnet Supporting application 10.121.10.0/24
AWS monitoring subnet Monitoring and collection 10.121.20.0/24
AWS management subnet Bastion or management 10.121.30.0/24 workload
AWS reserved subnet Future services 10.121.40.0/24
These addresses are project design values and may be adjusted during implementation where provider requirements or existing account networks require different ranges.

3.8.2 Network diagram

Figure 3.4: Detailed AWS-GCP network design

3.8.3 GCP route design

The GCP route design separates local application routing from inter-cloud routing.
The application subnet route tables will include:
local VPC route for 10.181.0.0/16;
route for 10.121.0.0/16 through Network Connectivity Center;
optional default route through an approved NAT path where package updates are
required.
The database route tables will not advertise general AWS reachability unless a specific requirement is established. This prevents AWS networks from gaining a direct path to the database tier.
The Network Connectivity Center route table will associate the Google Cloud VPC attachment and VPN attachment. Propagation will be limited so that only the intended Google Cloud VPC prefix is advertised to AWS.

3.8.4 AWS route design

The AWS VPC uses system routes for local communication and learned BGP routes for GCP prefixes. User-defined routes may be added where the project needs explicit next-hop control. The AWS supporting application subnet will receive access to the approved GCP application prefixes. It will not receive unrestricted access to the GCP database subnet unless a documented test specifically requires it.
The AWS management subnet will be isolated from application traffic and will use separate identity and NSG controls.

3.8.5 Routing preference

BGP is preferred because it supports dynamic route exchange and better failure detection than manually maintained static routes. GCP supports both static and dynamic routes for HA VPN attachments, and AWS Site-to-Site VPN supports BGP for site-to-site connections.
Static routes may be used as a documented fallback where BGP compatibility or lab restrictions prevent the desired configuration. Static and BGP routes will not be mixed for identical prefixes without a clear routing rationale because route preference may produce unexpected path selection. GCP documentation notes that static and BGP-advertised routes can have different preference behaviour.

3.9 VPN and BGP Design

3.9.1 VPN topology

The proposed implementation uses an active-active AWS Site-to-Site VPN and GCP Site-to- Site VPN connectivity.
Microsoft documents an AWS-GCP BGP pattern in which an active-active AWS gateway is connected to GCP through multiple site-to-site connections.
The design goals are:
more than one available encrypted path;
dynamic route exchange;
controlled route advertisement;
observable tunnel status;

measurable failover behaviour.

3.9.2 Autonomous System Numbers

Private Autonomous System Numbers will be used for the lab. The proposed values are: Component ASN
Google Cloud Network Connectivity Center 64512
AWS Site-to-Site VPN 65515 or supported configured private ASN
Reserved future appliance 64520
The actual AWS ASN will be verified against the selected gateway configuration. Duplicate ASNs will not be used on directly peered systems.

3.9.3 BGP advertisements

GCP will advertise only the approved GCP application or VPC prefix. AWS will advertise only the AWS VPC prefix required for the supporting service.
The initial project design advertises:
GCP to AWS: 10.181.0.0/16, subject to Network Connectivity Center route control;
AWS to GCP: 10.121.0.0/16.
Security groups and security groups remain responsible for fine-grained control. Route advertisement establishes reachability but not permission.

3.9.4 IPsec parameters

The VPN will use compatible Google Cloud and AWS settings. The preferred design is:
Parameter Proposed setting
IKE version IKEv2
Tunnel mode Route based
Authentication Strong pre-shared key for lab or certificate where supported
Encryption AES-256 or compatible approved algorithm
Integrity SHA-256 or compatible approved algorithm
Di ie-Hellman group Compatible high-strength group
Perfect forward secrecy Enabled where compatible
Routing BGP
Tunnel redundancy Multiple tunnels Key storage Protected secret store; excluded from source control
The final values will be selected from the mutually supported Google Cloud and AWS settings during implementation. The exact cryptographic parameter list will be recorded in Chapter Four without exposing secret material.

3.9.5 Tunnel-monitoring design

Tunnel status will be observed from both provider portals or command-line interfaces. The monitoring process will record:
tunnel state;
BGP peer state;
learned prefixes;
bytes transmitted and received;
route-table state;
failure and recovery timestamps.
Continuous ping or application probes will be used during failover testing.

3.10 Workload and Database Design

3.10.1 Need for a database component

A database is applicable because the representative system is a three-tier enterprise workload. The database provides persistent storage and enables the project to demonstrate that the data tier can be protected from direct public and cross-cloud access.
The database is not the research focus. It serves as a controlled workload component for evaluating segmentation and authorised application flows.

3.10.2 Database technology

PostgreSQL is proposed because it is widely available on Linux and can be deployed either as a managed cloud database or on a private virtual machine.
For strict cost control, the initial implementation may use PostgreSQL on a private Ubuntu instance. Where account credits permit, Cloud SQL for PostgreSQL is preferable because it reduces operating-system management and supports managed backup and encryption capabilities.

The final implementation choice will be documented in Chapter Four.

3.10.3 Logical data model

The database will contain synthetic records only. A simple transaction-processing model is sufficient.

Figure 3.5: Simplified entity-relationship model

No real names, account numbers, payment credentials or other personal data will be used. Email values, where required for demonstration, will be synthetic or hashed.

3.10.4 Database security design

The database will implement the following controls:
no public IP address;
placement in a private database subnet;
inbound TCP/5432 permitted only from the application-tier VPC firewall rule;
operating-system access restricted to the management path;
database authentication using a dedicated application account;
no administrator password embedded in Terraform source;
storage encryption where supported;
backup or snapshot configuration appropriate to the lab;
query and system logging sufficient for test evidence;
synthetic test data only.

3.10.5 Database availability

The design includes database subnets in two availability zones so that the architecture can support a managed multi-zone database or future secondary instance. The lab may use only one active instance because of cost.

Database failover is not the principal resilience test in this project. The primary resilience test concerns inter-cloud connectivity. Database redundancy will therefore be described accurately as a design capability where it is not fully deployed.

3.11 Security Model

3.11.1 Security design principles

The security model is based on the following principles:
verify explicitly;
use least privilege;
assume breach;
deny by default;
segment workload tiers;
encrypt traffic in transit;
prefer short-lived credentials;
centralise or correlate security visibility;
automate repeatable controls;
test controls rather than assuming effectiveness.

NIST SP 800-207 separates the Zero Trust policy-decision function into a policy engine and policy administrator and identifies policy-enforcement points that control access to resources.

The project does not claim to implement one central Zero Trust product. Instead, NIST logical functions are mapped across Google Cloud and AWS services.

3.11.2 Zero Trust component mapping

NIST logical function	Project implementation
Policy engine	Entra Conditional Access, Google Cloud IAM policy evaluation, AWS IAM, defined network policy
Policy administrator	Entra federation configuration, Google Cloud Workforce Identity Federation, cloud control planes
Policy enforcement point	GCP VPC firewall rules, AWS Security Groups, VPN gateways, application authentication, host firewalls
Identity information	Microsoft Entra ID identities and groups
Asset information	Google Cloud and AWS resource inventories
Threat intelligence or security context	Security Command Center, AWS Security Hub and GuardDuty and scanner findings
Continuous diagnostics	Cloud logs, flow logs, metrics and security scans
Resource	Web, application, database, management and monitoring components

3.11.3 Security trust zones

The system is divided into security zones:

Zone Trust position Permitted interaction
Public access zone Untrusted HTTPS to approved entry point only
GCP web zone Low trust Application-tier service port only GCP application Controlled Database port and approved zone AWS service
GCP database zone Highly restricted Application tier only
GCP management Privileged Controlled administration zone
Inter-cloud transit Untrusted transport with Authorised routed prefixes only zone encrypted tunnel
AWS service zone Controlled Approved application and monitoring flows
AWS management Privileged Controlled administration zone
Logging zone Restricted Log ingestion and authorised review

3.11.4 Defence-in-depth layers

Figure 3.6: Security-control layers

3.11.5 Security-group design

The proposed GCP VPC firewall rules are:

Security Inbound rules Outbound rules group
sg-public- HTTPS/443 from Internet HTTPS or application port to web entry tier
sg-web Application traffic from public TCP/8080 to application tier entry
sg-application TCP/8080 from web tier TCP/5432 to database; HTTPS/443 to approved AWS prefix
sg-database TCP/5432 from application Restricted responses and required VPC firewall rule updates
sg- SSH/22 from approved Approved private subnets management administrator source or bastion sg-test Controlled test traffic from Defined test destinations authorised sources

Security-group references will be used instead of broad CIDR rules for GCP-internal flows where possible.

3.11.6 AWS Security Group design

NSG Inbound rules Outbound rules
nsg-aws-service HTTPS/443 from approved GCP Approved return and application subnet monitoring traffic
nsg-monitoring Log ingestion from approved sources Management and alert endpoints
nsg-management Administration from approved source AWS private service endpoints
nsg-gateway- Provider-required gateway traffic Provider-required tunnel support traffic

AWS Security Group rules will be assigned explicit priorities and documented descriptions.

3.11.7 Administrative access

Routine direct SSH from arbitrary public addresses is prohibited. The preferred administrative paths are:

GCP Systems Manager Session Manager where supported;
AWS Bastion or a restricted management VM where a ordable;
short-lived, source-restricted SSH access during controlled testing;
federated management-plane access through Entra and Google Cloud Workforce Identity Federation.

Break-glass accounts will be minimised, protected with strong authentication and excluded from normal operation.

3.11.8 Secrets management

The following values are treated as secrets:

VPN pre-shared keys;
database passwords;
API tokens;
private keys;
cloud service-principal credentials.

They will not be committed to Git. Suitable storage options include GCP Secrets Manager, GCP Systems Manager Parameter Store, AWS KMS and Secrets Manager, environment variables injected at execution time or protected CI secret storage.

Terraform sensitive variables reduce accidental display but do not remove values from state automatically. State protection is therefore required.

3.12 Identity Federation Design

3.12.1 Federation approach

Microsoft Entra ID is selected as the workforce identity provider. Google Cloud Workforce Identity Federation will act as the GCP access broker where technically feasible.

The proposed federation uses:

SAML 2.0 for authentication;
SCIM for user and group provisioning where supported;
GCP permission sets for role mapping;
Entra groups for access assignment;
multifactor authentication for privileged roles;
temporary Google Cloud federated sessions rather than long-lived IAM user keys.

3.12.2 Identity groups

Entra group	GCP/AWS IAM role	Purpose
MC-Cloud-Admins	Controlled administrator role	Infrastructure administration
MC-Network-Admins	Network-specific role	VPN and routing management
MC-Security-Auditors	Read-only security role	Logs, scans and findings
MC-Terraform-Deployers	Deployment role	Approved Terraform changes
MC-Application-Ops	Restricted operations role	Workload support
MC-Project-Viewers	Read-only role	Demonstration and review

3.12.3 Identity sequence

Figure 3.7: Federated GCP administrator sign-in sequence

3.12.4 Federation fallback

Where Workforce Identity Federation integration is unavailable within the account structure, direct SAML federation to Google Cloud IAM roles may be used. The fallback must still provide temporary credentials and role-based access.

Separate permanent IAM users will be retained only where required for emergency access or a documented technical limitation.

3.13 Logging and Monitoring Design

3.13.1 Monitoring objectives

The monitoring design must support:

detection of unauthorised access;
investigation of network events;
verification of VPN and BGP health;
reconstruction of administrative changes;
collection of test evidence;
notification of critical findings.

3.13.2 Google Cloud log sources

The GCP design includes:

Cloud Audit Logs for management events;
Cloud Monitoring Logs and metrics;
VPC Flow Logs for selected VPCs or interfaces;
Network Connectivity Center Flow Logs where supported and a ordable;
Security Command Center findings;
GCP Config where enabled;
operating-system logs from test instances.

3.13.3 AWS log sources

The AWS design includes:

AWS Activity Log;
Entra sign-in and audit logs subject to licence availability;
AWS Site-to-Site VPN diagnostics;
Network Watcher diagnostics;
Log Analytics workspace;
Microsoft AWS Security Hub and GuardDuty recommendations and alerts;
operating-system logs from Amazon EC2 instances.

3.13.4 Centralised view

The preferred laboratory design uses AWS Log Analytics as the central multi-cloud view for selected telemetry, while native GCP records remain retained in GCP. This does not require every Google Cloud log to be exported. A defined subset, such as security findings, application logs or test metrics, will be forwarded or manually correlated.

The final implementation will distinguish:
native source of truth;
central summary;
retention period;
alerting mechanism;
evidence used in Chapter Five.

3.13.5 Monitoring flow

Figure 3.8: Multi-cloud logging and monitoring flow

3.14 Terraform Design

Figure 3.3. Staged Terraform deployment and evidence workflow.

3.14.1 Terraform architectural approach

Terraform is used as the principal Infrastructure-as-Code tool because it can manage Google Cloud and AWS through separate providers within a coordinated configuration.

HashiCorp describes Terraform as an Infrastructure-as-Code tool for building, changing and versioning infrastructure. Terraform modules group resources that are managed together, which supports the separation of the project into logical components.

The codebase will follow a modular structure rather than placing all resources in one file.

3.14.2 Proposed directory structure

```
secure-aws-aws-multicloud/
│
├── README.md
├── versions.tf
├── providers.tf
├── variables.tf
├── outputs.tf
├── locals.tf
├── terraform.tfvars.example
│
├── environments/
│   └── lab/
│       ├── backend.tf
│       ├── main.tf
│       ├── variables.tf
│       └── lab.tfvars
│
├── modules/
│   ├── aws-network/
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   ├── outputs.tf
│   │   └── security.tf
│   ├── aws-transit/
│   │   ├── main.tf
│   │   ├── routes.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   │
│   ├── aws-network/
│   │   ├── main.tf
│   │   └── subnets.tf
```

```
│   │   ├── nsg.tf
│   │   └── outputs.tf
│   │
│   ├── aws-vpn/
│   │   ├── gateway.tf
│   │   ├── connections.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   │
│   ├── aws-vpn/
│   │   ├── customer-gateways.tf
│   │   ├── vpn-connections.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   │   ├── workload/
│   │   ├── aws-compute.tf
│   │   ├── aws-compute.tf
│   │   ├── database.tf
│   │   ├── cloud-init/
│   │   └── outputs.tf
│   │
│   ├── identity/
│   │   ├── aws-roles.tf
│   │   ├── policies.tf
│   │   └── outputs.tf
│   │
│   └── monitoring/
│       ├── aws-logging.tf
│       ├── aws-monitor.tf
│       ├── alerts.tf
│       └── outputs.tf
│
├── scripts/
│   ├── validate-connectivity.sh
│   ├── run-iperf-tests.sh
│   ├── collect-routes.sh
│   └── failover-test.sh
│
├── tests/ │   ├── network-tests.md
│   ├── security-tests.md
│   └── expected-results.md
│
└── docs/
    ├── diagrams/
    ├── screenshots/
    └── runbooks/
```

3.14.3 Provider configuration

The root configuration will declare compatible provider versions.

```
required_version = ">= 1.9.0"
aws = {
source = "hashicorp/aws"
version = "~> 6.0"
}
awsrm = {
source = "hashicorp/awsrm"
version = "~> 4.0"
}
random = {
source = "hashicorp/random" version = "~> 3.0"
```

```
}
}
}
default_tags {
tags = local.common_tags
}
}
provider "awsrm" {
features {}
subscription_id = var.aws_subscription_id
}
```

Provider versions will be adjusted to versions validated during implementation. Version constraints prevent unreviewed upgrades from changing behaviour unexpectedly.

3.14.4 Common variables

```
description = "Project identifier used in resource names."
type        = string
default     = "secure-multicloud"
description = "Deployment environment."
type        = string
default     = "lab"
validation {
condition   = contains(["lab", "test"], var.environment)
error_message = "Environment must be lab or test."
}
}
description = "GCP region for the project."
type        = string
}
description = "AWS Region for the project."
type        = string
}
description = "Google Cloud VPC address space for gcp_miva_final_project."
type        = string
default     = "10.181.0.0/16"
description = "AWS VPC address space for aws_miva_final_project."
type        = string
default     = "10.121.0.0/16"
}
```

3.14.5 Tagging and naming

```
locals {
common_tags = {
Project     = var.project_name
Environment = var.environment
ManagedBy = "Terraform"
Purpose     = "MIVA-MIT-Professional-Project"
DataClass = "Synthetic"
}
}
```

Every supported resource will be tagged or labelled to show ownership, environment, purpose and management method.

3.14.6 Module relationships

Figure 3.9: Terraform module dependency design The VPN modules have cross-provider dependencies. AWS public gateway addresses are required to define GCP customer-gateway resources, while GCP tunnel and BGP information is required to complete AWS local-network gateway and connection settings.

The deployment may therefore occur in controlled stages:

1. deploy Google Cloud and AWS base networks;
2. deploy AWS Site-to-Site VPN and capture public IPs;

4. retrieve GCP tunnel configuration;
5. complete AWS Customer Gateways and connections;
6. verify BGP;
7. deploy workloads and security controls.
This staged pattern is documented rather than hidden by forced Terraform dependencies.

3.14.7 Terraform state design

Terraform state maps configuration to real infrastructure and is used to determine proposed changes.

The preferred remote-state design uses an encrypted AWS Storage Account or an encrypted Google Cloud S3 backend with locking. One provider will host the project's authoritative state backend to avoid circular bootstrapping.

The state design will include:
encryption at rest;
restricted identity access;
versioning;
locking where supported;
no public access;
backup or version recovery;
separation of bootstrap state from main environment state.
A small bootstrap configuration may create the remote-state resources before the main project is deployed.

3.14.8 Sensitive values

VPN pre-shared keys and database passwords will be passed through secure variables or retrieved from secrets-management services.

description = "Pre-shared key for the lab VPN."
type = string
sensitive = true
}
description = "Database application password."
type = string sensitive = true
}
Marking a value as sensitive protects normal display but does not guarantee that it will not appear in state. Access to state must therefore remain restricted.

3.14.9 Terraform execution workflow

Figure 3.10: Terraform deployment workflow

3.14.10 Terraform error handling

Terraform may partially deploy resources where one provider operation fails after another succeeds. The recovery process will include:
examining the error and state;
checking provider consoles for actual resource condition;
avoiding manual deletion unless necessary;
importing manually retained resources where appropriate;
applying the smallest controlled correction;
recording manual interventions.

3.15 Application Interaction Design

3.15.1 Normal application request

Figure 3.11: Enterprise application sequence

3.15.2 Unauthorised access sequence

Figure 3.12: Denied direct database-access attempt This sequence demonstrates that the VPN does not create permission. Reachability and authorisation remain separate.

3.16 Deployment Design

3.16.1 Physical deployment view

Figure 3.13: Deployment diagram

3.16.2 Deployment environments

The project uses two logical environments:
bootstrap environment, containing remote state and deployment prerequisites;
lab environment, containing the implemented architecture and test workload.
A separate production environment is not created because the project is a proof of concept. The Terraform structure nevertheless allows future environment separation.

3.17 Security-Control Traceability

3.17.1 Requirement-to-control mapping

Requirement Design control Verification method
FR-03 encrypted Google Cloud HA VPN and Tunnel configuration and connectivity AWS Site-to-Site VPN packet-path evidence
FR-05 restricted Security groups, security groups, route Positive and negative communication policies and host firewall connectivity tests
FR-08 federated identity Entra ID and Google Cloud IAM Federated sign-in test Identity Center
FR-09 no long-lived IAM roles, managed identity Source-code and instance application credentials or protected secret store review
FR-10 Google Cloud logging Cloud Audit Logs, Cloud Monitoring and Generate and retrieve VPC Flow Logs known events
FR-11 AWS logging Activity Log, Log Analytics Generate and retrieve and Defender known events
FR-12 security assessment Prowler and ScoutSuite Dated reports
FR-13 performance ping and iperf3 Captured test results measurement
FR-14 failure simulation Multi-tunnel VPN and Failover test controlled disablement
NFR-S02 deny by default Explicit allow rules only Port and path tests
NFR-S03 named privileged Federation and individual Authentication-log review identities accounts
NFR-M01 modular Terraform modules Repository inspection infrastructure
NFR-C02 documented Flow matrix and diagrams Design review data flows

3.17.2 NIST and CSA alignment

Security objective NIST Zero Trust relationship Relevant CSA control area
Explicit Verify before access Identity and access authentication management
Least privilege Per-session and resource-focused IAM and application security access
Network Limit implicit network trust Infrastructure and segmentation virtualisation security
Encryption Secure communication regardless Cryptography and key of location management
Continuous Collect security and asset Logging and monitoring visibility information
Controlled change Maintain known asset posture Change and configuration management
Recovery Maintain operational capability Business continuity and resilience
This mapping is used for design justification. It does not represent a formal compliance certification.

3.18 Performance-Test Design

3.18.1 Latency measurement

Latency will be measured between approved Google Cloud and AWS test endpoints.
The procedure will record:
source and destination addresses;
cloud regions;
date and time;
tunnel state;
packet size;
number of samples;
minimum, average, maximum and variability;
packet loss.
The project target is average round-trip latency below 100 milliseconds for the representative workload.

3.18.2 Throughput measurement

iperf3 will operate in server mode on one approved endpoint and client mode on the other. TCP throughput will be measured over defined intervals.
Tests may include:

GCP to AWS;
AWS to GCP;
one stream;
multiple parallel streams;
repeated runs.
Throughput will be interpreted against virtual-machine size, gateway capacity, Internet variability and encryption overhead.

3.18.3 Test isolation

Performance tests will be conducted during a controlled period. Security rules will permit only the required iperf3 port between the two test endpoints and will be removed after testing.

3.19 Resilience-Test Design

3.19.1 Recovery Time Objective

Recovery Time Objective is defined for this project as the maximum acceptable interval between loss of the active inter-cloud path and restoration of successful application or network communication.
The measured recovery time is:
where:
TfailureT_{failure}Tfailure is the first timestamp at which the monitored traffic fails following the controlled event; and
TrestoredT_{restored}Trestored is the first timestamp at which successful communication resumes and remains stable.

3.19.2 Failure scenarios

Scenario	Failure introduced	Expected behaviour
RS-01	Disable one VPN tunnel	Tra ic changes to available tunnel
RS-02	Remove or suppress one BGP	Alternate learned route is selected where route available
RS-03	Stop AWS supporting service	Network remains available but application health check fails
RS-04	Apply incorrect security rule in	Access fails and log evidence is generated controlled test
RS-05	Restore correct Terraform	Service returns to approved state configuration

3.19.3 Failover sequence

Figure 3.14: Tunnel-failure and recovery sequence

3.20 Security-Test Design

3.20.1 Automated configuration assessment

Prowler and ScoutSuite will be granted read-only audit permissions where possible.
The assessment process will:
1. record tool version;
2. record provider and region scope;
3. execute the scan;
4. preserve the original report;
5. classify findings;
6. remove false positives with documented reasons;
7. remediate valid findings;
8. rerun the scan;
9. compare initial and final results.

3.20.2 Manual security validation

Manual tests will include:
attempted direct access from Internet to database;
attempted web-tier access to database;
attempted AWS general-subnet access to GCP database;
approved GCP application-to-AWS HTTPS call;
failed authentication by an unassigned user;
review of privileged actions in audit logs;
verification that repository files contain no secrets;
route-table inspection;
security-group and NSG review;

verification of VPN encryption settings.

3.20.3 Severity treatment

High and critical findings will be remediated unless they are demonstrated to be false positives or unavoidable lab limitations. Medium findings will be prioritised according to exploitability and project scope. Low findings will be documented and addressed where practical.

3.21 Design Validation

3.21.1 Pre-implementation validation

Before deployment, the design will be reviewed for:
CIDR overlap;
missing routes;
inappropriate public exposure;
unsupported provider combinations;
Terraform dependency cycles;
excessive IAM permissions;
secret leakage;
cost-generating resources;
service quotas;
failure-test safety.

3.21.2 Terraform validation

The minimum code validation sequence is:
terraform fmt -check -recursive
terraform validate
The plan will be inspected before application. A successful plan does not prove security, but it confirms syntactic and dependency-level consistency.

3.21.3 Post-deployment validation

After deployment, validation will confirm:
expected resources exist;
VPN tunnels are active;
BGP peers are established;
routes are present;
approved traffic succeeds;
prohibited traffic fails;
logs are generated;
scanners can access required metadata;
no unintended public addresses exist;
Terraform plan shows no unexplained drift.

3.22 Ethical and Professional Considerations

The implementation will use only project-owned Google Cloud and AWS resources. No testing will target third-party systems.

Synthetic data will be used throughout. Screenshots and code listings will exclude subscription identifiers, account numbers, public IP addresses where unnecessary, pre- shared keys, passwords, private keys and security tokens.

Open-source tools will be used according to their licences. Academic and technical sources will be cited. Test findings will be reported accurately, including failed targets and implementation limitations.

The environment will be removed or reduced after testing to avoid unnecessary cost and exposure.

3.23 Risks and Design Mitigations

Risk	Likelihood	Impact	Design mitigation
CIDR overlap	Low	High	Reserve distinct 10.181.0.0/16 and 10.121.0.0/16 ranges
VPN parameter mismatch	Medium	High	Use compatible IKE/IPsec settings and staged validation
BGP session failure	Medium	High	Verify ASN, APIPA/BGP addresses, routes and peer state
Route leakage	Medium	High	Advertise only approved prefixes and review route tables
Excessive security rules	Medium	High	Derive rules from traffic-flow matrix
Terraform secret exposure	Medium	High	Protected variables, secret stores and restricted state

Partial multi-cloud	Medium	Medium	Modular state, staged apply and deployment documented recovery
Cloud cost overrun	Medium	Medium	Budgets, small resources and teardown plan
Identity-federation	Medium	Medium	Implement network first and retain delay SAML role fallback
Logging cost or licence	Medium	Medium	Select essential logs and document restriction excluded premium data
Scanner false positives	High	Low to	Manual validation and documented medium disposition
Failover test causes	Low	Medium	Maintenance window, rollback plan prolonged outage and continuous observation
Public exposure during	Low	High	Deny by default and automated testing post-deployment checks

3.24 Evaluation Matrix

The design is considered successful only when the project objectives can be evaluated through evidence.

Project objective	Design output	Chapter Five evidence
Review architecture and	Literature and gap analysis	Critical comparison security gaps
Design secure hub-and-	Diagrams, routes, flow matrix	Design review against spoke architecture and controls deployed environment
Implement with Terraform	Modular multi-provider code	Plan, apply output and resource evidence
Configure Zero Trust	Federation, segmentation,	Access tests and audit controls logging and least privilege evidence
Evaluate security,	Test plans and success criteria	Scan reports, metrics and performance and failover results resilience

3.25 Chapter Summary

This chapter presented the methodology and complete system design for the secure GCP- AWS multi-cloud architecture. Design Science Research Methodology was selected because the project creates and evaluates a practical information-technology artefact. Its six activities-problem identification, definition of solution objectives, design and development, demonstration, evaluation and communication-correspond directly with the project lifecycle and the MIVA Professional Master's Project structure.

The requirements analysis defined functional requirements for Google Cloud and AWS network provisioning, encrypted inter-cloud connectivity, BGP route exchange, workload segmentation, identity federation, monitoring, automated security assessment, performance measurement, failover testing and Terraform-based deployment. Non-functional requirements addressed security, performance, availability, maintainability, scalability and auditability.

The architecture uses an GCP hub-and-spoke environment built around a segmented VPC and Network Connectivity Center. AWS provides a hub VNet, an redundant Site-to-Site VPN, a supporting application or recovery service and central monitoring. The environments are connected through route-based IPsec tunnels with BGP. Google Cloud HA VPN supports Network Connectivity Center attachments and multiple encrypted tunnels, while AWS provides documented support for active-active BGP-enabled GCP connectivity.

A representative inventory-management workload was designed with web, application and database functions. The database remains private and accepts traffic only from the application tier. An AWS supporting service generates meaningful cross-cloud traffic without exposing the database directly.

The security model applies Zero Trust, least privilege, deny-by-default segmentation, defence in depth, encrypted connectivity, federated identity, protected secrets and continuous monitoring. NIST policy functions are mapped to Microsoft Entra ID, Google Cloud Workforce Identity Federation, IAM policies, VPC firewall rules, Network VPC Firewall Rules and cloud logging services.

The Terraform design separates Google Cloud networking, AWS networking, VPN, workload, identity and monitoring into reusable modules. It includes protected state, provider version constraints, parameterised environments, secure handling of sensitive values and staged deployment for cross-cloud VPN dependencies.

The chapter also presented use-case, architecture, network, entity-relationship, identity- sequence, application-sequence, deployment, logging and failover diagrams in Mermaid syntax. Finally, it established security, performance and resilience test designs that will be used to evaluate the implemented artefact.

Transition to Chapter Four

Chapter Four Will Describe The Actual Implementation Of The Architecture Designed In This

chapter. It will document the development environment, Google Cloud and AWS account preparation, Terraform installation and authentication, remote-state configuration, VPC and VNet deployment, Network Connectivity Center and VPN Gateway configuration, IPsec and BGP establishment, workload deployment, database setup, identity federation, security hardening, monitoring, troubleshooting and implementation challenges. It will also include relevant Terraform code listings, gcloud CLI commands, AWS CLI commands, Linux commands and implementation screenshots.

Chapter Four
SYSTEM IMPLEMENTATION

SYSTEM IMPLEMENTATION

4.0 Introduction

This chapter describes the implementation of the secure AWS-GCP multi-cloud architecture designed in Chapter Three. It explains how the laboratory environment is prepared, how the Google Cloud and AWS resources are provisioned with Terraform, how encrypted inter-cloud connectivity is established, and how identity, encryption, monitoring, logging and threat-detection controls are configured.

The implementation follows the iterative design-and-development activity of Design Science Research Methodology. The artefact is built in controlled stages so that each layer can be verified before dependent components are introduced. Base cloud authentication and Terraform state are established first. The Google Cloud and AWS networks are then deployed independently, followed by the VPN, routing, workloads, security controls, monitoring and identity federation. This order reduces troubleshooting complexity and supports repeatable validation.

The implementation presented in this chapter is a reproducible reference implementation. Cloud account identifiers, tenant identifiers, public IP addresses, secret values, generated VPN parameters and screenshots must come from the actual project environment. They are therefore represented by variables or labelled placeholders rather than fabricated values. Similarly, no claim is made that a test succeeded until the corresponding evidence is produced from the deployed environment in Chapter Five.

The core infrastructure is automated with Terraform. A small number of Microsoft Entra ID and Google Cloud Workforce Identity Federation federation steps remain administrative because they depend on tenant-specific enterprise-application metadata, generated certificates, SCIM tokens and account-assignment decisions. These steps are documented separately and must be evidenced with actual screenshots.

4.1 Implementation Environment

4.1.1 Hardware and administrative workstation

The infrastructure can be deployed from a Windows or Linux administrative workstation. The minimum practical workstation configuration is:

Component Recommended configuration Processor 64-bit, four cores or more
Memory 8 GB minimum; 16 GB preferred
Storage At least 20 GB free space
Operating system Windows 11, Ubuntu 22.04/24.04 or comparable Linux distribution
Internet access Stable broadband connection
Shell PowerShell 7, Bash or Windows Terminal
Source control Git
Code editor Visual Studio Code or equivalent
Cloud tools Terraform, gcloud CLI and AWS CLI
Test tools SSH client, curl, ping, traceroute and iperf3

Terraform is the principal provisioning tool. Terraform uses provider plugins to interact with remote platforms and requires each configuration to declare its provider sources and version constraints.

4.1.2 Cloud subscriptions and permissions

The implementation requires:

an Google Cloud project in which the operator can create VPC, Compute Engine, Network Connectivity Center, VPN,

IAM, KMS, Cloud Monitoring, Cloud Audit Logs, Security Command Center and related resources;

a Amazon Web Services (AWS) subscription in which the operator can create resource groups,

VNets, VPN Gateways, virtual machines, AWS KMS and Secrets Manager, Amazon CloudWatch and AWS Security Hub and GuardDuty resources;

a Microsoft Entra tenant for workforce groups and GCP federation;

authority to enable Google Cloud Workforce Identity Federation where the account structure permits it;

permission to register AWS resource providers where necessary.

The initial provisioning identity may require broad permissions because it creates foundational resources. Routine administration after deployment should use more restricted roles.

4.1.3 Selected cloud regions

The Google Cloud and AWS Regions should be selected to balance service availability, cost and expected latency. The regions must support all required gateway and security services. They should also be geographically close enough to make the project's target of less than 100 milliseconds average inter-cloud round-trip latency technically reasonable.

The exact regions used must be reported in the final project. This chapter uses variables rather than assuming specific regions.

Screenshot placeholder 4.1

Insert screenshot showing the selected GCP Region and AWS Region. The screenshot should display only the region names and should conceal account numbers, subscription IDs and user details.

4.1.4 Software installation

Terraform installation verification

terraform version

Expected output:

Terraform v1.x.x

on windows_amd64 or linux_amd64

The installed version should be recorded in the implementation evidence. HashiCorp describes Terraform as an Infrastructure-as-Code tool for building, changing and versioning cloud and on-premises infrastructure.

gcloud CLI verification

aws --version

aws sts get-caller-identity

The second command confirms the Google Cloud project and identity that will execute the deployment. Its screenshot must conceal the account number and principal ARN where these are not required for assessment.

AWS CLI verification

az version

az login

az account show --output table The AWS command confirms the selected subscription and tenant.

Git verification

git --version

Repository creation

mkdir secure-aws-aws-multicloud

cd secure-aws-aws-multicloud

git init

git branch -M main

A .gitignore file is created immediately to prevent accidental commitment of Terraform state, plans and secrets.

```
# Terraform local files
.terraform/
*.tfstate
*.tfstate.*
*.tfplan
crash.log
crash.*.log
# Variable files that may contain secrets
*.auto.tfvars
*.auto.tfvars.json
terraform.tfvars
# Keys and certificates
*.pem *.key
*.pfx
*.p12
# Local environment files
.env
.env.*
.vscode/
.idea/
# Generated reports
reports/
screenshots/private/
```

The example variable file may be committed, but it must contain placeholders only.

4.2 Implementation Structure

4.2.1 Repository layout

The implementation is organised as follows:

```
secure-aws-aws-multicloud/
│
├── README.md
├── .gitignore
│
├── bootstrap/
│   ├── versions.tf
│   ├── providers.tf │ ├── variables.tf
│   ├── main.tf
│   └── outputs.tf
│
├── infrastructure/
│   ├── versions.tf
│   ├── backend.tf.example
│   ├── providers.tf
│   ├── variables.tf
│   ├── locals.tf
│   ├── data.tf
│   ├── aws-network.tf
│   ├── aws-security.tf
│   ├── aws-workload.tf
│   ├── aws-monitoring.tf
│   ├── aws-network.tf
│   ├── aws-security.tf
│   ├── aws-workload.tf
│   ├── aws-monitoring.tf
│   ├── vpn.tf
│   ├── identity.tf
│   ├── outputs.tf
│   ├── terraform.tfvars.example
│   │
│   └── templates/ │   ├── web-cloud-init.yaml
│       ├── app-cloud-init.yaml
│       ├── db-cloud-init.yaml
│       └── aws-service-cloud-init.yaml
│
├── scripts/
│   ├── verify-aws.sh
│   ├── verify-aws.sh
│   ├── verify-vpn.sh
│   ├── run-latency-test.sh
│   ├── run-iperf-test.sh
│   └── collect-evidence.sh
│
└── docs/
    ├── screenshots/
    ├── diagrams/
    └── runbooks/
```

The code is divided by responsibility rather than being placed in one very large file. Terraform treats all .tf files in the same directory as one configuration, so this division improves readability without changing execution behaviour.

4.3 Remote-State Bootstrap

4.3.1 Purpose of remote state

Terraform state maps declared resources to the actual infrastructure. It can contain identifiers and sensitive attributes and must therefore be protected. The implementation stores state in an AWS Storage Account. The awsrm backend authenticates to the storage account data plane and stores the state as a blob in a named container.

State resources are created separately because Terraform cannot use a backend that does not yet exist.

4.3.2 Bootstrap provider declaration

File: bootstrap/versions.tf

```
required_version = ">= 1.9.0"
awsrm = {
source = "hashicorp/awsrm"
version = "~> 4.0"
}
random = {
source = "hashicorp/random"
version = "~> 3.6"
}
}
}
```

The required_version statement prevents execution with an unsupported Terraform version. The provider constraints permit compatible updates while preventing uncontrolled major- version changes.

File: bootstrap/providers.tf

```
provider "awsrm" { features {}
subscription_id = var.aws_subscription_id
}
```

The AWS account ID is supplied as a variable rather than hard-coded into the source repository.

4.3.3 Bootstrap variables

File: bootstrap/variables.tf

```
description = "AWS account used to host Terraform state."
type        = string
sensitive = true
}
description = "AWS Region for the state resources."
type        = string
}
description = "Short project identifier."
type        = string
default      = "mivamc"
}
description = "Environment name." type          = string
default       = "lab"
}
```

4.3.4 Bootstrap resources

File: bootstrap/main.tf

```
resource "random_string" "storage_su ix" {
length = 8
upper = false
special = false
}
location = var.aws_location
tags = {
Project        = var.project_name
Environment = var.environment
Purpose        = "Terraform remote state"
ManagedBy = "Terraform"
}
```

```
}
name = substr(
lower("st${var.project_name}${var.environment}${random_string.storage_su ix.result}"), 0,
)
resource_group_name           = awsrm_resource_group.terraform_state.name
location              = awsrm_resource_group.terraform_state.location
account_tier                  = "Standard"
account_replication_type            = "LRS"
min_tls_version               = "TLS1_2"
public_network_access_enabled = true
allow_nested_items_to_be_public = false
blob_properties {
versioning_enabled = true
delete_retention_policy {
days = 14
}
container_delete_retention_policy {
days = 14
}
}
tags = {
Project      = var.project_name Environment = var.environment
Purpose      = "Terraform remote state"
ManagedBy = "Terraform"
Classification = "Confidential"
}
}
name        = "tfstate"
storage_account_id = awsrm_storage_account.terraform_state.id
container_access_type = "private"
}
```

The storage account prohibits public container access, enforces TLS 1.2 or later, and enables versioning and retention. Public network access is temporarily enabled to allow the administrative workstation to reach the state backend. A production design should restrict it using private endpoints or approved IP rules.

4.3.5 Bootstrap outputs

File: bootstrap/outputs.tf

```
value = awsrm_resource_group.terraform_state.name
}
value = awsrm_storage_account.terraform_state.name
}
}
```

Bootstrap execution

```
cd bootstrap
terraform fmt -check
terraform validate
```

The output values are copied into infrastructure/backend.tf.

Screenshot placeholder 4.2

Insert screenshot showing the successful Terraform bootstrap apply summary. The screenshot should show the number of resources added but conceal the subscription ID and storage-account access details.

4.4 Main Terraform Configuration

4.4.1 Backend configuration

File: infrastructure/backend.tf

```
backend "awsrm" {
resource_group_name = "REPLACE_WITH_BOOTSTRAP_RESOURCE_GROUP"
storage_account_name = "REPLACE_WITH_BOOTSTRAP_STORAGE_ACCOUNT"
container_name   = "tfstate"
key          = "secure-aws-aws-multicloud-lab.tfstate"
```

use_awsad_auth = true
} }
The AWS identity executing Terraform requires storage data-plane access to the container.

4.4.2 Provider requirements

File: infrastructure/versions.tf
required_version = ">= 1.9.0"
aws = {
source = "hashicorp/aws"
version = "~> 6.0"
}
awsrm = {
source = "hashicorp/awsrm"
version = "~> 4.0"
}
awsad = {
source = "hashicorp/awsad"
version = "~> 3.0"
}
random = { source = "hashicorp/random"
version = "~> 3.6"
}
tls = {
source = "hashicorp/tls"
version = "~> 4.0"
}
cloudinit = {
source = "hashicorp/cloudinit"
version = "~> 2.3"
}
local = {
source = "hashicorp/local"
version = "~> 2.5"
}
}
}
Provider version constraints are part of reproducibility. The final report should record the exact versions selected in .terraform.lock.hcl.

4.4.3 Provider configuration

File: infrastructure/providers.tf
tags = local.common_tags
}
}
provider "awsrm" {
features {
key_vault {
purge_soft_delete_on_destroy = false
recover_soft_deleted_key_vaults = true
}
}
subscription_id = var.aws_subscription_id
tenant_id = var.aws_tenant_id
}
provider "awsad" {
tenant_id = var.aws_tenant_id
}
The code assumes authentication has been completed through supported environment credentials or interactive CLI sessions. No permanent GCP access key or AWS client secret is placed in the code.

4.4.4 Input variables

File: infrastructure/variables.tf

```
type          = string
default       = "secure-multicloud"
}
description = "Deployment environment."
type          = string
default       = "lab"
validation {
condition     = contains(["lab", "test"], var.environment)
error_message = "The environment must be lab or test."
}
}
description = "GCP deployment region."
type          = string
}
description = "AWS deployment region."
type          = string
description = "AWS account identifier."
type        = string
sensitive = true
}
description = "Microsoft Entra tenant identifier."
type        = string
sensitive = true
}
description = "Google Cloud VPC address space for gcp_miva_final_project."
type        = string
default     = "10.181.0.0/16"
}
description = "AWS VPC address space for aws_miva_final_project."
type        = string
default     = "10.121.0.0/16"
}
description = "Approved public administrator IP in CIDR notation." type         = string
sensitive = true
}
description = "Compute Engine instance size."
type        = string
default     = "t3.micro"
}
description = "Amazon EC2 instance size."
type        = string
default     = "Standard_B1s"
}
description = "Application database name."
type        = string
default     = "enterprise_lab"
}
description = "Database application account."
type        = string
default     = "app_user" }
description = "Database password supplied securely at runtime."
type          = string
sensitive = true
validation {
condition     = length(var.database_password) >= 16
error_message = "The database password must contain at least 16 characters."
}
```

```
}
description = "Pre-shared key for the first selected AWS-GCP tunnel."
type          = string
sensitive = true
validation {
condition      = length(var.vpn_shared_key_1) >= 16
error_message = "VPN keys must contain at least 16 characters."
}
}
description = "Pre-shared key for the second selected AWS-GCP tunnel." type          = string
sensitive = true
validation {
condition       = length(var.vpn_shared_key_2) >= 16
error_message = "VPN keys must contain at least 16 characters."
}
}
description = "Private ASN for Google Cloud Network Connectivity Center."
type          = number
default       = 64512
}
description = "Private ASN for AWS Site-to-Site VPN."
type          = number
default       = 65515
}
description = "Enable Google Cloud Security Command Center."
type          = bool
default       = true
description = "Enable selected Microsoft AWS Security Hub and GuardDuty plans."
type         = bool
default      = true
}
```

Secrets are supplied through environment variables or an uncommitted .tfvars file. Marking variables as sensitive reduces accidental display, but state access must still be restricted.

4.4.5 Local values

File: infrastructure/locals.tf

```
locals {
common_tags = {
Project        = var.project_name
Environment = var.environment
ManagedBy = "Terraform"
Purpose        = "MIVA MIT Professional Project"
DataClass      = "Synthetic"
Architecture = "AWS-GCP-MultiCloud"
}
public_a = {
cidr = "10.181.10.0/24"
public = true az     =0
}
public_b = {
cidr = "10.181.11.0/24"
public = true
az     =1
}
web_a = {
cidr = "10.181.20.0/24"
public = false
az     =0
}
web_b = {
```

```
cidr = "10.181.21.0/24"
public = false
az     =1
}
app_a = {
cidr = "10.181.30.0/24"
public = false
az     =0
}
app_b = {
cidr = "10.181.31.0/24"
public = false
az     =1 }
db_a = {
cidr = "10.181.40.0/24"
public = false
az     =0
}
db_b = {
cidr = "10.181.41.0/24"
public = false
az     =1
}
management = {
cidr = "10.181.50.0/24"
public = false
az     =0
}
transit_a = {
cidr = "10.181.60.0/28"
public = false
az     =0
}
transit_b = {
cidr = "10.181.60.16/28"
public = false
az     =1
} }
}
```

The local map avoids repeated subnet declarations and ensures consistent CIDR assignments.

4.4.6 Data sources

File: infrastructure/data.tf

```
state = "available"
}
most_recent = true
owners          = ["099720109477"]
filter {
name = "name"
values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
}
filter {
name = "virtualization-type"
values = ["hvm"]
}
}
data "awsrm_client_config" "current" {} data "awsad_client_config" "current" {}
```

The GCP AMI is selected dynamically from Canonical’s publisher account rather than hard- coding an image ID that may di er by region.

4.5 GCP Network Implementation

4.5.1 Google Cloud VPC

File: infrastructure/aws-network.tf
enable_dns_support = true
enable_dns_hostnames = true
tags = {
Name = "${local.name_prefix}-aws-vpc"
}
}
The VPC provides the isolated GCP network. DNS support and hostnames are enabled because application and management services may depend on private name resolution.

4.5.2 Internet gateway

tags = {
Name = "${local.name_prefix}-igw"
}
} Only public ingress and outbound translation components use the Internet gateway. Private application and database instances do not receive public IPv4 addresses.

4.5.3 GCP subnets

cidr_block = each.value.cidr
map_public_ip_on_launch = each.value.public
tags = {
Tier = split("_", each.key)[0]
}
}
The use of for_each reduces duplication while maintaining separate tier names. Public IP assignment is enabled only for the two public subnets.

4.5.4 NAT gateway

domain = "vpc"
tags = {
Name = "${local.name_prefix}-nat-eip"
}
tags = {
Name = "${local.name_prefix}-nat"
}
}
The NAT Gateway allows private workloads to retrieve operating-system packages without accepting unsolicited inbound Internet connections. A single NAT Gateway is used to control laboratory cost. A production design would normally deploy one per Availability Zone to avoid a single-zone dependency and cross-zone data charges.

4.5.5 GCP route policys

route {
cidr_block = "0.0.0.0/0"
}
tags = {
}
for_each = toset(["public_a", "public_b"])
}
route {
cidr_block = "0.0.0.0/0"
}
tags = {
Name = "${local.name_prefix}-web-rt"
}
}
}
route {
cidr_block = "0.0.0.0/0"

```
}
tags = {
Name = "${local.name_prefix}-application-rt"
}
}
for_each = toset(["app_a", "app_b"])
}
Name = "${local.name_prefix}-database-rt"
}
}
for_each = toset(["db_a", "db_b"])
}
route {
cidr_block    = "0.0.0.0/0"
}
tags = {
Name = "${local.name_prefix}-management-rt"
}
}
```

The database route table deliberately lacks an Internet default route. This prevents the database tier from initiating general Internet communication.

4.5.6 Network Connectivity Center

```
description          = "GCP transit hub for the secure multi-cloud lab"
auto_accept_shared_attachments = "disable"
default_route_table_association = "disable"
default_route_table_propagation = "disable"
dns_support                = "enable"
vpn_ecmp_support           = "enable"
tags = {
Name = "${local.name_prefix}-tgw"
}
}
subnet_ids = [
dns_support        = "enable"
ipv6_support       = "disable"
tags = {
Name = "${local.name_prefix}-tgw-vpc-attachment"
}
}
tags = {
Name = "${local.name_prefix}-tgw-rt"
}
}
}
}
```

Network Connectivity Center is the GCP-side network hub. GCP supports HA VPN attachments to Network Connectivity Center and supports both dynamic and static routes.

Default association and propagation are disabled so that routing relationships are created explicitly. This reduces the risk of automatically exposing future attachments.

4.5.7 VPC routes to AWS

```
destination_cidr_block = var.aws_vnet_cidr
depends_on = [
]
}
destination_cidr_block = var.aws_vnet_cidr
depends_on = [
]
```

} No AWS route is created in the web or database route table. This implements network- level least privilege.

Screenshot placeholder 4.3
Insert Google Cloud VPC resource-map screenshot showing public, web, application, database, management and transit subnets. Conceal account identifiers and public IP addresses where unnecessary.
Screenshot placeholder 4.4
Insert Google Cloud Network Connectivity Center screenshot showing the VPC and VPN attachments and their states.

4.6 GCP VPC Firewall Rules

File: infrastructure/aws-security.tf

4.6.1 Load-balancer VPC firewall rule

```
name        = "${local.name_prefix}-alb-sg"
description = "Allow public HTTPS to the approved entry point"
ingress {
description = "HTTPS from the Internet"
protocol    = "tcp"
from_port = 443
to_port     = 443
cidr_blocks = ["0.0.0.0/0"]
}
ingress { description = "HTTP for controlled lab redirect or testing"
protocol    = "tcp"
from_port = 80
to_port     = 80
cidr_blocks = ["0.0.0.0/0"]
}
egress {
description   = "Forward to web tier"
protocol    = "tcp"
from_port    = 80
to_port     = 80
}
tags = {
Name = "${local.name_prefix}-alb-sg"
}
}
```

Port 80 is included only for lab testing or redirection. A production implementation should redirect HTTP to HTTPS and provide a trusted certificate.

4.6.2 Web-tier VPC firewall rule

```
name        = "${local.name_prefix}-web-sg"
description = "Permit traffic from the load balancer only"
description   = "Web traffic from ALB"
protocol    = "tcp"
from_port    = 80
to_port     = 80
}
egress {
description   = "Application requests"
protocol    = "tcp"
from_port    = 8080
to_port     = 8080
}
tags = {
Name = "${local.name_prefix}-web-sg"
}
}
```

The web tier cannot communicate directly with the database because no database rule is defined.

4.6.3 Application-tier VPC firewall rule

```
name        = "${local.name_prefix}-application-sg" description = "Application-tier controls"
```

```
ingress {
description   = "Application traffic from web tier"
protocol      = "tcp"
from_port     = 8080
to_port       = 8080
}
ingress {
description = "Approved HTTPS response and test traffic from AWS service"
protocol      = "tcp"
from_port = 443
to_port       = 443
cidr_blocks = ["10.121.10.0/24"]
}
egress {
description   = "PostgreSQL to database tier"
protocol      = "tcp"
from_port     = 5432
to_port       = 5432
} egress {
description = "HTTPS to AWS supporting service"
protocol      = "tcp"
from_port = 443
to_port       = 443
cidr_blocks = ["10.121.10.0/24"]
}
egress {
description = "Package and service HTTPS"
protocol      = "tcp"
from_port = 443
to_port       = 443
cidr_blocks = ["0.0.0.0/0"]
}
tags = {
Name = "${local.name_prefix}-application-sg"
}
}
```

The application VPC firewall rule allows only the documented database and AWS service flows. General unrestricted outbound access is not used.

4.6.4 Database VPC firewall rule

```
name          = "${local.name_prefix}-database-sg" description = "Permit PostgreSQL from application tier only"
ingress {
description   = "PostgreSQL from application tier"
protocol      = "tcp"
from_port     = 5432
to_port       = 5432
}
egress {
description = "Return established traffic"
protocol      = "-1"
from_port = 0
to_port       =0
}
tags = {
Name = "${local.name_prefix}-database-sg"
}
}
```

No rule permits access from AWS or the public Internet.

4.6.5 Management VPC firewall rule

```
description = "Restricted management and testing"
ingress {
description = "Approved administrator SSH"
protocol    = "tcp"
from_port = 22
to_port     = 22
cidr_blocks = [var.administrator_cidr]
}
egress {
description = "SSH to internal GCP workloads"
protocol    = "tcp"
from_port = 22
to_port     = 22
}
egress {
description = "Controlled test access to AWS service"
protocol    = "tcp"
from_port = 5201
to_port     = 5201
cidr_blocks = ["10.121.10.0/24"] }
egress {
description = "HTTPS updates"
protocol     = "tcp"
from_port = 443
to_port      = 443
cidr_blocks = ["0.0.0.0/0"]
}
tags = {
Name = "${local.name_prefix}-management-sg"
}
}
```

A stronger production pattern would use GCP Systems Manager Session Manager and eliminate inbound SSH completely.

4.7 Google Cloud IAM and KMS Implementation

4.7.1 Compute Engine assume-role policy

```
statement {
effect = "Allow"
principals {
type   = "Service"
identifiers = ["ec2.amazonaws.com"] }
actions = ["sts:AssumeRole"]
}
}
```

This trust policy allows Compute Engine instances to assume the workload role.

4.7.2 KMS key

```
description         = "KMS key for multi-cloud lab encryption"
deletion_window_in_days = 14
enable_key_rotation       = true
tags = {
Name = "${local.name_prefix}-kms"
}
}
name          = "alias/${local.name_prefix}-kms"
}
```

The customer-managed key provides a project-specific encryption boundary. Key rotation is enabled. The deletion window reduces the risk of immediate accidental destruction.

4.7.3 Workload IAM policy

```
statement { sid    = "ReadProjectSecrets"
effect = "Allow"
actions = [
"secretsmanager:GetSecretValue",
"secretsmanager:DescribeSecret"
]
resources = [
]
}
statement {
sid    = "DecryptProjectSecrets"
effect = "Allow"
actions = [
"kms:Decrypt",
"kms:DescribeKey"
]
resources = [
]
} statement {
sid    = "WriteCloud MonitoringLogs"
effect = "Allow"
actions = [
"logs:CreateLogStream",
"logs:PutLogEvents",
"logs:DescribeLogStreams"
]
resources = [
]
}
}
name = "${local.name_prefix}-workload-policy"
}
name          = "${local.name_prefix}-workload-role"
}
policy_arn = "arn:aws:iam::aws:policy/Google CloudSSMManagedInstanceCore"
}
name = "${local.name_prefix}-workload-profile"
}
```

The workload receives only the permissions needed to retrieve its database secret, use the KMS key and send logs. The Systems Manager managed policy supports secure instance administration without permanent SSH keys where SSM connectivity is available.

4.7.4 Database secret

```
name              = "${local.name_prefix}/database/credentials"
recovery_window_in_days = 7
tags = {
Name = "${local.name_prefix}-database-secret"
} }
secret_string = jsonencode({
username = var.database_username
password = var.database_password
database = var.database_name
})
}
```

The database credentials are encrypted with the customer-managed KMS key. They are not embedded in application source files.

4.8 GCP Workload Implementation

File: infrastructure/aws-workload.tf

4.8.1 SSH key for controlled fallback

```
resource "tls_private_key" "lab" {
algorithm = "ED25519"
}
key_name = "${local.name_prefix}-key"
public_key = tls_private_key.lab.public_key_openssh
} resource "local_sensitive_file" "private_key" {
content       = tls_private_key.lab.private_key_openssh
file_permission = "0600"
}
```

This key exists only as a controlled fallback. The private folder must be excluded from Git. A stronger implementation generates and stores keys outside Terraform because Terraform state retains generated key material.

4.8.2 Database cloud-init template

File: infrastructure/templates/db-cloud-init.yaml

```
#cloud-config
package_update: true
packages:
postgresql
postgresql-contrib
awscli
jq
write_files:
path: /usr/local/bin/configure-database.sh
permissions: "0700"
content: |
#!/usr/bin/env bash
set -euo pipefail
SECRET_JSON="$(aws secretsmanager get-secret-value \
--query SecretString \
--output text)"
DB_USER="$(echo "$SECRET_JSON" | jq -r '.username')"
DB_PASSWORD="$(echo "$SECRET_JSON" | jq -r '.password')"
DB_NAME="$(echo "$SECRET_JSON" | jq -r '.database')"
sudo -u postgres psql <<SQL
DO
\$do\$
BEGIN
IF NOT EXISTS (
SELECT FROM pg_catalog.pg_roles WHERE rolname = '${database_username}'
) THEN
CREATE ROLE ${database_username} LOGIN PASSWORD '${database_password}';
END IF;
END
\$do\$;
CREATE DATABASE ${database_name} OWNER ${database_username};
SQL
PG_VERSION="$(ls /etc/postgresql | sort -V | tail -1)"
PG_CONF="/etc/postgresql/${PG_VERSION}/main/postgresql.conf"
PG_HBA="/etc/postgresql/${PG_VERSION}/main/pg_hba.conf" sed -i "s/^#listen_addresses.*/listen_addresses
= '*'/" "$PG_CONF"
echo "host ${database_name} ${database_username} 10.181.30.0/23 scram-sha- 256" \
>> "$PG_HBA"
systemctl restart postgresql
runcmd:
/usr/local/bin/configure-database.sh
```

For readability, the template displays variable substitution. In a hardened implementation, the password should not be inserted into a generated script or cloud-init record. The preferred production method is for the instance to

retrieve the secret at runtime and pass it securely to PostgreSQL without exposing it in Terraform-rendered user data.

A safer simplified implementation is to configure PostgreSQL networking through cloud-init and create database credentials manually through an SSM session after deployment. That approach is recommended for the assessed artefact where cloud-init logs could expose sensitive commands.

4.8.3 Database instance

```
associate_public_ip_address = false root_block_device {
encrypted = true
volume_type = "gp3"
volume_size = 12
}
user_data = <<-EOT
set -euo pipefail
apt-get update
DEBIAN_FRONTEND=noninteractive apt-get install -y postgresql postgresql-contrib awscli jq
PG_VERSION="$(ls /etc/postgresql | sort -V | tail -1)"
sed -i "s/^#listen_addresses.*/listen_addresses = '*'/" \
"/etc/postgresql/${PG_VERSION}/main/postgresql.conf"
echo "host ${var.database_name} ${var.database_username} 10.181.30.0/23 scram- sha-256" \
>> "/etc/postgresql/${PG_VERSION}/main/pg_hba.conf"
systemctl restart postgresql
EOT
metadata_options { http_endpoint = "enabled"
http_tokens = "required"
}
tags = {
Name = "${local.name_prefix}-database"
Tier = "Database"
}
}
```

IMDSv2 is required through http_tokens = "required". The database instance has no public IP address and its EBS volume is encrypted with the project KMS key.

The database account and schema are created after deployment through a protected SSM session:

```
SECRET_JSON=$(aws secretsmanager get-secret-value \
--secret-id secure-multicloud-lab/database/credentials \
--query SecretString \
--output text)
DB_USER=$(echo "$SECRET_JSON" | jq -r '.username')
DB_PASSWORD=$(echo "$SECRET_JSON" | jq -r '.password')
DB_NAME=$(echo "$SECRET_JSON" | jq -r '.database')
sudo -u postgres psql <<SQL
CREATE ROLE ${DB_USER} LOGIN PASSWORD '${DB_PASSWORD}';
CREATE DATABASE ${DB_NAME} OWNER ${DB_USER};
SQL
```

This operational step should not be shown in a screenshot containing the password.

4.8.4 Application instance

```
associate_public_ip_address = false
root_block_device {
encrypted = true
volume_type = "gp3"
volume_size = 10
}
user_data = <<-EOT
set -euo pipefail
apt-get update
DEBIAN_FRONTEND=noninteractive apt-get install -y \
python3 python3-pip python3-venv \
postgresql-client curl jq awscli iperf3 mkdir -p /opt/multicloud-app
```

```
python3 -m venv /opt/multicloud-app/venv
/opt/multicloud-app/venv/bin/pip install flask psycopg2-binary requests gunicorn
cat > /opt/multicloud-app/app.py <<'PYTHON'
import json
import os
import subprocess
from flask import Flask, jsonify
app = Flask(__name__)
@app.get("/health")
def health():
return jsonify({
"service": "aws-application-tier",
"status": "healthy"
})
@app.get("/aws-health")
def aws_health():
import requests
endpoint = os.environ.get(
"AWS_SERVICE_URL",
"http://10.121.10.10:8080/health"
) response = requests.get(endpoint, timeout=5)
return jsonify({
"aws_status_code": response.status_code,
"aws_response": response.json()
})
if __name__ == "__main__":
app.run(host="0.0.0.0", port=8080)
PYTHON
cat > /etc/systemd/system/multicloud-app.service <<'SERVICE'
[Unit]
Description=GCP Multi-Cloud Application Service
After=network-online.target
Wants=network-online.target
[Service]
Type=simple
WorkingDirectory=/opt/multicloud-app
ExecStart=/opt/multicloud-app/venv/bin/gunicorn \
--bind 0.0.0.0:8080 app:app
Restart=always
RestartSec=5
Environment=AWS_SERVICE_URL=http://10.121.10.10:8080/health
[Install] WantedBy=multi-user.target
SERVICE
systemctl daemon-reload
systemctl enable --now multicloud-app
EOT
metadata_options {
http_endpoint = "enabled"
http_tokens = "required"
}
tags = {
Name = "${local.name_prefix}-application"
Tier = "Application"
}
depends_on = [
]
}
```

The application exposes a local health endpoint and a second endpoint that calls the AWS supporting service through its private IP address. This provides a controlled inter-cloud application flow.

4.8.5 Web instance

```
associate_public_ip_address = false
root_block_device {
encrypted = true
volume_type = "gp3"
volume_size = 10
}
user_data = <<-EOT
set -euo pipefail
apt-get update
DEBIAN_FRONTEND=noninteractive apt-get install -y nginx curl
cat > /etc/nginx/sites-available/default <<'NGINX'
server {
listen 80 default_server;
proxy_set_header Host $host;
proxy_set_header X-Real-IP $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
}
}
NGINX
nginx -t
systemctl enable --now nginx
EOT
metadata_options {
http_endpoint = "enabled"
http_tokens = "required"
}
tags = {
Name = "${local.name_prefix}-web"
Tier = "Web"
}
depends_on = [
]
```

} The web instance acts as a reverse proxy. It cannot reach the database directly because its VPC firewall rule permits outbound access only to the application tier.

4.8.6 Application Load Balancer

```
name          = substr("${local.name_prefix}-alb", 0, 32)
load_balancer_type = "application"
internal      = false
security_groups = [
]
subnets = [
]
drop_invalid_header_fields = true
tags = {
Name = "${local.name_prefix}-alb"
}
}
name          = substr("${local.name_prefix}-web-tg", 0, 32) port = 80
protocol = "HTTP"
health_check {
enabled      = true
path         = "/health"
protocol     = "HTTP"
port         = "traffic-port"
healthy_threshold = 2
unhealthy_threshold = 3
timeout      =5
```

```
interval      = 30
matcher       = "200"
}
tags = {
Name = "${local.name_prefix}-web-tg"
}
}
port          = 80
port          = 80
protocol      = "HTTP"
default_action {
type          = "forward"
}
}
```

The laboratory listener uses HTTP to avoid requiring a public DNS name and certificate. The final report must identify this as a lab limitation. Production use requires HTTPS with an approved certificate and an HTTP-to-HTTPS redirect.

4.9 AWS Network Implementation

File: infrastructure/aws-network.tf

4.9.1 Resource group

```
name          = "rg-${local.name_prefix}"
location = var.aws_location
tags = local.common_tags
}
```

All principal AWS project resources are grouped for lifecycle and cost management.

4.9.2 Virtual network and subnets

```
location    = awsrm_resource_group.main.location
resource_group_name = awsrm_resource_group.main.name
address_space     = [var.aws_vnet_cidr]
tags = local.common_tags
}
name       = "GatewaySubnet"
resource_group_name = awsrm_resource_group.main.name
virtual_network_name = awsrm_virtual_network.main.name
address_prefixes    = ["10.121.0.0/27"]
}
name       = "snet-service"
resource_group_name = awsrm_resource_group.main.name
virtual_network_name = awsrm_virtual_network.main.name
address_prefixes    = ["10.121.10.0/24"]
}
name       = "snet-monitoring"
resource_group_name = awsrm_resource_group.main.name
virtual_network_name = awsrm_virtual_network.main.name address_prefixes      = ["10.121.20.0/24"]
}
name          = "snet-management"
resource_group_name = awsrm_resource_group.main.name
virtual_network_name = awsrm_virtual_network.main.name
address_prefixes      = ["10.121.30.0/24"]
}
```

AWS requires the gateway subnet to use the exact name GatewaySubnet.

4.9.3 AWS Network VPC Firewall Rules

File: infrastructure/aws-security.tf

```
name          = "nsg-${local.name_prefix}-service"
location      = awsrm_resource_group.main.location
resource_group_name = awsrm_resource_group.main.name
security_rule {
```

```
name                    = "Allow-GCP-Application-HTTPS"
priority                = 100
direction               = "Inbound"
access                  = "Allow"
protocol                = "Tcp"
source_port_range       = "*"
destination_port_range  = "8080"
source_address_prefix   = "10.181.30.0/23" destination_address_prefix = "10.121.10.0/24"
}
security_rule {
name                    = "Allow-GCP-Management-Iperf"
priority                = 110
direction               = "Inbound"
access                  = "Allow"
protocol                = "Tcp"
source_port_range       = "*"
destination_port_range  = "5201"
source_address_prefix   = "10.181.50.0/24"
destination_address_prefix = "10.121.10.0/24"
}
security_rule {
name                    = "Deny-Other-VNet-Inbound"
priority                = 4000
direction               = "Inbound"
access                  = "Deny"
protocol                = "*"
source_port_range       = "*"
destination_port_range  = "*"
source_address_prefix   = "VirtualNetwork"
destination_address_prefix = "*"
} tags = local.common_tags
}
subnet_id               = awsrm_subnet.service.id
network_security_group_id = awsrm_network_security_group.service.id
}
```

The NSG permits only the GCP application flow and the controlled iperf3 test. The deny rule makes the intended restriction visible rather than relying only on default AWS rules.

4.9.4 Management NSG

```
name                = "nsg-${local.name_prefix}-management"
location            = awsrm_resource_group.main.location
resource_group_name = awsrm_resource_group.main.name
security_rule {
name                    = "Allow-Approved-SSH"
priority                = 100
direction               = "Inbound"
access                  = "Allow"
protocol                = "Tcp"
source_port_range       = "*"
destination_port_range  = "22"
source_address_prefix   = var.administrator_cidr
destination_address_prefix = "*" }
tags = local.common_tags
}
subnet_id                 = awsrm_subnet.management.id
network_security_group_id = awsrm_network_security_group.management.id
}
```

Where AWS Bastion is implemented, direct public SSH should be removed.

4.10 AWS KMS and Secrets Manager Implementation

4.10.1 AWS KMS and Secrets Manager resource

```
name = substr(
replace("kv-${local.name_prefix}", "-", ""),
0,
)
location              = awsrm_resource_group.main.location
resource_group_name           = awsrm_resource_group.main.name
tenant_id             = data.awsrm_client_config.current.tenant_id
sku_name              = "standard"
purge_protection_enabled = true
soft_delete_retention_days = 14 enable_rbac_authorization = true
public_network_access_enabled = true
tags = local.common_tags
}
```

Purge protection and soft deletion reduce the risk that keys and secrets are permanently removed accidentally. AWS IAM role-based access control is selected instead of legacy access policies.

4.10.2 AWS encryption key

```
name              = "multicloud-lab-key"
key_vault_id = awsrm_key_vault.main.id
key_type          = "RSA"
key_size          = 3072
key_opts = [
"decrypt",
"encrypt",
"sign",
"unwrapKey",
"verify",
"wrapKey"
]
rotation_policy {
automatic {
time_before_expiry = "P30D" }
expire_after          = "P365D"
notify_before_expiry = "P45D"
}
}
```

This key demonstrates customer-managed key governance within AWS. It can later be linked to disk-encryption sets or application cryptography.

4.10.3 AWS KMS and Secrets Manager secret

```
name          = "application-environment"
value         = "lab"
key_vault_id = awsrm_key_vault.main.id
depends_on = [
awsrm_role_assignment.terraform_keyvault_admin
]
}
```

The project stores a non-sensitive demonstration value. Real application secrets should be written using a protected pipeline or secure administrative process rather than placed directly in Terraform state.

4.10.4 Terraform AWS KMS and Secrets Manager role

```
scope         = awsrm_key_vault.main.id
role_definition_name = "AWS KMS and Secrets Manager Administrator"
principal_id              = data.awsrm_client_config.current.object_id
```

} This role permits the deployment identity to create the demonstration key and secret. It should be reduced or removed after provisioning where operational requirements permit.

Screenshot placeholder 4.5

Insert AWS KMS and Secrets Manager screenshot showing soft delete, purge protection and RBAC. Do not display secret values or key material.

4.11 AWS Workload Implementation

File: infrastructure/aws-workload.tf

4.11.1 Service network interface

```
name           = "nic-${local.name_prefix}-service"
location       = awsrm_resource_group.main.location
resource_group_name = awsrm_resource_group.main.name
ip_configuration {
name                 = "internal"
subnet_id            = awsrm_subnet.service.id
private_ip_address_allocation = "Static"
private_ip_address       = "10.121.10.10"
}
tags = local.common_tags
}
```

The static private address is used by the GCP application and by the test scripts.

4.11.2 Amazon EC2 instance SSH key

```
resource "tls_private_key" "aws_service" {
algorithm = "ED25519" }
```

As with the GCP key, this material remains in Terraform state. For production, a separately governed public key should be provided as an input.

4.11.3 AWS supporting service VM

```
name          = "vm-${local.name_prefix}-service"
computer_name         = "awsservice"
resource_group_name = awsrm_resource_group.main.name
location       = awsrm_resource_group.main.location
size          = var.aws_vm_size
admin_username        = "awsadmin"
network_interface_ids = [
awsrm_network_interface.service.id
]
disable_password_authentication = true
admin_ssh_key {
username = "awsadmin"
public_key = tls_private_key.aws_service.public_key_openssh
}
identity {
type = "SystemAssigned"
} os_disk {
name           = "osdisk-${local.name_prefix}-service"
caching         = "ReadWrite"
storage_account_type = "Standard_LRS"
disk_size_gb         = 30
}
source_image_reference {
publisher = "Canonical"
o er = "ubuntu-24_04-lts"
sku      = "server"
version = "latest"
}
custom_data = base64encode(<<-CLOUDINIT
#cloud-config
package_update: true
packages:
python3
python3-pip
```

```
python3-venv
iperf3
curl
jq
write_files:
path: /opt/aws-service/app.py
permissions: "0644"
content: |
from flask import Flask, jsonify
import socket
import datetime
app = Flask(__name__)
@app.get("/health")
def health():
return jsonify({
"service": "aws-supporting-service",
"status": "healthy",
"host": socket.gethostname(),
"timestamp": datetime.datetime.utcnow().isoformat() + "Z"
})
path: /etc/systemd/system/aws-service.service
permissions: "0644"
content: |
[Unit]
Description=AWS Multi-Cloud Supporting Service
After=network-online.target
Wants=network-online.target [Service]
Type=simple
WorkingDirectory=/opt/aws-service
ExecStart=/opt/aws-service/venv/bin/gunicorn \
--bind 0.0.0.0:8080 app:app
Restart=always
RestartSec=5
[Install]
WantedBy=multi-user.target
runcmd:
mkdir -p /opt/aws-service
python3 -m venv /opt/aws-service/venv
/opt/aws-service/venv/bin/pip install flask gunicorn
systemctl daemon-reload
systemctl enable --now aws-service
systemctl enable --now iperf3
CLOUDINIT
)
tags = local.common_tags
```

} The AWS service exposes its health endpoint on TCP/8080 and operates an iperf3 server on TCP/5201. Its system-assigned managed identity can be granted access to AWS KMS and Secrets Manager without storing AWS credentials on the VM.

4.11.4 Managed-identity AWS KMS and Secrets Manager access

```
scope        = awsrm_key_vault.main.id
role_definition_name = "AWS KMS and Secrets Manager Secrets User"
principal_id         = awsrm_linux_virtual_machine.service.identity[0].principal_id
}
```

The managed identity can retrieve authorised secrets while avoiding embedded credentials.

4.12 AWS Site-to-Site VPN Implementation

4.12.1 Public IP addresses

```
name         = "pip-${local.name_prefix}-vpn-1"
```

```
location      = awsrm_resource_group.main.location
resource_group_name = awsrm_resource_group.main.name
allocation_method = "Static"
sku           = "Standard"
zones         = ["1", "2", "3"]
tags = local.common_tags
}
name          = "pip-${local.name_prefix}-vpn-2" location      = awsrm_resource_group.main.location
resource_group_name = awsrm_resource_group.main.name
allocation_method = "Static"
sku           = "Standard"
zones         = ["1", "2", "3"]
tags = local.common_tags
}
```

The two public addresses support an active-active AWS Site-to-Site VPN. AWS supports active-active VPN gateways and BGP for highly available cross-premises connectivity.

Zone configuration must be adjusted if the selected AWS Region or gateway SKU does not support the specified zones.

4.12.2 Active-active VPN Gateway

```
name          = "vng-${local.name_prefix}"
location      = awsrm_resource_group.main.location
resource_group_name = awsrm_resource_group.main.name
type          = "Vpn"
vpn_type      = "RouteBased"
active_active = true
enable_bgp = true
sku           = "VpnGw2AZ"
generation    = "Generation2"
ip_configuration {
name                 = "vng-ipconfig-1" public_ip_address_id    = awsrm_public_ip.vpn_1.id
private_ip_address_allocation = "Dynamic"
subnet_id          = awsrm_subnet.gateway.id
}
ip_configuration {
name               = "vng-ipconfig-2"
public_ip_address_id    = awsrm_public_ip.vpn_2.id
private_ip_address_allocation = "Dynamic"
subnet_id          = awsrm_subnet.gateway.id
}
bgp_settings {
asn = var.aws_vpn_asn
}
tags = local.common_tags
}
```

The SKU must be confirmed against current regional availability and project cost. Microsoft advises selecting a VPN Gateway SKU based on throughput, features and availability requirements.

Provisioning a VPN Gateway may take a substantial period. Terraform should not be interrupted merely because the resource remains in a creating state for several minutes.

Screenshot placeholder 4.6

Insert AWS Site-to-Site VPN overview showing active-active mode, BGP enabled and two public IP configurations. Conceal public IP addresses in the published report where full disclosure is unnecessary.

4.13 AWS-GCP VPN Implementation

4.13.1 Implementation approach

Google Cloud HA VPN provides two tunnels for each VPN connection. GCP recommends configuring both tunnels for redundancy because maintenance may temporarily remove one tunnel.

Microsoft’s current AWS-GCP BGP tutorial uses an active-active AWS Site-to-Site VPN with two GCP site-to-site connections.

File: infrastructure/vpn.tf
bgp_asn = var.aws_vpn_asn
ip_address = awsrm_public_ip.vpn_1.ip_address
type = "ipsec.1"
tags = {
Name = "${local.name_prefix}-aws-cgw-1"
}
}
bgp_asn = var.aws_vpn_asn
ip_address = awsrm_public_ip.vpn_2.ip_address type = "ipsec.1"
tags = {
Name = "${local.name_prefix}-aws-cgw-2"
}
}

4.13.3 GCP VPN connections

type = "ipsec.1"
static_routes_only = false
tunnel1_preshared_key = var.vpn_shared_key_1
tunnel1_ike_versions = ["ikev2"]
tunnel1_phase1_encryption_algorithms = ["AES256"]
tunnel1_phase1_integrity_algorithms = ["SHA2-256"]
tunnel1_phase1_dh_group_numbers = [14]
tunnel1_phase2_encryption_algorithms = ["AES256"]
tunnel1_phase2_integrity_algorithms = ["SHA2-256"]
tunnel1_phase2_dh_group_numbers = [14] tunnel1_dpd_timeout_action = "restart"
tags = {
Name = "${local.name_prefix}-vpn-1"
}
}
type = "ipsec.1"
static_routes_only = false
tunnel1_preshared_key = var.vpn_shared_key_2
tunnel1_ike_versions = ["ikev2"]
tunnel1_phase1_encryption_algorithms = ["AES256"]
tunnel1_phase1_integrity_algorithms = ["SHA2-256"]
tunnel1_phase1_dh_group_numbers = [14]
tunnel1_phase2_encryption_algorithms = ["AES256"]
tunnel1_phase2_integrity_algorithms = ["SHA2-256"]
tunnel1_phase2_dh_group_numbers = [14]
tunnel1_dpd_timeout_action = "restart" tags = {
Name = "${local.name_prefix}-vpn-2"
}
}

GCP permits custom tunnel settings, including IKE versions, encryption algorithms, integrity algorithms and Dead Peer Detection actions.

The chosen parameters must be confirmed as mutually supported by the selected AWS gateway SKU. A failed Phase 1 or Phase 2 negotiation commonly indicates an incompatible cryptographic proposal or shared key.

4.13.4 Network Connectivity Center VPN attachment propagation

}
}
}

These resources allow BGP-learned AWS routes to enter the selected Network Connectivity Center route table.

4.13.5 AWS Customer Gateways

name = "lng-${local.name_prefix}-aws-1"
location = awsrm_resource_group.main.location
resource_group_name = awsrm_resource_group.main.name
bgp_settings {

```
}
tags = local.common_tags
}
name        = "lng-${local.name_prefix}-aws-2"
location    = awsrm_resource_group.main.location
resource_group_name = awsrm_resource_group.main.name
}
tags = local.common_tags
}
```

The GCP tunnel public address becomes the AWS local gateway address. The GCP inside BGP address and Network Connectivity Center ASN identify the remote BGP peer.

4.13.6 AWS VPN connections

```
name          = "conn-${local.name_prefix}-aws-1"
location      = awsrm_resource_group.main.location
resource_group_name = awsrm_resource_group.main.name
type              = "IPsec"
virtual_network_gateway_id = awsrm_virtual_network_gateway.main.id
shared_key          = var.vpn_shared_key_1
enable_bgp          = true
ipsec_policy {
dh_group     = "DHGroup14"
ike_encryption = "AES256"
ike_integrity = "SHA256" ipsec_encryption = "AES256"
ipsec_integrity = "SHA256"
pfs_group     = "PFS14"
sa_datasize = 102400000
sa_lifetime   = 27000
}
tags = local.common_tags
}
name          = "conn-${local.name_prefix}-aws-2"
location      = awsrm_resource_group.main.location
resource_group_name = awsrm_resource_group.main.name
type              = "IPsec"
virtual_network_gateway_id = awsrm_virtual_network_gateway.main.id
shared_key          = var.vpn_shared_key_2
enable_bgp          = true
ipsec_policy {
dh_group     = "DHGroup14"
ike_encryption = "AES256"
ike_integrity = "SHA256"
ipsec_encryption = "AES256" ipsec_integrity = "SHA256"
pfs_group     = "PFS14"
sa_datasize = 102400000
sa_lifetime   = 27000
}
tags = local.common_tags
}
```

The same shared keys and compatible cryptographic parameters are supplied to both providers.

4.13.7 Provider-schema validation note

VPN-resource attributes can change across major versions of the Google Cloud and AWSRM providers. The configuration must therefore be validated against the selected .terraform.lock.hcl versions before deployment:

```
terraform validate
terraform providers schema -json > provider-schema.json
```

This is not an admission that the design is incomplete; it is a necessary Infrastructure-as- Code control because provider APIs and schemas evolve.

4.13.8 VPN verification commands

GCP

```
aws ec2 describe-vpn-connections \
--vpn-connection-ids \
--query 'VpnConnections[].VgwTelemetry[].{ OutsideIp:OutsideIpAddress,
Status:Status,
StatusMessage:StatusMessage,
AcceptedRoutes:AcceptedRouteCount
}' \
--output table
```

AWS

```
--resource-group "$(terraform output -raw aws_resource_group_name)" \
--output table
```

AWS BGP peers

```
--resource-group "$(terraform output -raw aws_resource_group_name)" \
--name "$(terraform output -raw aws_vpn_gateway_name)" \
--output table
```

Screenshot placeholder 4.7

Insert Google Cloud HA VPN tunnel-status screenshot showing the selected tunnels as UP and accepted route counts greater than zero.

Screenshot placeholder 4.8

Insert AWS VPN Connection screenshot showing both connections as Connected.

Screenshot placeholder 4.9

Insert AWS BGP peer-status output showing the GCP peers as Connected and displaying learned routes. Conceal public gateway addresses where appropriate.

4.14 GCP Monitoring and Logging

File: infrastructure/aws-monitoring.tf

4.14.1 Cloud Monitoring log groups

```
name         = "/${local.name_prefix}/application"
retention_in_days = 30
tags = {
Name = "${local.name_prefix}-application-logs"
}
}
name         = "/${local.name_prefix}/vpc-flow"
retention_in_days = 30
tags = {
Name = "${local.name_prefix}-vpc-flow-logs"
}
}
```

Cloud Monitoring retention is explicitly defined so that logs are not retained indefinitely by accident.

4.14.2 VPC Flow Logs role

```
statement {
effect = "Allow" principals {
type   = "Service"
identifiers = ["vpc-flow-logs.amazonaws.com"]
}
actions = ["sts:AssumeRole"]
}
}
statement {
effect = "Allow"
actions = [
"logs:CreateLogGroup",
"logs:CreateLogStream",
"logs:PutLogEvents",
"logs:DescribeLogGroups",
```

```
"logs:DescribeLogStreams"
]
resources = ["*"]
}
}
}
name = "${local.name_prefix}-flow-logs-policy"
}
```

4.14.3 VPC Flow Logs

```
traffic_type         = "ALL"
max_aggregation_interval = 60
tags = {
Name = "${local.name_prefix}-vpc-flow-log"
}
}
```

Both accepted and rejected flows are captured. Flow logs do not contain packet payloads, but they provide evidence of source, destination, ports, protocol and disposition.

4.14.4 Cloud Audit Logs bucket

```
resource "random_string" "cloudtrail_su ix" {
length = 8 upper = false
special = false
}
force_destroy = true
tags = {
Name = "${local.name_prefix}-cloudtrail"
}
}
block_public_acls = true
block_public_policy        = true
ignore_public_acls = true
restrict_public_buckets = true
}
status = "Enabled"
}
}
rule {
apply_server_side_encryption_by_default {
sse_algorithm       = "aws:kms"
}
}
}
```

force_destroy is suitable only for the disposable laboratory. A production audit bucket should not normally permit automatic deletion of retained records.

4.14.5 Cloud Audit Logs bucket policy

```
statement {
sid    = "GCPCloud Audit LogsAclCheck"
effect = "Allow"
principals { type   = "Service"
identifiers = ["cloudtrail.amazonaws.com"]
}
actions = ["s3:GetBucketAcl"]
resources = [
]
}
statement {
sid    = "GCPCloud Audit LogsWrite"
effect = "Allow"
```

```
principals {
type   = "Service"
identifiers = ["cloudtrail.amazonaws.com"]
}
actions = ["s3:PutObject"]
resources = [
] condition {
test   = "StringEquals"
variable = "s3:x-amz-acl"
values = ["bucket-owner-full-control"]
}
}
}
}
```

4.14.6 Cloud Audit Logs

```
name                  = "${local.name_prefix}-trail"
include_global_service_events = true
is_multi_region_trail         = true
enable_log_file_validation = true
event_selector {
read_write_type          = "All"
include_management_events = true
} depends_on = [
]
tags = {
Name = "${local.name_prefix}-trail"
}
}
```

Multi-region management events and log-file validation support auditability.

4.14.7 Cloud Monitoring alarm for VPN tunnel state

```
alarm_name            = "${local.name_prefix}-vpn-1-tunnel-state"
alarm_description = "Alarm when the selected VPN tunnel is down"
namespace             = "GCP/VPN"
metric_name           = "TunnelState"
statistic      = "Maximum"
period         = 60
evaluation_periods = 2
threshold      =1
comparison_operator = "LessThanThreshold"
treat_missing_data = "breaching"
dimensions = {
} tags = local.common_tags
}
```

An additional alarm should be created for the second VPN connection in the same pattern.

4.14.8 Security Command Center

```
count = var.enable_guardduty ? 1 : 0
enable                 = true
finding_publishing_frequency = "FIFTEEN_MINUTES"
tags = {
Name = "${local.name_prefix}-guardduty"
}
}
```

Security Command Center provides managed threat detection using GCP telemetry. Enabling the detector does not guarantee that findings will exist; it enables the service to analyse supported data sources.

Screenshot placeholder 4.10

Insert Cloud Audit Logs screenshot showing the project trail as logging.

Screenshot placeholder 4.11

Insert VPC Flow Logs screenshot showing the flow log as Active.
Screenshot placeholder 4.12
Insert Security Command Center screenshot showing the detector enabled. Do not manufacture findings.

4.15 Amazon CloudWatch and Logging

File: infrastructure/aws-monitoring.tf

4.15.1 Log Analytics workspace

```
name        = "law-${local.name_prefix}"
location    = awsrm_resource_group.main.location
resource_group_name = awsrm_resource_group.main.name
sku         = "PerGB2018"
retention_in_days = 30
tags = local.common_tags
}
```

The workspace provides a central AWS repository for gateway, activity and virtual-machine telemetry.

4.15.2 Amazon CloudWatch Action Group

```
name        = "ag-${local.name_prefix}-security"
resource_group_name = awsrm_resource_group.main.name
short_name          = "mcsecure"
tags = local.common_tags
}
```

Email, SMS or webhook receivers can be added after confirming the approved project contact details. They are not fabricated in the reference code.

4.15.3 VPN Gateway diagnostics

```
name               = "diag-${local.name_prefix}-vpn"
target_resource_id         = awsrm_virtual_network_gateway.main.id
log_analytics_workspace_id = awsrm_log_analytics_workspace.main.id enabled_log {
category = "GatewayDiagnosticLog"
}
enabled_log {
category = "TunnelDiagnosticLog"
}
enabled_log {
category = "RouteDiagnosticLog"
}
enabled_log {
category = "IKEDiagnosticLog"
}
enabled_metric {
category = "AllMetrics"
}
}
```

Diagnostic categories can vary by region and provider version. They should be checked before applying the final plan.

4.15.4 AWS KMS and Secrets Manager diagnostics

```
name               = "diag-${local.name_prefix}-keyvault" target_resource_id         = awsrm_key_vault.main.id
log_analytics_workspace_id = awsrm_log_analytics_workspace.main.id
enabled_log {
category = "AuditEvent"
}
enabled_metric {
category = "AllMetrics"
}
}
```

AWS KMS and Secrets Manager audit events provide evidence of secret and key operations.

4.15.5 Network VPC Firewall Rule diagnostics

```
name                = "diag-${local.name_prefix}-service-nsg"
target_resource_id          = awsrm_network_security_group.service.id
log_analytics_workspace_id = awsrm_log_analytics_workspace.main.id
enabled_log {
category = "NetworkSecurityGroupEvent"
}
enabled_log {
category = "NetworkSecurityGroupRuleCounter"
}
```

} Where current AWS service behaviour replaces or limits legacy NSG flow logs, the implementation should use the currently supported virtual-network flow-log mechanism and document the exact service used.

4.15.6 Amazon CloudWatch Agent

```
name                = "AWSMonitorLinuxAgent"
virtual_machine_id        = awsrm_linux_virtual_machine.service.id
publisher           = "Microsoft.AWS.Monitor"
type                = "AWSMonitorLinuxAgent"
type_handler_version       = "1.0"
automatic_upgrade_enabled = true
}
```

The Amazon CloudWatch Agent collects guest-level data, but a Data Collection Rule is required to define which logs or performance counters are sent.

4.15.7 Data Collection Rule

```
name          = "dcr-${local.name_prefix}-linux"
resource_group_name = awsrm_resource_group.main.name
location      = awsrm_resource_group.main.location
destinations {
log_analytics {
workspace_resource_id = awsrm_log_analytics_workspace.main.id
name          = "law-destination"
}
} data_flow {
streams       = ["Microsoft-Syslog"]
destinations = ["law-destination"]
}
data_sources {
syslog {
name          = "linux-syslog"
streams       = ["Microsoft-Syslog"]
facility_names = ["auth", "authpriv", "daemon", "syslog", "user"]
log_levels    = ["Warning", "Error", "Critical", "Alert", "Emergency"]
}
}
tags = local.common_tags
}
name                = "dcra-${local.name_prefix}-linux"
target_resource_id = awsrm_linux_virtual_machine.service.id
data_collection_rule_id = awsrm_monitor_data_collection_rule.linux.id
}
```

The rule collects selected Linux security and service messages without ingesting every verbose event.

4.15.8 VPN availability alert

```
resource_group_name = awsrm_resource_group.main.name
scopes        = [awsrm_virtual_network_gateway.main.id]
description         = "Detect prolonged absence of VPN ingress traffic"
severity      =2
frequency           = "PT5M"
window_size         = "PT15M"
```

```
criteria {
metric_namespace = "Microsoft.Network/virtualNetworkGateways"
metric_name         = "TunnelIngressBytes"
aggregation   = "Total"
operator      = "LessThanOrEqual"
threshold     =0
}
action {
action_group_id = awsrm_monitor_action_group.security.id
}
tags = local.common_tags
}
```

An absence-of-traffic alert may produce false positives during legitimate idle periods. For the final environment, connection-status or BGP-specific health criteria should be preferred where exposed as metrics.

Screenshot placeholder 4.13

Insert AWS Log Analytics screenshot showing the workspace and data-retention setting. Screenshot placeholder 4.14

Insert VPN Gateway diagnostic setting showing tunnel, route and IKE diagnostic logs enabled.

4.16 Microsoft AWS Security Hub and GuardDuty

4.16.1 Defender subscription plans

```
count = var.enable_defender ? 1 : 0
tier         = "Standard"
resource_type = "VirtualMachines"
}
count = var.enable_defender ? 1 : 0
tier         = "Standard"
resource_type = "KeyVaults"
}
count = var.enable_defender ? 1 : 0
tier         = "Standard"
resource_type = "StorageAccounts"
```

} Defender plan names and chargeable features must be checked against the current AWS account before deployment. Enabling a Standard tier can incur charges. The project budget should therefore be configured and monitored before applying these resources.

4.16.2 Defender operational interpretation

AWS Security Hub and GuardDuty is used for security posture and workload recommendations. Its presence does not prove compliance. Recommendations must be reviewed, assigned and remediated where applicable.

Screenshot placeholder 4.15

Insert AWS Security Hub and GuardDuty Environment Settings screenshot showing the enabled plans.

Screenshot placeholder 4.16

Insert AWS Security Hub and GuardDuty recommendations summary. The screenshot must show actual recommendations from the deployed environment and must not use fabricated scores.

4.17 Microsoft Entra ID and Google Cloud Workforce Identity Federation

4.17.1 Scope of automation

Terraform can create Microsoft Entra VPC firewall rules and selected Google Cloud Workforce Identity Federation permission sets and assignments. The complete SAML and SCIM trust still requires tenant- specific administrative steps because:

GCP generates tenant-specific SAML metadata and SCIM credentials;
Entra generates enterprise-application identifiers and signing certificates;
SCIM access tokens must be copied securely;
group and account assignments are organisational decisions;
certificate rollover must be governed.

Microsoft’s current integration guidance explains that Entra ID can control access to Google Cloud Workforce Identity Federation, provide automatic sign-in and manage identities centrally. Microsoft also documents automatic provisioning and deprovisioning to Google Cloud Workforce Identity Federation.

4.17.2 Entra groups

File: infrastructure/identity.tf
resource "awsad_group" "cloud_admins" { display_name = "MC-Cloud-Admins"
security_enabled = true
mail_enabled = false
description = "Administrators for the AWS-GCP project"
}
resource "awsad_group" "network_admins" {
display_name = "MC-Network-Admins"
security_enabled = true
mail_enabled = false
description = "Network administrators for VPN and routing"
}
resource "awsad_group" "security_auditors" {
display_name = "MC-Security-Auditors"
security_enabled = true
mail_enabled = false
description = "Read-only cloud security auditors"
}
resource "awsad_group" "terraform_deployers" {
display_name = "MC-Terraform-Deployers"
security_enabled = true
mail_enabled = false
description = "Approved Infrastructure-as-Code operators"
} Users are not automatically added because user membership must be based on the actual student and supervisor-approved identities.

4.17.3 AWS IAM role assignments

scope = awsrm_resource_group.main.id
role_definition_name = "Security Reader"
principal_id = awsad_group.security_auditors.object_id
}
scope = awsrm_resource_group.main.id
role_definition_name = "Network Contributor"
principal_id = awsad_group.network_admins.object_id
}
scope = awsrm_resource_group.main.id
role_definition_name = "Contributor"
principal_id = awsad_group.cloud_admins.object_id
}
The broad Contributor role does not permit management of AWS IAM role assignments. This separation reduces the chance that a routine cloud administrator can grant additional privileges.

4.17.4 Google Cloud Workforce Identity Federation discovery

Google Cloud Workforce Identity Federation must be enabled before Terraform can discover its instance.
sso_instance_arn = try(
null
)
identity_store_id = try(
null
)
}
The environment must contain one intended Workforce Identity Federation instance. In an GCP Organizations deployment, the organisation instance is normally preferred.

4.17.5 GCP permission sets

name = "MultiCloudSecurityAuditor"
description = "Read-only security review access"
instance_arn = local.sso_instance_arn
session_duration = "PT4H"

```
}
instance_arn          = local.sso_instance_arn
managed_policy_arn = "arn:aws:iam::aws:policy/SecurityAudit"
name          = "MultiCloudNetworkAdministrator"
description = "Restricted networking administration"
instance_arn          = local.sso_instance_arn
session_duration = "PT2H"
}
```

A custom inline policy is used for network administration rather than attaching full AdministratorAccess.

```
statement {
effect = "Allow"
actions = [
"ec2:Describe*",
"ec2:CreateRoute",
"ec2:ReplaceRoute",
"ec2:DeleteRoute",
"ec2:CreateTags",
"ec2:ModifyVpnConnectionOptions",
"ec2:ModifyVpnTunnelOptions",
"cloudwatch:GetMetricData",
"cloudwatch:ListMetrics",
"logs:DescribeLogGroups",
"logs:StartQuery",
"logs:GetQueryResults" ]
resources = ["*"]
}
}
instance_arn         = local.sso_instance_arn
}
```

The exact permission list should be refined after observing the tasks actually required.

4.17.6 Manual Entra-GCP federation procedure

The following sequence must be performed with real tenant metadata.

Stage 1: Prepare Google Cloud Workforce Identity Federation

1. Open Google Cloud Workforce Identity Federation.
2. Confirm the intended identity-center instance.
3. Select Settings.
4. Change the identity source to an external identity provider where appropriate.
5. Download or copy the GCP SAML service-provider metadata.
6. Enable automatic provisioning.
7. Copy the generated SCIM endpoint and access token into a secure temporary location.
8. Do not place the SCIM token in the report, screenshots or Terraform variables.

Stage 2: Configure the Entra enterprise application

1. Open Microsoft Entra admin centre. 2. Navigate to Enterprise applications.
3. Add the Google Cloud Workforce Identity Federation enterprise application.
4. Configure SAML-based single sign-on using the GCP metadata.
5. Download the Entra federation metadata XML or record the login URL, issuer and signing certificate.
6. Upload or enter the Entra identity-provider metadata in Google Cloud Workforce Identity Federation.
7. Test SAML sign-in with one approved test user.

Stage 3: Configure SCIM provisioning

1. Open the enterprise application’s Provisioning section.
2. Select automatic provisioning.
3. Enter the GCP SCIM endpoint and secret token.
4. Test the connection.
5. Scope provisioning to assigned users and groups.
6. Assign MC-Cloud-Admins, MC-Network-Admins and MC-Security-Auditors as appropriate.
7. Start provisioning.
8. Confirm that the assigned groups appear in Google Cloud Workforce Identity Federation.

Stage 4: Map Google Cloud projects and permission sets

1. Select the intended Google Cloud project.
2. Assign the provisioned Entra-backed group.
3. Attach the corresponding GCP permission set.
4. Test access through the GCP access portal.
5. Confirm that the user receives temporary credentials.
6. Confirm that an unassigned user is denied.

GCP Site-to-Site network connectivity is independent of the SAML process. Federation controls workforce management-plane access, while the VPN controls private data-plane connectivity. Screenshot placeholder 4.17

Insert Microsoft Entra enterprise-application screenshot showing SAML single sign-on configured. Conceal tenant IDs, certificate fingerprints and login identifiers where appropriate.

Screenshot placeholder 4.18

Insert Entra provisioning screenshot showing SCIM provisioning status as successful.

Screenshot placeholder 4.19

Insert Google Cloud Workforce Identity Federation screenshot showing the provisioned groups and permission-set assignments.

Screenshot placeholder 4.20

Insert successful federated GCP access-portal screenshot showing the available role. Do not expose the user's email address unnecessarily.

4.18 Terraform Outputs

File: infrastructure/outputs.tf

```
}
}
}
}
sensitive = true
}
}
value = awsrm_resource_group.main.name
}
value = awsrm_virtual_network.main.name
}
}
value = awsrm_network_interface.service.private_ip_address
}
value = awsrm_log_analytics_workspace.main.name
}
value = awsrm_key_vault.main.name
}
value = {
}
}
```

Secret values, preshared keys and private keys are not exposed as outputs.

4.19 Example Variable File

File: infrastructure/terraform.tfvars.example

```
project_name = "secure-multicloud"
environment = "lab"
aws_location = "REPLACE_WITH_APPROVED_AWS_REGION"
aws_subscription_id = "REPLACE_SECURELY"
aws_tenant_id    = "REPLACE_SECURELY"
administrator_cidr = "REPLACE_WITH_APPROVED_IP/32"
database_name        = "enterprise_lab"
database_username = "app_user"
# Do not commit actual secrets.
database_password = "SUPPLY_THROUGH_TF_VAR_database_password"
vpn_shared_key_1 = "SUPPLY_THROUGH_TF_VAR_vpn_shared_key_1"
vpn_shared_key_2 = "SUPPLY_THROUGH_TF_VAR_vpn_shared_key_2"
enable_guardduty = true
enable_defender = true
```

Secrets should instead be supplied in the shell:
export TF_VAR_database_password='REDACTED_STRONG_VALUE'
export TF_VAR_vpn_shared_key_1='REDACTED_STRONG_VALUE'
export TF_VAR_vpn_shared_key_2='REDACTED_STRONG_VALUE' On PowerShell:
$env:TF_VAR_database_password = "REDACTED_STRONG_VALUE"
$env:TF_VAR_vpn_shared_key_1 = "REDACTED_STRONG_VALUE"
$env:TF_VAR_vpn_shared_key_2 = "REDACTED_STRONG_VALUE"

4.20 Terraform Deployment Procedure

4.20.1 Initialise the backend and providers

cd infrastructure
The expected result is confirmation that the AWS backend and all required providers were initialised successfully.

4.20.2 Format and validate

terraform fmt -recursive
terraform validate
A successful validation confirms that the configuration is syntactically valid for the selected providers. It does not prove that cloud quotas, permissions and account-specific prerequisites are satisfied.

4.20.3 Generate the plan

-var-file="lab.tfvars" \
-out="multicloud.tfplan"
The plan must be reviewed for:
unintended public IP addresses;
permissive security rules;
deletion of existing resources;
unexpected Defender charges;
incorrect regions;
overlapping address spaces;
secret values shown in clear text;
provider replacements.

4.20.4 Apply

The VPN Gateway is likely to be the longest-running resource.

4.20.5 Reconcile generated VPN dependencies

Because the GCP VPN tunnel addresses are generated after the AWS public gateway IPs exist, a second plan may be required:
-var-file="lab.tfvars" \
-out="vpn-completion.tfplan"
The second pass completes resources whose values were unknown during the first planning stage.

4.20.6 Final drift check

Expected result:
No changes. Your infrastructure matches the configuration.
This result should be captured as evidence only after all intended manual changes have been imported or represented in code.
Screenshot placeholder 4.21

4.21 Post-Deployment Validation

4.21.1 GCP network validation

aws ec2 describe-vpcs \
--output table
aws ec2 describe-transit-gateway-attachments \
--filters \
--output table

4.21.2 AWS network validation

--resource-group "$(terraform output -raw aws_resource_group_name)" \
--name "$(terraform output -raw aws_vnet_name)" \
--output table

4.21.3 GCP-to-AWS application test

From the GCP application instance:
curl --fail --show-error \
http://10.121.10.10:8080/health
Expected structure:
{
"host": "awsservice",
"service": "aws-supporting-service",
"status": "healthy", "timestamp": "<ACTUAL_TIMESTAMP>"
}
The response must be generated from the actual environment.

4.21.4 Web application test

Expected result:
{
"service": "aws-application-tier",
"status": "healthy"
}

4.21.5 Cross-cloud service test through the web tier

This request traverses:
1. user to GCP load balancer;
2. load balancer to GCP web tier;
3. web tier to GCP application tier;
4. GCP application tier to Network Connectivity Center;
5. GCP VPN to AWS Site-to-Site VPN;
6. AWS VPC to the AWS supporting service.
Screenshot placeholder 4.23
Insert terminal screenshot showing a successful response from /aws-health.

4.22 Security-Control Validation During Implementation

4.22.1 Direct web-to-database denial

From the GCP web instance:
Expected result: timeout or connection failure.
A successful connection would indicate a defect in security-group or host configuration and must be corrected before Chapter Five.

4.22.2 AWS-to-database denial

From the AWS supporting VM:
nc -vz -w 5 10.181.40.X 5432
Expected result: failure. The actual private database address must not be published where unnecessary.

4.22.3 Public database exposure check

From an external test workstation:
nmap -Pn -p 5432 <DATABASE_PUBLIC_ADDRESS>
The database should not possess a public address. The test should therefore report that no direct public destination exists.

4.22.4 Secret-repository check

git grep -nEi \
Each result must be reviewed. Variable declarations are acceptable; actual secret values are not.

4.23 Cloud Monitoring Agent Configuration

The base implementation creates Cloud Monitoring log groups and permits the Compute Engine workload role to publish logs. The Cloud Monitoring Agent can be installed to collect application and operating-system telemetry.

4.23.1 Agent configuration

{
"agent": { "metrics_collection_interval": 60,
"run_as_user": "root"
},

```
"logs": {
"logs_collected": {
"files": {
"collect_list": [
{
"file_path": "/var/log/syslog",
"log_group_name": "/secure-multicloud-lab/system",
"log_stream_name": "{instance_id}/syslog",
"retention_in_days": 30
},
{
"file_path": "/var/log/nginx/access.log",
"log_group_name": "/secure-multicloud-lab/web",
"log_stream_name": "{instance_id}/access",
"retention_in_days": 30
}
]
}
}
},
"metrics": {
"namespace": "SecureMultiCloudLab",
"metrics_collected": { "mem": {
"measurement": [
"mem_used_percent"
],
"metrics_collection_interval": 60
},
"disk": {
"measurement": [
"used_percent"
],
"resources": [
"/"
],
"metrics_collection_interval": 60
}
}
}
}
```

The agent is started with:

```
sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
-a fetch-config \
-m ec2 \
-c file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json \
-s
```

The actual Cloud Monitoring Agent package-installation method varies by Ubuntu release and region and should be recorded during implementation.

4.24 Security Command Center and Defender Verification

4.24.1 Security Command Center detector status

```
DETECTOR_ID=$(aws guardduty list-detectors \
--query 'DetectorIds[0]' \
--output text)
aws guardduty get-detector \
--detector-id "$DETECTOR_ID" \
--output table
```

The detector should report ENABLED.

No malicious action should be performed merely to generate a finding. Security Command Center provides a sample-finding function that may be used where available and authorised.

4.24.2 Defender status

az security pricing list --output table
The output shows the configured Defender plans and their pricing tiers.

4.25 Implementation of Prowler and ScoutSuite

Although formal results are presented in Chapter Five, the tools are installed during implementation.

4.25.1 Python virtual environment

python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip

4.25.2 Prowler

pip install prowler prowler --version
GCP assessment:
mkdir -p reports/prowler/aws
prowler aws \
--output-directory reports/prowler/aws
AWS assessment depends on the Prowler version and supported authentication pattern. The exact command used must be taken from the installed tool's help output:
prowler aws --help

4.25.3 ScoutSuite

pip install scoutsuite
scout --version
GCP:
mkdir -p reports/scoutsuite/aws
scout aws --report-dir reports/scoutsuite/aws
AWS:
mkdir -p reports/scoutsuite/aws
scout aws --report-dir reports/scoutsuite/aws
The scanner identities should receive audit or read-only permissions, not resource- modification privileges.
Screenshot placeholder 4.24
Insert Prowler execution screenshot showing the tool version and completed GCP scan. Do not insert invented finding counts.
Screenshot placeholder 4.25
Insert ScoutSuite report dashboard showing the actual Google Cloud and AWS assessment summaries.

4.26 Implementation Challenges and Mitigation

4.26.1 Cross-provider dependency ordering

This circular operational dependency is mitigated through staged Terraform application. Unknown values are allowed to resolve during the first apply, after which a second plan completes dependent resources.

4.26.2 Long VPN Gateway provisioning time

AWS Site-to-Site VPN provisioning may take significantly longer than ordinary network resources. Interrupting Terraform can leave the resource in a transitional state.

4.26.3 BGP mismatch

Potential causes of a failed BGP session include:
duplicate or incorrect ASN;
wrong inside BGP address;
unsupported APIPA selection;
incorrect AWS local-network gateway definition;
route propagation not enabled;
tunnel not established;
security-association mismatch.
The administrator should first confirm that IPsec is established, then inspect BGP peer state and learned routes. BGP cannot establish across a tunnel that is not operational.

4.26.4 IPsec negotiation failure

Phase 1 failure normally indicates disagreement over IKE version, encryption, integrity, Di ie-Hellman group or authentication. Phase 2 failure commonly relates to IPsec algorithms, Perfect Forward Secrecy, security-association lifetime or traffic-selector behaviour.

The project uses matching explicit policies where supported and records final negotiated settings.

4.26.5 Terraform state exposure

Terraform state may contain shared keys, private keys and other sensitive attributes. The mitigation includes remote encrypted storage, private container access, AWS IAM, versioning and exclusion of state from Git.

A stronger implementation should avoid generating private keys in Terraform and rotate any laboratory secret that may have been exposed during debugging.

4.26.6 Entra federation prerequisites

SAML and SCIM configuration depends on administrative privileges and Workforce Identity Federation settings. The network implementation can proceed independently, while federation is completed after the core infrastructure is stable.

If SCIM cannot be enabled, manual user provisioning may be used for the proof of concept, but the limitation must be clearly reported.

4.26.7 Cost control

VPN Gateway, NAT Gateway, Network Connectivity Center, Defender plans and running virtual machines can incur recurring charges.

The following controls are implemented:
cloud budgets and billing alerts;
small VM sizes;
no production data;
immediate teardown after evidence collection;
limited log retention;
chargeable Defender plans controlled through a variable;
only one NAT Gateway in the lab;
no Cloud Interconnect or ExpressRoute.

4.26.8 Lab security versus production security

Certain choices are deliberately lab-specific:
one NAT Gateway;
HTTP listener where no domain and certificate are available;
one active instance per workload tier;
public AWS KMS and Secrets Manager endpoint during initial deployment;
generated SSH keys within Terraform;
relatively short log retention;
limited central SIEM integration.

These choices must not be presented as recommended production architecture. Their production alternatives are identified in the implementation discussion.

4.27 Implementation Evidence Register

The following evidence should be collected from the actual environment:

Evidence ID Required evidence

IE-01 Terraform and provider versions
IE-02 Successful bootstrap apply
IE-03 Google Cloud VPC and subnet structure
IE-05 AWS VPC and subnet structure
IE-06 AWS redundant Site-to-Site VPN
IE-07 GCP tunnel status
IE-08 AWS connection status
IE-09 BGP peer and learned-route status
IE-10 Successful GCP-to-AWS application call
IE-11 Failed unauthorised database-access attempt
IE-12 GCP Cloud Audit Logs enabled IE-13 Google Cloud VPC Flow Logs enabled
IE-14 Security Command Center enabled
IE-15 AWS Log Analytics receiving logs
IE-16 VPN diagnostics enabled

IE-17 AWS KMS and Secrets Manager controls
IE-18 AWS Security Hub and GuardDuty status
IE-19 Entra SAML configuration
IE-20 SCIM provisioning result
IE-21 Google Cloud Workforce Identity Federation assignment
IE-22 Final no-change Terraform plan
IE-23 Initial Prowler report
IE-24 Initial ScoutSuite report

Each screenshot should include a caption, date and brief interpretation. Identifiers and secrets should be concealed without obscuring the control being evidenced.

4.28 Resource Teardown

The environment must be destroyed only after Chapter Five testing evidence has been collected.

-destroy \
-var-file="lab.tfvars" \
-out="destroy.tfplan"

Some protected resources may intentionally resist destruction:
AWS KMS and Secrets Manager purge-protected objects;
Secrets Manager secrets with recovery windows;
KMS keys with deletion windows;
retained log records;
state-storage resources.

The bootstrap state should be destroyed only after the main infrastructure has been removed and the final state file has been archived securely.

4.29 Chapter Summary

This chapter presented the implementation of the secure AWS-GCP multi-cloud architecture designed in Chapter Three. The implementation followed the iterative design- and-development phase of Design Science Research Methodology, beginning with the administrative workstation, cloud authentication and Terraform state before progressing to networking, workloads, VPN connectivity, identity, encryption, monitoring and threat detection.

The GCP environment was implemented with a segmented VPC containing public, web, application, database, management and transit subnets. Separate route tables and VPC firewall rules restrict traffic according to the approved flow matrix. Google Cloud Network Connectivity Center acts as the central GCP routing hub, while the application and management subnets receive explicit routes towards AWS. The database subnet does not receive a general AWS or Internet route.

The GCP workload includes a public Application Load Balancer, a private web reverse proxy, a private application service and a private PostgreSQL database. The web tier cannot reach the database directly. The GCP application communicates with a supporting AWS service through the encrypted inter-cloud path.

The AWS environment was implemented with a hub VNet containing gateway, service, monitoring and management subnets. The AWS supporting VM uses a fixed private address and a system-assigned managed identity. AWS Network VPC Firewall Rules permit only the approved GCP application and performance-test flows.

The security implementation includes Google Cloud IAM roles, Google Cloud Workforce Identity Federation permission sets, Microsoft Entra groups, AWS IAM, Cloud KMS, GCP Secrets Manager, AWS KMS and Secrets Manager and AWS managed identities. Entra-to-GCP federation uses SAML for authentication and SCIM for lifecycle provisioning, with tenant-specific steps documented for manual completion.

GCP monitoring includes Cloud Audit Logs, encrypted Cloud Monitoring log groups, VPC Flow Logs, VPN alarms and Security Command Center. AWS monitoring includes Log Analytics, VPN Gateway diagnostics, AWS KMS and Secrets Manager diagnostics, the Amazon CloudWatch Agent, Data Collection Rules and AWS Security Hub and GuardDuty. Scanner installation procedures were provided for Prowler and ScoutSuite, although their findings will be reported only after actual execution.

The chapter deliberately uses screenshot placeholders because valid evidence must be generated from the deployed project environment. Fabricated tunnel states, security scores, log records and scan findings would undermine the professional and academic integrity of the project.

Transition to Chapter Five

Chapter Five Will Present The Testing Strategy, Actual System Outputs And Evaluation Results.

It will examine functional connectivity, network segmentation, Prowler and ScoutSuite findings, authentication controls, Google Cloud and AWS logs, inter-cloud latency, iperf3 throughput, tunnel-failure recovery and the extent to which each project objective was achieved. Where the actual measurements di er from the proposed thresholds, the chapter will report and analyse the variance rather than replacing it with assumed results.

Chapter Five
TESTING, RESULTS, AND EVALUATION

TESTING, RESULTS AND EVALUATION

5.0 Introduction

This chapter presents the testing methodology developed for the secure AWS-GCP multi- cloud architecture implemented in Chapter Four. It explains how the technical artefact is tested for functional correctness, security, network performance, operational visibility and resilience. The chapter also defines how the results are recorded, analysed and evaluated against the project aim and objectives.

The testing stage corresponds to the evaluation activity of Design Science Research Methodology. In DSRM, an artefact is not considered successful merely because it has been designed or deployed. Its usefulness must be demonstrated through systematic evaluation against the objectives established for the solution. The testing undertaken in this project therefore examines whether the implemented architecture behaves as intended under normal operation, unauthorised access attempts, performance measurement and controlled failure conditions.

The evaluation covers five broad areas:

1. functional testing of the AWS-GCP architecture and enterprise workload;
2. security testing of network segmentation, identity, encryption, logging and cloud configuration;
3. performance testing using ping, traceroute and iperf3;
4. resilience testing through controlled VPN and routing failures; and
5. evaluation of the project objectives against documented evidence.

Prowler and ScoutSuite are used for cloud-security posture assessment. Prowler supports provider-specific security assessments, while ScoutSuite gathers configuration data through cloud-provider APIs and highlights potential areas of exposure for manual analysis.

The performance evaluation uses ping for round-trip latency and packet-loss measurement, traceroute or tracepath for path observation, and iperf3 for active throughput testing. iperf3 is designed to measure maximum achievable bandwidth across an IP path and can report throughput, retransmissions, jitter and loss depending on the selected test protocol.

The chapter does not invent experimental results. Where actual cloud outputs have not yet been supplied, result fields are presented as controlled placeholders marked [INSERT ACTUAL RESULT]. These fields must be replaced with evidence obtained from the implemented environment before the final report is submitted. The interpretation framework is complete, but the conclusions must be updated to reflect the measurements actually obtained.

5.1 Purpose of the Evaluation

The purpose of the evaluation is to determine whether the implemented artefact adequately solves the practical problem identified in Chapter One. That problem concerned the security and operational weaknesses that can arise when Google Cloud and AWS are connected through ad hoc networking, inconsistent access controls and fragmented monitoring.

The evaluation therefore addresses more than basic connectivity. A VPN tunnel reporting an operational status is not sufficient evidence that the architecture is secure or effective. The architecture must also demonstrate that:

approved traffic can pass between the required components;
prohibited traffic is blocked;
management access is restricted to authorised identities;
inter-cloud traffic is encrypted;
configuration weaknesses are identified and remediated;
relevant events are recorded in Google Cloud and AWS;
latency and throughput are appropriate for the representative workload;
the architecture can recover from a controlled tunnel or route failure; and
the implementation remains consistent with the Terraform configuration.

The evaluation combines quantitative and qualitative evidence. Latency, packet loss, throughput, scanner counts and recovery time are quantitative measures. Route inspection, security-rule review, identity-federation verification and log analysis provide qualitative evidence that explains the measured outcomes.

5.2 Testing Methodology

Figure 5.1. Security, performance and resilience evaluation framework.

5.2.1 Evaluation approach

The project uses a mixed technical evaluation approach consisting of:
black-box testing;
configuration-based testing;
controlled white-box inspection;
active network measurement;
fault-injection testing; and
requirements-based evaluation.
Black-box testing examines externally observable behaviour. For example, a web request is submitted and the resulting application response is observed without relying on the internal implementation.
Configuration-based testing examines deployed cloud settings through provider APIs, Prowler, ScoutSuite and manual review. It determines whether services such as Cloud Audit Logs, Security Command Center, Amazon CloudWatch, AWS KMS and Secrets Manager and AWS Security Hub and GuardDuty are configured as intended.
White-box inspection examines Terraform files, route tables, VPC firewall rules, security groups, IAM policies and logs. It is used where understanding the configuration is necessary to explain the observed behaviour.
Active network measurement generates controlled traffic using ping, traceroute and iperf3. Fault injection introduces a limited and reversible failure, such as disabling one VPN connection, to observe whether connectivity recovers through an alternative path.

5.2.2 Testing sequence

The tests are performed in the following order:
Security and functional tests are completed before intensive performance and failover testing. This prevents invalid performance measurements being collected from an incorrectly routed or insecure environment.

5.2.3 Test environment controls

The following conditions must be recorded for every major test session:
Test condition Information to record
Test date and time Actual timestamp and timezone GCP Region Region used for Google Cloud resources
AWS Region Region used for AWS resources
GCP instance type Type and operating system
Amazon EC2 instance size Size and operating system
VPN condition Number of active tunnels and BGP state
Test source Private IP and workload tier
Test destination Private IP and workload tier
Tool version ping, traceroute, iperf3, Prowler or ScoutSuite version
Security-rule state Normal rules or temporary test rule
Concurrent traffic Whether other known workload traffic was active
Terraform state Whether the final plan reported no unexplained changes
This information is necessary because network results can be affected by region selection, virtual-machine capacity, current Internet conditions, encryption overhead and competing traffic.

5.2.4 Test evidence

Evidence should consist of:
terminal outputs saved as text files;
screenshots of relevant Google Cloud and AWS pages;
Terraform plan output;
Prowler and ScoutSuite reports;
Cloud Audit Logs and AWS Activity Log records;
Cloud Monitoring and Amazon CloudWatch metrics;
VPC Flow Logs and VPN Gateway diagnostic logs;
CSV result files;
chart images generated from recorded measurements; and
a test-execution log signed or dated by the researcher.
GCP Cloud Audit Logs records actions taken through the Google Cloud console, CLI, SDKs and APIs and is suitable for operational auditing and investigation. AWS Site-to-Site VPN can be monitored through Amazon CloudWatch metrics, diagnostic logs and BGP status views.

5.3 Test Objectives and Success Criteria

5.3.1 Consolidated test criteria

Test area Success criterion

Infrastructure Terraform completes without unresolved errors deployment
Configuration Final Terraform plan reports no unexplained drift consistency
AWS-GCP VPN Selected VPN connections report operational status
BGP Expected peers are connected and approved routes are learned
Web workload Application health endpoint returns HTTP 200
Inter-cloud service GCP application successfully calls AWS service
Database protection Database is not publicly accessible
Segmentation Web and unauthorised AWS sources cannot access the database
Identity federation Assigned user receives the correct temporary Google Cloud IAM role Unauthorised identity
Unassigned user cannot obtain the Google Cloud IAM role
Logging Relevant test events appear in the selected log sources
Prowler No unresolved critical or high findings within project scope
ScoutSuite No unresolved critical or high findings within project scope
Latency Average AWS-GCP round-trip time below 100 ms
Packet loss No sustained packet loss under normal test conditions
Throughput Measured and reported without unsupported claims
Failover Connectivity recovers through the remaining path or documented recovery procedure
Recovery time Measured RTO is recorded and assessed
Encryption IPsec/IKE status and configured cryptographic policies are verified
The requirement of zero unresolved high or critical findings must be interpreted carefully. A scanner may produce findings that are false positives, not applicable to the lab or caused by deliberate proof-of-concept limitations. Such findings may be classified as resolved only where the reason is documented and supported. They must not simply be deleted from the report.

5.4 Functional Testing

5.4.1 Purpose

Functional testing confirms that the principal system components operate according to the architecture. It addresses Terraform deployment, cloud-resource availability, application communication, routing, database access and monitoring.

5.4.2 Functional test cases

Test Test case Method Expected Actual Status ID result result
FT- Terraform terraform validate Configuration [INSERT] [PASS/FAIL] 02 validation is valid
FT- GCP VPN gcloud CLI/console Selected [INSERT] [PASS/FAIL] 05 status tunnels operational
FT- AWS VPN AWS CLI/portal Connections [INSERT] [PASS/FAIL] 06 status show Connected
FT- BGP peer AWS and Google Cloud route Peers [INSERT] [PASS/FAIL] 07 status inspection established
FT- GCP route Network Connectivity Center route AWS prefix [INSERT] [PASS/FAIL] 08 learning table visible
FT- AWS route VPN Gateway BGP GCP prefix [INSERT] [PASS/FAIL] 09 learning routes visible FT- GCP curl /health HTTP 200 [INSERT] [PASS/FAIL] 10 application response health
FT- AWS curl HTTP 200 [INSERT] [PASS/FAIL] 11 service 10.121.10.10:8080/health response health
FT- End-to-end curl /aws-health Valid AWS [INSERT] [PASS/FAIL] 12 cloud call response returned through GCP web path
FT- Application- PostgreSQL client test Authorised [INSERT] [PASS/FAIL] 13 to-database connection succeeds
FT- Google Cloud logs Cloud Monitoring/Cloud Audit Logs Known event [INSERT] [PASS/FAIL] 14 generated review appears
FT- AWS logs Log Analytics review Known event [INSERT] [PASS/FAIL] 15 generated appears

5.4.3 End-to-end application test

The following command is executed from an approved external test workstation:
curl --fail --show-error \
"http://<GCP-LOAD-BALANCER-DNS>/aws-health"

The expected response should confirm that the request reached the GCP web tier, passed to the GCP application tier, crossed the encrypted VPN and received a valid response from the AWS supporting service.

Result record

Test date: [INSERT]

GCP endpoint: [REDACTED OR INSERT APPROVED VALUE] HTTP status: [INSERT]

Total request duration: [INSERT]

AWS service response: [INSERT]

Outcome: [PASS/FAIL]

Screenshot placeholder 5.1

Insert a terminal screenshot showing the actual successful end-to-end application response. The timestamp and returned service identifier should remain visible.

5.4.4 Functional result summary

Measure	Result
Total functional tests executed	[INSERT]
Passed	[INSERT]
Failed	[INSERT]
Pass rate	[INSERT]%
Tests requiring retest	[INSERT]

The functional pass rate is calculated as:

A failed functional test must be analysed before performance testing continues. For example, a failure of the AWS service health check could result from the application service, NSG, route table, BGP or VPN. It should not immediately be attributed to VPN failure without examining the complete path.

5.5 Security Testing

5.5.1 Security-testing objectives

The security tests determine whether the implemented controls achieve the following outcomes:

unauthorised network paths are blocked;

legitimate application paths remain available;

the database is not publicly exposed;

administrative access is restricted;

cloud credentials are not embedded in source files;

the inter-cloud connection uses IPsec;

privileged actions are logged;

cloud-security services are enabled;

high-risk configuration findings are identified and remediated;

federated identities receive only their assigned permissions.

5.5.2 Security-testing methods

The security assessment combines:

1. manual port and reachability tests;
2. security-group and NSG inspection;
3. route-table inspection;
4. Prowler assessment;
5. ScoutSuite assessment;
6. IAM and Entra sign-in tests;
7. Cloud Audit Logs and AWS log review;
8. Terraform source and state-handling review;
9. encryption configuration verification; and
10. controlled negative testing.

The testing remains within the project-owned environment. It does not include destructive exploitation, denial-of-service testing or attacks against GCP, Microsoft or third-party infrastructure.

5.6 Network Segmentation Testing

5.6.1 Segmentation test matrix

Test ID	Source	Destination	Port	Expected outcome	Actual outcome	Status
ST-01	Internet	GCP load balancer	80/443	Permit approved entry	[INSERT]	[PASS/FAIL]

ST- Internet GCP 5432 Deny/no [INSERT] [PASS/FAIL] 02 database public address
ST- GCP load GCP web tier 80 Permit [INSERT] [PASS/FAIL] 03 balancer
ST- GCP web tier GCP 8080 Permit [INSERT] [PASS/FAIL] 04 application tier
ST- GCP web tier GCP 5432 Deny [INSERT] [PASS/FAIL] 05 database
ST- GCP GCP 5432 Permit [INSERT] [PASS/FAIL] 06 application database tier
ST- GCP AWS service 8080 Permit [INSERT] [PASS/FAIL] 07 application tier
ST- GCP AWS iperf3 5201 Permit [INSERT] [PASS/FAIL] 08 management server during host test ST- AWS service GCP 5432 Deny [INSERT] [PASS/FAIL] 09 subnet database
ST- AWS service GCP web tier 22 Deny [INSERT] [PASS/FAIL] 10 subnet
ST- Unapproved GCP 22 Deny [INSERT] [PASS/FAIL] 11 public source management service
ST- AWS GCP Any Deny [INSERT] [PASS/FAIL] 12 unapproved application unapproved subnet tier port

5.6.2 Web-to-database denial

The following command is executed from the GCP web instance:
nc -vz -w 5 <GCP-DATABASE-PRIVATE-IP> 5432
Expected result:
Connection timed out
or:
Connection refused
A timeout normally indicates that a network control silently discarded the traffic. A refusal can indicate that the traffic reached the destination but no service accepted it. The result must therefore be interpreted alongside VPC Flow Logs and database host logs.

5.6.3 AWS-to-database denial

From the AWS service VM:
nc -vz -w 5 <GCP-DATABASE-PRIVATE-IP> 5432
The connection should fail. The relevant VPC Flow Log entry should be examined for a rejected flow where the network path reaches the Google Cloud VPC.
Example Cloud Monitoring Logs Insights query:
fields @timestamp, srcAddr, dstAddr, srcPort, dstPort, action | filter dstPort = 5432
| sort @timestamp desc
| limit 50

5.6.4 Segmentation result analysis

The segmentation design is considered effective where every approved path succeeds and every prohibited path fails.
A denied connection alone does not prove that all security layers are correct. The failure may arise from a stopped service instead of a security rule. For this reason, the corresponding permitted path must also be tested. For example, application-to-database access should succeed while web-to-database access fails.
Segmentation result summary
Outcome Count
Expected permits that succeeded [INSERT]
Expected permits that failed [INSERT]
Expected denials that were blocked [INSERT]
Expected denials that succeeded unexpectedly [INSERT]
Overall segmentation effectiveness [INSERT]%
Segmentation effectiveness may be calculated as:
Any prohibited connection that succeeds is a critical design failure requiring immediate remediation and retesting.

5.7 Identity and Access-Control Testing

5.7.1 Federation test cases

Test Test Expected result Actual Status ID result
IAM- Assigned Entra cloud Approved role [INSERT] [PASS/FAIL] 01 administrator signs in to displayed GCP
IAM- Assigned security auditor Read-only security role [INSERT] [PASS/FAIL] 02 signs in displayed

IAM- Unassigned Entra user Access denied [INSERT] [PASS/FAIL] 03 attempts access
IAM- Security auditor attempts Action denied [INSERT] [PASS/FAIL] 04 resource creation
IAM- Network administrator views View succeeds [INSERT] [PASS/FAIL] 05 VPN
IAM- Network administrator Action denied [INSERT] [PASS/FAIL] 06 attempts unrelated IAM change
IAM- Disabled Entra user Access denied after [INSERT] [PASS/FAIL] 07 attempts new session synchronisation
IAM- Privileged sign-in without Authentication blocked [INSERT] [PASS/FAIL] 08 required MFA
IAM- Federated Google Cloud federated session Temporary credentials [INSERT] [PASS/FAIL] 09 inspected confirmed IAM- Entra and GCP audit records Corresponding sign-in [INSERT] [PASS/FAIL] 10 reviewed records found

5.7.2 Temporary GCP credential verification

After federated sign-in, the session identity is examined:
aws sts get-caller-identity
The resulting ARN should indicate an assumed role or Workforce Identity Federation session rather than a permanent IAM user.

5.7.3 Least-privilege test

A read-only security auditor may attempt an unauthorised operation, such as creating a VPC firewall rule:
aws ec2 create-security-group \
--group-name unauthorised-test \
--description "This operation should fail" \
--vpc-id <PROJECT-VPC-ID>
Expected result:
AccessDenied or UnauthorizedOperation
No privilege escalation attempt is performed beyond ordinary API authorisation testing.

5.7.4 Identity result interpretation

The identity design is effective where:
authorised users obtain only assigned roles;
unauthorised users cannot obtain cloud sessions;
read-only users cannot make changes;
privileged roles require MFA;
sign-ins are visible in Entra and Google Cloud logs;
sessions are temporary.
A successful federated login is not enough to demonstrate least privilege. Role permissions must also be tested.

5.8 Encryption Testing

5.8.1 Inter-cloud VPN encryption

The Google Cloud and AWS consoles or CLI outputs are reviewed to verify:
tunnel status;
IKE version;
IPsec policy;
encryption algorithm;
integrity algorithm;
Di ie-Hellman group;
Perfect Forward Secrecy setting;
BGP state.
Google Cloud HA VPN encrypts traffic using IPsec tunnels, while AWS Site-to-Site VPN provides site-to-site encrypted connectivity and exposes diagnostic and BGP information through AWS monitoring facilities.

5.8.2 Encryption test table

Control Expected setting Actual Status setting
VPN type Route-based IPsec [INSERT] [PASS/FAIL]
IKE version IKEv2 [INSERT] [PASS/FAIL]
Phase 1 encryption AES-256 or selected [INSERT] [PASS/FAIL] compatible equivalent
Phase 1 integrity SHA-256 or selected [INSERT] [PASS/FAIL] compatible equivalent Phase 2 encryption AES-256 or selected [INSERT] [PASS/FAIL] compatible equivalent
PFS Enabled where configured [INSERT] [PASS/FAIL]

GCP EBS encryption Enabled using KMS [INSERT] [PASS/FAIL]
Cloud Monitoring log-group KMS enabled [INSERT] [PASS/FAIL] encryption
Cloud Audit Logs bucket Cloud KMS [INSERT] [PASS/FAIL] encryption
AWS KMS and Secrets Manager Soft delete and purge [INSERT] [PASS/FAIL] protection
protection
Amazon EC2 instance disk encryption Platform or customer-managed [INSERT] [PASS/FAIL] encryption

5.8.3 Packet-path interpretation

Packet capture from a workload interface will show the original packet before it enters the managed cloud VPN gateway, not necessarily the encrypted Internet-side encapsulation. The most defensible evidence is therefore the provider tunnel configuration and operational IPsec/IKE status rather than claiming to have captured encrypted gateway traffic from an Compute Engine or Amazon EC2 instance.

5.9 Logging and Monitoring Testing

5.9.1 Google Cloud logging tests

GCP Cloud Audit Logs is tested by performing a known, harmless action and then locating the event. Cloud Audit Logs records actions performed through the console, CLI and service APIs.
Example action:
aws ec2 describe-vpcs
Example event search: aws cloudtrail lookup-events \
--lookup-attributes AttributeKey=EventName,AttributeValue=DescribeVpcs \
--max-results 10
VPC Flow Logs are tested by generating one permitted and one prohibited flow and verifying the corresponding ACCEPT or REJECT record. Security Command Center is tested by confirming that the detector is enabled; malicious traffic is not generated solely for the project. Security Command Center uses GCP data sources, including VPC network telemetry, to identify supported threat patterns.

5.9.2 AWS logging tests

The following AWS events are generated and reviewed:
resource-group query or authorised resource update;
VPN connection status query;
BGP status query;
AWS KMS and Secrets Manager secret read using the managed identity;
permitted and denied service traffic;
Amazon EC2 instance service start or restart.
AWS Site-to-Site VPN monitoring includes metrics, resource logs, Activity Log information and BGP status.
Example Log Analytics query:
AWSDiagnostics
| where ResourceType has "VIRTUALNETWORKGATEWAYS"
| sort by TimeGenerated desc
| take 100
AWS KMS and Secrets Manager query:
AWSDiagnostics
| where ResourceType == "VAULTS"
| sort by TimeGenerated desc | take 100

5.9.3 Logging test cases

Test ID Event generated Expected log Actual Status source result
LOG- GCP API query Cloud Audit Logs [INSERT] [PASS/FAIL]
LOG- Permitted GCP application VPC Flow Logs [INSERT] [PASS/FAIL] 02 flow
LOG- Denied GCP database flow VPC Flow Logs [INSERT] [PASS/FAIL]
LOG- GCP application event Cloud Monitoring [INSERT] [PASS/FAIL]
LOG- AWS resource query Activity Log [INSERT] [PASS/FAIL]
LOG- AWS VPN event VPN diagnostics [INSERT] [PASS/FAIL]
LOG- AWS KMS and Secrets Manager access AWS KMS and Secrets Manager [INSERT]
[PASS/FAIL] 07 diagnostics
LOG- Entra sign-in Entra sign-in log [INSERT] [PASS/FAIL]
LOG- Security Command Center detector status Security Command Center [INSERT] [PASS/FAIL]
09 console/API
LOG- Defender recommendation AWS Security Hub and GuardDuty [INSERT] [PASS/FAIL]

5.9.4 Monitoring result analysis

Monitoring is considered effective where the researcher can associate a known test action with a corresponding event record. Merely enabling a log source is not sufficient. The record must be retrievable and contain enough information to identify the time, resource, action and outcome.

5.10 Prowler Security Assessment

5.10.1 Purpose of Prowler

Prowler is used to perform an automated assessment of Google Cloud and AWS configurations against supported security checks and compliance frameworks. Current Prowler CLI usage requires the target provider to be specified, such as aws or aws.

The purpose of the Prowler test is not to obtain a perfect numerical score. It is to identify configuration weaknesses, classify their relevance, remediate valid findings and demonstrate improvement between the initial and final assessments.

5.10.2 Prowler test preparation

The following information must be recorded:

Prowler version: [INSERT]
Execution date: [INSERT]
Google Cloud project scope: [INSERT/REDACT]
AWS account scope: [INSERT/REDACT]
GCP Regions assessed: [INSERT]
AWS Regions assessed: [INSERT]
Compliance framework: [INSERT]
Output formats: CSV, JSON-OCSF and HTML, as applicable

The scanner identity should use read-only security-audit permissions. It should not be granted administrator access merely to avoid permission errors.

```
mkdir -p reports/prowler/aws-initial prowler aws \
--output-directory reports/prowler/aws-initial
```

Before the final command is used, the installed version's help output should be reviewed:

```
prowler aws --help
```

5.10.4 AWS Prowler execution

```
mkdir -p reports/prowler/aws-initial
prowler aws \
--output-directory reports/prowler/aws-initial
```

The exact authentication switches must match the installed Prowler version and approved AWS authentication method.

5.10.5 Prowler initial-result table

Provider	Critical	High	Medium	Low	Informational	Total
GCP initial scan	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]
AWS initial	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT] scan

5.10.6 Finding-classification method

Each finding is classified into one of four categories:

Classification	Meaning
Valid and in scope	A genuine weakness affecting the implemented artefact
Valid but accepted lab constraint	Genuine finding caused by a documented proof-of-concept limitation
False positive	Tool conclusion is not supported when the complete configuration is examined
Not applicable	The tested service or requirement is outside the defined project scope

5.10.7 Prowler remediation register

Finding ID	Provider	Severity	Finding	Classification	Remediation	Retest outcome
PR-01	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]
PR-02	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]
PR-03	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]
PR-04	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]
PR-05	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]	[INSERT]

5.10.8 Final Prowler scan

Following remediation:
mkdir -p reports/prowler/aws-final
mkdir -p reports/prowler/aws-final
prowler aws \
--output-directory reports/prowler/aws-final
prowler aws \
--output-directory reports/prowler/aws-final

5.10.9 Prowler before-and-after results

Provider Stage Critical High Medium Low Total failed checks GCP Initial [INSERT] [INSERT] [INSERT] [INSERT] [INSERT]
GCP Final [INSERT] [INSERT] [INSERT] [INSERT] [INSERT]
AWS Initial [INSERT] [INSERT] [INSERT] [INSERT] [INSERT]
AWS Final [INSERT] [INSERT] [INSERT] [INSERT] [INSERT]
The remediation rate is calculated as:
Figure 5.1: Prowler findings before and after remediation
Insert a clustered column chart based on the actual values in the preceding table. The chart should compare initial and final Critical, High, Medium and Low findings separately for Google Cloud and AWS.

5.11 ScoutSuite Security Assessment

5.11.1 Purpose of ScoutSuite

ScoutSuite provides an additional multi-cloud configuration assessment. It uses cloud- provider APIs to collect configuration information and produces a navigable report highlighting areas that require security review.
Using both Prowler and ScoutSuite improves coverage, but duplicate findings must not be counted as separate vulnerabilities without analysis.

5.11.2 ScoutSuite execution

GCP:
mkdir -p reports/scoutsuite/aws-initial
scout aws \
--report-dir reports/scoutsuite/aws-initial AWS:
mkdir -p reports/scoutsuite/aws-initial
scout aws \
--report-dir reports/scoutsuite/aws-initial
The exact authentication method should be recorded.

5.11.3 ScoutSuite result table

Provider Danger Warning Good/Informational Total rules evaluated
GCP initial [INSERT] [INSERT] [INSERT] [INSERT]
AWS initial [INSERT] [INSERT] [INSERT] [INSERT]
GCP final [INSERT] [INSERT] [INSERT] [INSERT]
AWS final [INSERT] [INSERT] [INSERT] [INSERT]
ScoutSuite's category names and risk ratings must be reported exactly as shown by the installed version.

5.11.4 Cross-tool finding reconciliation

Control area Prowler ScoutSuite Duplicate? Final finding finding treatment
Public network [INSERT] [INSERT] [YES/NO] [INSERT] exposure
IAM privilege [INSERT] [INSERT] [YES/NO] [INSERT]
Logging [INSERT] [INSERT] [YES/NO] [INSERT]
Encryption [INSERT] [INSERT] [YES/NO] [INSERT] Key management [INSERT] [INSERT] [YES/NO] [INSERT]
Network [INSERT] [INSERT] [YES/NO] [INSERT] segmentation
The cross-tool analysis prevents inflated findings and identifies controls detected by only one scanner.
Figure 5.2: Final security findings by assessment tool
Insert a horizontal bar chart comparing final valid findings from Prowler and ScoutSuite by control category.

5.12 Performance-Testing Methodology

5.12.1 Performance objectives

The performance evaluation answers three questions:

1. What round-trip latency is observed between Google Cloud and AWS?
2. What TCP and, where appropriate, UDP throughput is achievable across the VPN?
3. Is the observed performance suitable for the representative enterprise workload?

The test is not intended to establish universal AWS-GCP performance. It evaluates one lab environment under documented conditions.

5.12.2 Performance variables

The dependent measurements are:
round-trip latency in milliseconds;
packet loss in percentage;
TCP throughput in megabits per second;
retransmissions;
UDP throughput where tested;
UDP jitter in milliseconds;
UDP datagram loss;
application-response time.
The controlled or recorded conditions include:
GCP Region;
AWS Region;
VM sizes;
operating systems;
number of active tunnels;
test direction;
iperf3 duration;
number of parallel streams;
TCP window or default tuning;
time of day;
background workload.

5.13 Ping Latency Testing

5.13.1 Purpose of ping testing

Ping uses Internet Control Message Protocol echo requests and replies to measure round- trip time and packet loss. The results provide a baseline view of network responsiveness.

Ping does not measure application latency, database performance or maximum throughput. ICMP traffic may also be treated differently from application traffic. It is therefore supplemented with HTTP and iperf3 measurements.

5.13.2 Test configuration

A minimum of five repeated ping sets should be collected in each direction. Each set should contain enough packets to reduce the influence of one abnormal response.

GCP to AWS:
ping -c 100 -i 0.2 10.121.10.10 \
| tee reports/performance/ping-aws-to-aws-run-01.txt AWS to GCP:
ping -c 100 -i 0.2 <GCP-APPLICATION-PRIVATE-IP> \
| tee reports/performance/ping-aws-to-aws-run-01.txt

Where ICMP is not permitted by the security design, a temporary, source-specific rule may be enabled for the test and removed immediately afterwards. The rule change must be recorded.

5.13.3 Ping result table

Directio n	Ru n	Packet s sent	Packet s receive d	Loss %	Minimu m RTT	Averag e RTT	Maximu m RTT	Standar deviatio n	n	n	s	m
GCP → AWS	1	[INSERT T]	[INSERT T]	[INSERT] ms	[INSERT T] ms	[INSERT] ms	[INSERT T] ms	[INSERT] ms				
GCP → AWS	2	[INSERT T]	[INSERT T]	[INSERT] ms	[INSERT T] ms	[INSERT] ms	[INSERT T] ms	[INSERT] ms				
GCP → AWS	3	[INSERT T]	[INSERT T]	[INSERT] ms	[INSERT T] ms	[INSERT] ms	[INSERT T] ms	[INSERT] ms				
GCP → AWS	4	[INSERT T]	[INSERT T]	[INSERT] ms	[INSERT T] ms	[INSERT] ms	[INSERT T] ms	[INSERT] ms				
GCP → AWS	5	[INSERT T]	[INSERT T]	[INSERT] ms	[INSERT T] ms	[INSERT] ms	[INSERT T] ms	[INSERT] ms				

AWS → 1 [INSER [INSER [INSERT] [INSERT [INSER [INSERT [INSERT GCP T] T] %] ms T] ms] ms] ms
AWS → 2 [INSER [INSER [INSERT] [INSERT [INSER [INSERT [INSERT GCP T] T] %] ms T] ms] ms] ms AWS → 3 [INSER [INSER [INSERT] [INSERT [INSER [INSERT [INSERT GCP T] T] %] ms T] ms] ms] ms
AWS → 4 [INSER [INSER [INSERT] [INSERT [INSER [INSERT [INSERT GCP T] T] %] ms T] ms] ms] ms
AWS → 5 [INSER [INSER [INSERT] [INSERT [INSER [INSERT [INSERT GCP T] T] %] ms T] ms] ms] ms

5.13.4 Aggregate latency

Direction Mean of run Median Lowest run Highest run Overall averages average average packet loss
GCP → [INSERT] ms [INSERT] [INSERT] ms [INSERT] ms [INSERT]% AWS ms
AWS → [INSERT] ms [INSERT] [INSERT] ms [INSERT] ms [INSERT]% GCP ms

The project's latency criterion is:
L¯<100 ms\bar{L} < 100\text{ ms}L¯<100 ms
where L¯\bar{L}L¯ is the arithmetic mean of the average round-trip times from the repeated runs.
The arithmetic mean is:
where LiL_iLi is the average latency from test run iii.

5.13.5 Latency chart

Figure 5.3: Average AWS-GCP round-trip latency by test run
Insert a two-series line chart. The horizontal axis should show Runs 1-5. The vertical axis should show average latency in milliseconds. One series should represent GCP-to-AWS traffic and the other AWS-to-GCP traffic. Add a horizontal reference line at 100 ms.

5.13.6 Latency result interpretation

The latency criterion is achieved where the overall mean remains below 100 ms. The analysis should also consider variation. A mean below 100 ms can conceal unstable behaviour if occasional values are much higher.
The discussion should therefore address:
minimum, average and maximum values;
di erence between test directions;
packet loss;
standard deviation;
time-based variation;
region distance;
VPN encryption overhead;
VM processing limits.
Result-analysis paragraph template
The mean GCP-to-AWS round-trip latency was [INSERT] ms, while the mean AWS-to- GCP latency was [INSERT] ms. Both/one/neither direction remained below the predefined threshold of 100 ms. The di erence of [INSERT] ms between the two directions was [small/material] and may be associated with route asymmetry, transient Internet conditions, gateway processing or instance scheduling. Packet loss was [INSERT]%, indicating [INTERPRETATION]. The result therefore [supports/does not support] the performance requirement NFR-P01.

5.14 Traceroute Testing

5.14.1 Purpose

Traceroute is used to observe the logical path towards the remote private endpoint and identify where responses are returned or suppressed. In a managed IPsec VPN, provider internal hops may not respond, and private gateway details may be hidden. Asterisks do not necessarily indicate packet loss or failure.
Linux command:
traceroute -n 10.121.10.10 Alternative:
tracepath 10.121.10.10
AWS-to-GCP:
traceroute -n <GCP-APPLICATION-PRIVATE-IP>

5.14.2 Traceroute result table

Direction Observed Non-responding Destination Interpretation responding hops hops reached

GCP →	[INSERT]	[INSERT]	[YES/NO]	[INSERT] AWS
AWS →	[INSERT]	[INSERT]	[YES/NO]	[INSERT] GCP

5.14.3 Traceroute interpretation

Traceroute should not be used to claim visibility of the provider's complete Internet path. The cloud-managed VPN gateway may encapsulate the traffic after it leaves the workload network, and many infrastructure routers do not return TTL-expiry messages.

The principal value of the test is confirming that the destination is reached through the private network path and identifying obvious routing loops or premature termination.

5.15 iperf3 Throughput Testing

5.15.1 Purpose

iperf3 actively measures network throughput between a client and server. It supports TCP and UDP measurements and reports values such as bandwidth, retransmissions, jitter and datagram loss, depending on the test mode.

The Amazon EC2 instance operates the iperf3 server:

The GCP management or application host operates the client.

5.15.2 TCP test scenarios

Test	Direction	Streams	Duration	Purpose
TCP-01	GCP → AWS	1	30 seconds	Single-stream baseline
TCP-02	GCP → AWS	4	30 seconds	Parallel-stream throughput
TCP-03	GCP → AWS	8	30 seconds	Higher-concurrency throughput
TCP-04	AWS → GCP	1	30 seconds	Reverse baseline
TCP-05	AWS → GCP	4	30 seconds	Reverse parallel streams
TCP-06	AWS → GCP	8	30 seconds	Reverse higher concurrency

5.15.3 TCP commands

Single stream:

```
iperf3 \
-c 10.121.10.10 \
-t 30 \
-O 3 \
--get-server-output \
--json \
> reports/performance/tcp-aws-aws-p1-run01.json
```

Four streams:

```
iperf3 \
-c 10.121.10.10 \
-P 4 \
-t 30 \
-O 3 \
--get-server-output \ --json \
> reports/performance/tcp-aws-aws-p4-run01.json
```

Reverse test:

```
iperf3 \
-c 10.121.10.10 \
-R \
-P 4 \
-t 30 \
-O 3 \
--get-server-output \
--json \
> reports/performance/tcp-aws-aws-p4-run01.json
```

The -O 3 option omits the initial three seconds from normal reporting to reduce the influence of TCP startup. The -P option uses parallel streams. Parallel streams can improve utilisation of a high-bandwidth, higher-latency path but do not necessarily represent normal application behaviour.

5.15.4 TCP result table

Direction	Streams	Run	Sender throughput	Receiver throughput	Retransmissions	
GCP →	1	1	[INSERT] Mbps	[INSERT] Mbps	[INSERT]	AWS

GCP → 1 2 [INSERT] Mbps [INSERT] Mbps [INSERT] AWS
GCP → 1 3 [INSERT] Mbps [INSERT] Mbps [INSERT] AWS GCP → 4 1 [INSERT] Mbps [INSERT] Mbps [INSERT] AWS
GCP → 4 2 [INSERT] Mbps [INSERT] Mbps [INSERT] AWS
GCP → 4 3 [INSERT] Mbps [INSERT] Mbps [INSERT] AWS
GCP → 8 1 [INSERT] Mbps [INSERT] Mbps [INSERT] AWS
AWS → 1 1 [INSERT] Mbps [INSERT] Mbps [INSERT] GCP
AWS → 4 1 [INSERT] Mbps [INSERT] Mbps [INSERT] GCP
AWS → 8 1 [INSERT] Mbps [INSERT] Mbps [INSERT] GCP
All planned repetitions should be added to the final table.

5.15.5 UDP test scenarios

UDP testing is optional but useful for measuring jitter and loss at controlled o ered rates.
Example 10 Mbps test:

```
iperf3 \
-c 10.121.10.10 \
-u \
-b 10M \
-t 30 \
--json \
> reports/performance/udp-10m-run01.json
```

Example 50 Mbps test:

```
iperf3 \
-c 10.121.10.10 \
-u \
-b 50M \
-t 30 \
--json \
> reports/performance/udp-50m-run01.json
```

The o ered rate should be increased cautiously. The project should not generate excessive traffic that could breach cloud-provider rules, affect other users or produce avoidable charges.

5.15.6 UDP result table

Direction	O ered	Achieved	Jitter	Lost	Loss rate rate	throughput	datagrams
GCP →	10 Mbps	[INSERT]	[INSERT]	[INSERT]	[INSERT]% AWS		ms
GCP →	25 Mbps	[INSERT]	[INSERT]	[INSERT]	[INSERT]% AWS		ms
GCP →	50 Mbps	[INSERT]	[INSERT]	[INSERT]	[INSERT]% AWS		ms
AWS →	10 Mbps	[INSERT]	[INSERT]	[INSERT]	[INSERT]% GCP		ms
AWS →	25 Mbps	[INSERT]	[INSERT]	[INSERT]	[INSERT]% GCP		ms
AWS →	50 Mbps	[INSERT]	[INSERT]	[INSERT]	[INSERT]% GCP		ms

5.15.7 Throughput aggregation

Direction	Streams	Average receiver	Minimum	Maximum	Average throughput	retransmissions
GCP →	1	[INSERT] Mbps	[INSERT]	[INSERT]	[INSERT] AWS	
GCP →	4	[INSERT] Mbps	[INSERT]	[INSERT]	[INSERT] AWS	
GCP →	8	[INSERT] Mbps	[INSERT]	[INSERT]	[INSERT] AWS	
AWS →	1	[INSERT] Mbps	[INSERT]	[INSERT]	[INSERT] GCP	
AWS →	4	[INSERT] Mbps	[INSERT]	[INSERT]	[INSERT] GCP	
AWS →	8	[INSERT] Mbps	[INSERT]	[INSERT]	[INSERT] GCP	

5.15.8 Throughput chart

Figure 5.4: Average TCP throughput by direction and parallel-stream count
Insert a grouped column chart. The horizontal axis should show 1, 4 and 8 parallel streams. The vertical axis should show average receiver throughput in Mbps. The two series should represent GCP-to-AWS and AWS-to-GCP tests.

5.15.9 Throughput analysis

The measured throughput should be compared with the lowest likely capacity in the path, including:
source VM network capability;
destination VM network capability;
GCP VPN limits;
AWS Site-to-Site VPN SKU;
Internet conditions;

TCP window growth;
round-trip latency;
encryption processing;
test duration;
number of streams.
iperf guidance notes that TCP throughput is influenced by the bandwidth-delay product and TCP window behaviour.
A result below the advertised gateway capacity does not automatically indicate a faulty VPN. Small VM sizes often become the limiting factor before the gateway.
Throughput-analysis paragraph template
The highest mean receiver throughput was [INSERT] Mbps, recorded in the [INSERT DIRECTION] direction using [INSERT] parallel streams. Single-stream throughput was [INSERT] Mbps, increasing/decreasing to [INSERT] Mbps with four streams. This pattern indicates [INTERPRETATION]. Retransmissions averaged [INSERT], suggesting [LOW/MODERATE/HIGH] TCP loss or congestion. The result was below the theoretical gateway capacity, but this di erence is consistent/inconsistent with the selected VM sizes, measured latency and encryption overhead. The architecture was therefore considered [SUITABLE/UNSUITABLE] for the project's lightweight representative workload.

5.16 Application-Response Testing

5.16.1 Purpose

Network latency does not directly equal application-response time. The end-to-end application test measures the time required for a request to pass through the GCP entry point, web tier, application tier and AWS supporting service.

5.16.2 curl timing command

curl --silent --output /dev/null \
--write-out \ 'dns=%{time_namelookup}
connect=%{time_connect}
first_byte=%{time_starttransfer}
total=%{time_total}
status=%{http_code}\n' \
"http://<GCP-LOAD-BALANCER-DNS>/aws-health"
A minimum of 30 requests should be recorded.

5.16.3 Application-response table

Request Connection time Time to first byte Total response time HTTP status

1	[INSERT]	[INSERT]	[INSERT]	[INSERT]
2	[INSERT]	[INSERT]	[INSERT]	[INSERT]
3	[INSERT]	[INSERT]	[INSERT]	[INSERT]
…	…	…	…	…
30	[INSERT]	[INSERT]	[INSERT]	[INSERT]

5.16.4 Aggregate result

Metric Result
Mean response time [INSERT] ms
Median response time [INSERT] ms
95th percentile [INSERT] ms
Minimum [INSERT] ms
Maximum [INSERT] ms Successful response rate [INSERT]%
Figure 5.5: End-to-end application-response time
Insert a line chart of total response time for the 30 requests. Add a second line or annotation for the mean response time.

5.17 Failover and Resilience Testing

5.17.1 Purpose

The resilience test determines whether the architecture can continue or recover following the loss of one inter-cloud VPN path.
GCP VPN connections provide multiple tunnel endpoints, and AWS Site-to-Site VPN can expose BGP peer status and learned route information for monitoring and troubleshooting.

The test does not assume that redundancy works merely because multiple tunnels exist. Recovery must be observed and measured.

5.17.2 Recovery Time Objective

The measured recovery time is:
T_first failure is the timestamp of the first failed probe after the selected path is disabled.
T_stable restoration is the timestamp at which successful responses resume and remain stable for the defined confirmation period.

5.17.3 Continuous probe

The following script records one application probe each second:

```
#!/usr/bin/env bash
set -euo pipefail
TARGET="http://10.121.10.10:8080/health" OUTPUT="reports/failover/failover-probes.csv"
echo "timestamp,http_code,total_time" > "$OUTPUT"
while true; do
RESULT=$(curl \
--silent \
--output /dev/null \
--max-time 2 \
--write-out "%{http_code},%{time_total}" \
"$TARGET" || echo "000,2.000")
printf '%s,%s\n' \
"$(date --iso-8601=seconds)" \
"$RESULT" \
>> "$OUTPUT"
sleep 1
done
```

The script is started before the failure is introduced.

5.17.4 Failover procedure

1. Confirm both selected VPN paths and BGP peers are operational.
2. Start the continuous application probe.
3. Record the initial route state.
4. Record the exact failure-injection time. 5. Disable one selected GCP VPN connection, AWS connection or advertised route through an approved reversible method.
6. Observe failed probes.
7. Observe BGP status and route withdrawal.
8. Confirm traffic resumes through the remaining path.
9. Record the restoration time.
10. Restore the disabled path.
11. Confirm both paths return to the intended state.
12. Save Google Cloud and AWS logs.

The test should be conducted during an approved maintenance window.

5.17.5 Failover test cases

Test ID	Failure condition	Expected result	Actual result	Recovery time	Status
FO-01	Disable selected VPN path 1	Tra ic recovers through path 2	[INSERT]	[INSERT] s	[PASS/FAIL]
FO-02	Restore path 1	Redundancy returns	[INSERT]	[INSERT] s	[PASS/FAIL]
FO-03	Disable selected VPN path 2	Tra ic recovers through path 1	[INSERT]	[INSERT] s	[PASS/FAIL]
FO-04	Withdraw one BGP route	Alternative route selected	[INSERT]	[INSERT] s	[PASS/FAIL]
FO-05	Stop AWS service without network failure	Network remains available; health check fails	[INSERT]	[INSERT] s	[PASS/FAIL]
FO-06	Restore service	Health response resumes	[INSERT]	[INSERT] s	[PASS/FAIL]

5.17.6 Failover event log

Event	Timestamp
Continuous probe started	[INSERT]

Failure injected [INSERT]
First failed request [INSERT]
BGP peer detected down [INSERT]
Alternative route became active [INSERT]
First successful response [INSERT]
Stable responses confirmed [INSERT]
Original path restored [INSERT]

5.17.7 Failover chart

Figure 5.6: Application availability during VPN failover
Insert a time-series chart using the continuous probe file. The horizontal axis should show elapsed seconds relative to failure injection. The vertical axis should show response time in seconds. Failed requests may be represented at the timeout value and marked distinctly.
Annotate the failure and stable-restoration points.

5.17.8 Failover interpretation

The resilience criterion is achieved where:
failure of one path does not create permanent loss of communication;
the alternative path is used;
the measured recovery time is acceptable for the representative workload;
the event is visible in monitoring records;
restoration of the original path does not create persistent instability.
AWS Site-to-Site VPN exposes VPN and BGP monitoring information through Amazon CloudWatch and BGP-status views. Current AWS documentation distinguishes platform metrics from diagnostic logs and provides dedicated VPN monitoring references.
Failover-analysis paragraph template
The first path was disabled at [INSERT TIME]. The first failed application probe occurred at [INSERT TIME], while stable successful responses resumed at [INSERT TIME]. The measured recovery time was therefore [INSERT] seconds. During the transition, [INSERT] requests failed and [INSERT] requests exceeded the normal latency range. The corresponding BGP peer changed to [INSERT STATUS], and the alternative route became active after [INSERT] seconds. The result [demonstrates/does not demonstrate] successful path redundancy. The recovery time was [acceptable/not acceptable] for the representative workload because [INSERT REASON].

5.18 Result-Processing and Chart Generation

5.18.1 Results dataset

The final result files should be structured as follows:

```
results/
├── functional-tests.csv
├── segmentation-tests.csv
├── identity-tests.csv
├── ping-results.csv
├── iperf-tcp-results.csv
├── iperf-udp-results.csv
├── application-response.csv
├── failover-probes.csv
├── prowler-summary.csv
└── scoutsuite-summary.csv
```

5.18.2 Ping chart script

```
from pathlib import Path
import matplotlib.pyplot as plt
import pandas as pd
INPUT_FILE = Path("results/ping-results.csv")
OUTPUT_FILE = Path("results/latency-by-run.png")
def main() -> None:
if not INPUT_FILE.exists():
raise FileNotFoundError(f"Missing results file: {INPUT_FILE}")
results = pd.read_csv(INPUT_FILE)
required_columns = {"direction", "run", "average_rtt_ms"}
missing = required_columns.di erence(results.columns)
```

```
if missing:
raise ValueError(f"Missing columns: {sorted(missing)}")
for direction, group in results.groupby("direction"):
ordered = group.sort_values("run")
plt.plot(
ordered["run"],
ordered["average_rtt_ms"],
marker="o", label=direction,
)
plt.axhline(
y=100,
linestyle="--",
label="100 ms threshold",
)
plt.xlabel("Test run")
plt.ylabel("Average round-trip latency (ms)")
plt.title("AWS-GCP round-trip latency")
plt.legend()
plt.tight_layout()
plt.savefig(OUTPUT_FILE, dpi=300)
plt.close()
if __name__ == "__main__":
main()
```

This script produces Figure 5.3 after actual results are entered.

5.18.3 Throughput chart script

```
from pathlib import Path
import matplotlib.pyplot as plt
import pandas as pd INPUT_FILE = Path("results/iperf-tcp-results.csv")
OUTPUT_FILE = Path("results/tcp-throughput.png")
def main() -> None:
if not INPUT_FILE.exists():
raise FileNotFoundError(f"Missing results file: {INPUT_FILE}")
results = pd.read_csv(INPUT_FILE)
required_columns = {
"direction",
"parallel_streams",
"receiver_mbps",
}
missing = required_columns.di erence(results.columns)
if missing:
raise ValueError(f"Missing columns: {sorted(missing)}")
summary = (
results.groupby(["parallel_streams", "direction"])["receiver_mbps"]
.mean()
.unstack()
.sort_index()
) summary.plot(kind="bar")
plt.xlabel("Parallel streams")
plt.ylabel("Average receiver throughput (Mbps)")
plt.title("Average TCP throughput across the AWS-GCP VPN")
plt.xticks(rotation=0)
plt.tight_layout()
plt.savefig(OUTPUT_FILE, dpi=300)
plt.close()
if __name__ == "__main__":
main()
```

5.18.4 Failover chart script

```
from pathlib import Path
```

```
import matplotlib.pyplot as plt
import pandas as pd
INPUT_FILE = Path("results/failover-probes.csv")
OUTPUT_FILE = Path("results/failover-response-time.png")
def main() -> None:
if not INPUT_FILE.exists():
raise FileNotFoundError(f"Missing results file: {INPUT_FILE}")
results = pd.read_csv(INPUT_FILE, parse_dates=["timestamp"]) required_columns = {"timestamp", "http_code",
"total_time"}
missing = required_columns.di erence(results.columns)
if missing:
raise ValueError(f"Missing columns: {sorted(missing)}")
results = results.sort_values("timestamp")
start_time = results["timestamp"].iloc[0]
results["elapsed_seconds"] = (
results["timestamp"] - start_time
).dt.total_seconds()
plt.plot(
results["elapsed_seconds"],
results["total_time"],
)
plt.xlabel("Elapsed time (seconds)")
plt.ylabel("Application response time (seconds)")
plt.title("Application behaviour during VPN failover")
plt.tight_layout()
plt.savefig(OUTPUT_FILE, dpi=300)
plt.close() if __name__ == "__main__":
main()
```

The scripts must be run only after the actual CSV files have been populated.

5.19 Consolidated Results

5.19.1 Overall evaluation table

Evaluation	Metric	Target	Actual	Outcome area	result
Functional	End-to-end service	Successful HTTP	[INSERT]	[MET/NOT call	response MET]
Functional	Terraform drift	No unexplained	[INSERT]	[MET/NOT change	MET]
Security	Public database	None	[INSERT]	[MET/NOT exposure	MET]
Security	Prohibited traffic	All blocked	[INSERT]	[MET/NOT MET]	
Identity	Unauthorised user	Access denied	[INSERT]	[MET/NOT MET]	
Identity	Privileged MFA	Enforced	[INSERT]	[MET/NOT MET]	
Security	Prowler critical	Zero unresolved	[INSERT]	[MET/NOT posture	findings MET]
Security	Prowler high	Zero unresolved	[INSERT]	[MET/NOT posture	findings MET] Security ScoutSuite serious Zero unresolved or [INSERT] [MET/NOT posture findings formally treated MET]
Logging	Known test events	All required sources	[INSERT]	[MET/NOT retrievable	MET]
Latency	Mean RTT	Below 100 ms	[INSERT]	[MET/NOT ms	MET]
Packet loss	Normal operation	No sustained loss	[INSERT]%	[MET/NOT MET]	
Throughput	TCP throughput	Measured and	[INSERT]	[MET/NOT suitable for workload	Mbps MET]
Resilience	Single-path failure	Recovery	[INSERT]	[MET/NOT demonstrated	MET]
Resilience	Measured RTO	Documented and	[INSERT] s	[MET/NOT acceptable	MET]
Encryption	IPsec/IKE	Verified	[INSERT]	[MET/NOT MET]	

5.19.2 Success-rate calculation

The result should not be used as a substitute for professional judgement. Some criteria, such as a prohibited database connection succeeding, are more significant than a minor logging delay.

5.20 Analysis of Results

5.20.1 Functional effectiveness

The functional analysis should determine whether the architecture operated as one integrated system rather than as two unrelated cloud deployments.

A successful /aws-health response provides evidence that the GCP entry point, web service, application service, GCP routing, Network Connectivity Center, VPN, AWS gateway, AWS routing and AWS supporting service all operated together. However, the test does not prove that every component is highly available or secure. Those matters are evaluated separately.

Final narrative to populate

Of the [INSERT] functional tests conducted, [INSERT] passed and [INSERT] failed, producing a pass rate of [INSERT]%. The principal end-to-end application test [succeeded/failed]. The result showed that [INSERT INTERPRETATION]. The failed tests concerned [INSERT] and were addressed through [INSERT REMEDIATION]. After remediation, [INSERT] tests were repeated and [INSERT] passed.

5.20.2 Security effectiveness

Security effectiveness must be assessed through combined evidence. The architecture should not be declared secure solely because Prowler or ScoutSuite reported a high score.

The strongest evidence consists of:

successful authorised flows;
failed unauthorised flows;
no public database endpoint;
federated temporary sessions;
denied excessive-role actions;
IPsec tunnel verification;
retrievable audit logs;
no unresolved critical or high configuration weaknesses.

Final narrative to populate

The segmentation tests showed that [INSERT] of [INSERT] prohibited flows were blocked. The direct web-to-database test returned [INSERT RESULT], while the authorised application-to-database test returned [INSERT RESULT]. This combination demonstrates [INSERT INTERPRETATION]. Prowler identified [INSERT] initial findings, of which [INSERT] were valid and in scope. Following remediation, [INSERT] unresolved high-risk findings remained. ScoutSuite identified [INSERT], including [INSERT KEY THEMES]. The security objective was therefore [ACHIEVED/PARTIALLY ACHIEVED/NOT ACHIEVED].

5.20.3 Performance effectiveness

Performance analysis must compare actual measurements with the predefined criterion and the workload's needs.

A lightweight health-check service can tolerate a different latency profile from a highly interactive payment or trading application. The project should therefore avoid generalising the result to all enterprise workloads.

Final narrative to populate

Mean inter-cloud latency was [INSERT] ms, which was [INSERT] ms below/above the 100 ms project threshold. The 95th percentile application-response time was [INSERT] ms. Average TCP throughput ranged from [INSERT] Mbps for one stream to [INSERT] Mbps for [INSERT] parallel streams. These measurements indicate that the architecture was [INSERT ASSESSMENT] for the representative workload. The results should not be interpreted as a production service-level commitment because the test used [INSERT VM SIZES], an Internet-based VPN and a limited observation period.

5.20.4 Resilience effectiveness

A successful failover result requires more than restoration after manual repair. The report should distinguish:

automatic path failover;
BGP convergence;
manual route correction;
service restart;
complete recovery.

Final narrative to populate

During the controlled failure of [INSERT PATH], the first application failure was recorded at [INSERT TIME]. Stable communication resumed after [INSERT] seconds. The transition caused [INSERT] failed requests. Monitoring records showed [INSERT EVENT], while the BGP peer status changed from [INSERT] to [INSERT]. The architecture therefore demonstrated [AUTOMATIC/MANUAL/UNSUCCESSFUL] recovery. The measured RTO was considered [ACCEPTABLE/UNACCEPTABLE] for the lab workload because [INSERT REASON].

5.21 Discussion of Results

5.21.1 Relationship between security and connectivity

The expected result of the design is that the VPN provides private reachability without creating unrestricted trust. Where the GCP application can reach the AWS service but the AWS service cannot reach the GCP database, the architecture demonstrates separation between connectivity and authorisation.

This is consistent with the Zero Trust principle applied in Chapter Three: network location or tunnel membership does not independently establish permission. Access remains subject to routes, VPC firewall rules, security groups, identity and application controls.

5.21.2 Value of multiple security-assessment methods

Prowler and ScoutSuite use provider APIs and predefined checks to identify security concerns. Their findings should overlap in areas such as public exposure, logging and identity, but each tool may provide different coverage or classification. Prowler supports multiple cloud providers and provider-specific CLI scans, while ScoutSuite is designed to collect cloud configuration and present attack-surface information for review.

The use of two tools reduces dependence on one scanner. It also introduces the risk of duplicate counting. The reconciliation table is therefore important to the reliability of the findings.

5.21.3 Interpretation of latency

The measured latency represents the complete round trip across the GCP network, Network Connectivity Center, IPsec gateway, Internet transport, AWS gateway and AWS VPC.

A value below 100 ms supports the project criterion but does not prove that every application will perform adequately. Applications that perform many sequential cross- cloud database operations may amplify latency. The architecture should therefore keep tightly coupled components close together and use inter-cloud calls for appropriately designed service interactions rather than highly chatty database traffic.

5.21.4 Interpretation of throughput

Throughput must be interpreted as an end-to-end result, not as a direct measurement of the VPN Gateway's advertised maximum. The smallest VM, operating-system network stack, TCP behaviour or current Internet path may become the bottleneck.

Where multiple streams materially improve throughput, the result may indicate that one TCP flow did not fully use the available bandwidth-delay product. This pattern does not guarantee that the application will obtain the same aggregate throughput unless it also uses concurrent flows.

5.21.5 Interpretation of failover

Failover performance depends on tunnel detection, BGP withdrawal, route selection and application retry behaviour. A network path may recover while an application still reports errors because existing TCP sessions were lost.

The project therefore measures application probes rather than relying only on gateway status. This provides stronger evidence of user-visible recovery.

5.21.6 Monitoring effectiveness

Cloud Audit Logs, VPC Flow Logs, Security Command Center, Amazon CloudWatch and VPN diagnostics provide different forms of evidence. Cloud Audit Logs focuses primarily on GCP control-plane activity, while VPC Flow Logs provide network-flow metadata. AWS VPN monitoring provides gateway metrics, logs and BGP information. These sources should be considered complementary rather than interchangeable.

5.22 Evaluation Against Project Objectives

5.22.1 Objective One

Objective: To review multi-cloud architectures, interconnection approaches, Zero Trust frameworks and relevant Nigerian requirements and identify the limitations that justify the proposed solution.

Evidence: Chapter Two literature review, comparative analysis and gap matrix.

Evaluation question Outcome

Were relevant cloud concepts reviewed? Yes

Were Google Cloud and AWS technologies reviewed? Yes Were Zero Trust and CSA frameworks considered? Yes

Was a practical gap identified? Yes

Was the Nigerian context considered? Yes

Evaluation: Objective One was achieved through the literature and technology review. The review established that existing guidance is often provider-specific, conceptual or insufficiently evaluated as an integrated AWS-GCP implementation.

5.22.2 Objective Two

Objective: To design a secure hub-and-spoke AWS-GCP architecture incorporating segmentation, identity, encryption and centralised observability.

Evidence: Chapter Three architecture diagram, network diagram, traffic-flow matrix, security model, identity model and monitoring design.

Design requirement Evidence Outcome
Hub-and-spoke Network Connectivity Center and AWS Achieved architecture hub design
Segmented workload Separate subnet and Achieved tiers security policies
Encrypted IPsec VPN design Achieved in design; verify actual connectivity implementation
Identity federation Entra-GCP design [INSERT ACTUAL STATUS]
Central monitoring Google Cloud and AWS monitoring [INSERT ACTUAL STATUS] design

Evaluation: Objective Two is [ACHIEVED/PARTIALLY ACHIEVED], subject to the actual implementation and test evidence inserted in this chapter.

5.22.3 Objective Three

Objective: To implement the architecture across Google Cloud and AWS using Terraform. Evidence: Terraform source code, successful plan and apply outputs, remote state, deployed resources and final drift check.

Implementation measure Result
GCP infrastructure managed by Terraform [INSERT]
AWS infrastructure managed by Terraform [INSERT]
VPN resources managed by Terraform [INSERT]
Monitoring managed by Terraform [INSERT]
Final no-change plan [INSERT]
Manual steps documented [INSERT]

Evaluation narrative

Objective Three was [ACHIEVED/PARTIALLY ACHIEVED/NOT ACHIEVED]. Terraform provisioned [INSERT] of the principal architecture components. The remaining manual components were [INSERT] and were required because [INSERT REASON]. The final plan reported [INSERT].

5.22.4 Objective Four

Objective: To configure and validate Zero Trust and defence-in-depth controls.

Control Test evidence Result
Deny-by-default segmentation Segmentation tests [INSERT]
Least-privilege identity Role tests [INSERT]
Federated access SAML/SCIM tests [INSERT]
MFA Entra sign-in test [INSERT]
Encryption in transit VPN configuration [INSERT] Encryption at rest KMS and AWS KMS and Secrets Manager tests [INSERT]
Central visibility Log tests [INSERT]
Security scanning Prowler/ScoutSuite [INSERT]

Evaluation narrative

Objective Four was [INSERT OUTCOME]. The strongest evidence was [INSERT]. The principal remaining limitation was [INSERT]. The implementation reflects Zero Trust at the network, identity and monitoring layers, although [INSERT ANY APPLICATION-LEVEL LIMITATION].

5.22.5 Objective Five

Objective: To test and evaluate the architecture against security, performance and resilience criteria.

Evaluation Metric Result Outcome dimension
Security High/critical findings [INSERT] [MET/NOT MET]
Security Blocked unauthorised [INSERT] [MET/NOT MET] flows
Performance Mean latency [INSERT] [MET/NOT MET] ms
Performance TCP throughput [INSERT] [REPORTED] Mbps
Resilience Recovery [INSERT] [MET/NOT MET] demonstrated
Resilience RTO [INSERT] s [ACCEPTABLE/NOT ACCEPTABLE]

Evaluation narrative Objective Five was [ACHIEVED/PARTIALLY ACHIEVED/NOT ACHIEVED]. The architecture achieved [INSERT] of the predefined evaluation criteria. The principal successful results were [INSERT], while the criteria not achieved were [INSERT]. These limitations are addressed in the recommendations in Chapter Six.

5.23 Overall Evaluation of the Artefact

The artefact should be classified using the following scale:

Classification	Meaning
Fully successful	All critical functional, security and resilience criteria met; performance threshold met
Substantially	Critical criteria met, with minor non-critical limitations successful
Partially successful	Core connectivity works, but one or more important security, performance or resilience criteria not met
Unsuccessful	Core architecture cannot operate securely or reliably

Final classification

Based on the completed test results, the artefact is classified as [INSERT CLASSIFICATION].

Justification

This classification is supported by [INSERT NUMBER] passed functional tests, [INSERT] correctly enforced segmentation rules, [INSERT] unresolved high-risk security findings, mean latency of [INSERT] ms, throughput of [INSERT] Mbps, and a measured recovery time of [INSERT] seconds.

5.24 Limitations of the Evaluation

The evaluation is subject to several limitations. First, it is performed at laboratory scale. The selected VMs, gateway tiers, traffic volume and test duration do not reproduce the complete behaviour of a large bank, telecommunications operator or government platform.

Second, the performance tests use Internet-based VPN connectivity. Results may vary by time, region, routing and provider maintenance.

Third, Prowler and ScoutSuite are configuration-assessment tools. They do not provide a full penetration test, application-code review or legal-compliance audit.

Fourth, the workload uses synthetic data and a simplified three-tier application. It does not simulate every dependency, user pattern or transaction type found in a production enterprise environment.

Fifth, some security services may have restricted functionality because of account type, licence or project budget. These limitations must be identified beside the affected result.

Sixth, a limited failover test cannot prove long-term availability. It demonstrates the system's response to the selected controlled failures only.

5.25 Chapter Summary

This chapter presented the methodology for testing and evaluating the secure AWS-GCP multi-cloud architecture.

The evaluation was structured around Design Science Research Methodology and assessed the artefact through functional, security, performance and resilience testing. Functional tests verified Terraform deployment, VPN connectivity, BGP route exchange, workload health, cross-cloud communication and logging.

Security testing combined manual segmentation tests, identity and access-control tests, encryption verification, logging review, Prowler and ScoutSuite. The design requires approved traffic to succeed while direct web-to-database, AWS-to-database and unauthorised administrative access are blocked.

Prowler and ScoutSuite were used as complementary configuration-assessment tools. The chapter defined procedures for initial scanning, finding classification, remediation, rescanning and reconciliation of duplicate results. Prowler supports provider-specific security assessment, while ScoutSuite collects cloud configurations and highlights risk areas for investigation.

Performance testing was designed around repeated ping, traceroute, application-response and iperf3 tests. Ping measures round-trip latency and packet loss, while iperf3 measures active throughput and can report TCP retransmissions or UDP jitter and loss.

The resilience test introduced controlled tunnel or routing failures while continuous application probes recorded the interruption and restoration of service. The measured Recovery Time Objective was defined as the interval between the first failed probe and stable restoration of communication.

The chapter also provided result tables, formulas, chart specifications and scripts for converting actual test data into publication-quality figures. The final evaluation must be based on observed implementation evidence, including security-assessment outputs, connectivity measurements, throughput tests and controlled failover results. Every field marked [INSERT] should be replaced with the corresponding recorded evidence.

Finally, the evaluation framework mapped the test evidence to all five project objectives. The final evaluation must be based on observed implementation evidence, including security-assessment outputs, connectivity measurements, throughput tests and controlled failover results. The final success classification should therefore be completed only after this evidence has been recorded and analysed.

Transition to Chapter Six

Chapter Six Will Summarise The Project, State The Final Conclusion, Explain Its Professional

contributions and present recommendations for enterprise adoption and future enhancement. Its conclusions must be based on the completed results in this chapter rather than on the expected behaviour of the design.

Chapter Six
SUMMARY, CONCLUSION, AND RECOMMENDATIONS

SUMMARY, CONCLUSION AND RECOMMENDATIONS

6.0 Introduction

This chapter concludes the Professional Master's Project on the design and implementation of a secure AWS-GCP multi-cloud architecture for enterprise workloads. It summarises the work undertaken in Chapters One to Five, evaluates the extent to which the project objectives were achieved, identifies the professional and technical contributions of the project, and provides practical recommendations for Nigerian enterprises considering AWS-GCP multi-cloud adoption.

The project was conceived as an applied and solution-driven information technology project rather than a conventional theoretical thesis. This approach is consistent with the MIVA Open University Professional Master's Project Guide, which requires students to identify a real-world information technology problem, design and implement a practical solution, and evaluate the resulting system in relation to security, performance and effectiveness.

The project addressed the practical difficulty of securely integrating enterprise workloads across Google Cloud Platform and Amazon Web Services (AWS). Although both cloud platforms provide mature networking, identity, encryption and monitoring services, their use within one enterprise environment introduces additional complexity. Di erent resource models, security controls, identity systems and logging formats must be brought together without creating unrestricted trust between the two clouds.

The solution developed in this project is a lab-scale, hub-and-spoke multi-cloud architecture that combines segmented Google Cloud and AWS networks, encrypted IPsec connectivity, dynamic routing, federated identity, least-privilege access, centralised monitoring and Infrastructure as Code. The architecture is designed around Zero Trust and defence-in-depth principles and is implemented primarily through Terraform.

The findings and final conclusions of the project must, however, be understood in relation to the actual implementation evidence. Chapter Five deliberately uses result placeholders where real Prowler, ScoutSuite, ping, iperf3 and failover outputs have not yet been inserted.

Consequently, this chapter distinguishes between objectives that have been fully achieved through documented research and system design and objectives whose final achievement depends on the completion and insertion of actual deployment and testing evidence.

6.1 Summary of the Project

6.1.1 Summary of Chapter One

Chapter One Introduced The Project And Established Its Practical Context. It Explained That

cloud computing has become an important platform for delivering enterprise applications, data services, identity functions, analytics and disaster recovery. Enterprises increasingly use more than one public cloud provider to access differentiated services, reduce concentration risk and create additional resilience options.

The chapter distinguished multi-cloud architecture from simple cloud adoption. A multi- cloud environment does not become secure or resilient merely because workloads exist in two provider environments. Additional connectivity can create new routes for lateral movement, while duplicated identities and fragmented monitoring can weaken access control and incident detection.

The practical problem identified was the tendency for AWS-GCP environments to evolve through ad hoc VPNs, broad routes and independently managed security rules. Such arrangements may create inconsistent policy enforcement, excessive network trust, duplicated credentials, weak visibility and untested recovery assumptions.

Chapter One Established That The Project Would Address This Problem By Designing,

implementing and evaluating a secure AWS-GCP architecture for a representative three- tier enterprise workload. The approved proposal provided the foundation for the aim, objectives, scope and deliverables, while erroneous references to Google Cloud services in parts of the draft proposal were interpreted as drafting inconsistencies because the technical services described were Amazon Web Services (AWS) services.

The chapter also defined the scope of the project. The implementation was limited to Google Cloud and AWS and used synthetic data at proof-of-concept scale. It did not include a third cloud provider, a formal regulatory audit, unrestricted penetration testing, full application- security assessment, production-scale traffic or dedicated connectivity such as GCP Cloud Interconnect and AWS ExpressRoute.

The significance of the project was established in relation to enterprise architecture, cybersecurity practice, Infrastructure as Code and the Nigerian digital environment. Nigeria's National Cloud Policy 2025 establishes a Cloud First direction and provides technical and governance expectations for secure national cloud adoption, while the Nigeria Data Protection Act 2023 creates legal obligations affecting personal-data security, accountability and cross-border processing.

6.1.2 Summary of Chapter Two

Chapter Two Reviewed The Concepts, Theories, Technologies And Previous Research Relevant to the project.

The conceptual review covered cloud computing, enterprise workloads, multi-cloud computing, hub-and-spoke networking, segmentation, interoperability, defence in depth and shared responsibility. The review established that multi-cloud environments can improve flexibility and provide additional resilience options, but they also increase management and security complexity.

The theoretical review identified Design Science Research as the principal methodological foundation of the project. DSRM was considered appropriate because the project develops and evaluates a technical artefact designed to solve a practical enterprise problem.

Zero Trust was reviewed as the principal security philosophy. NIST SP 800-207 explains that Zero Trust shifts protection away from static network perimeters and towards users, assets and resources. Network location should not create automatic trust. NIST SP 800-207A extends this principle to cloud-native applications operating across multiple locations and emphasises granular application-level access control.

The technology review examined Google Cloud VPC, Network Connectivity Center, HA VPN, IAM, Cloud Audit Logs, Cloud Monitoring, Security Command Center and KMS. It also reviewed Amazon VPC, VPN Gateway, Virtual WAN, Microsoft Entra ID, Amazon CloudWatch, AWS Security Hub and GuardDuty and AWS KMS and Secrets Manager.

The chapter examined IPsec VPNs, BGP, route-based connectivity, identity federation, SAML, SCIM and Terraform Infrastructure as Code. Terraform was found to be suitable for implementing the architecture because it supports both Google Cloud and AWS providers and makes infrastructure configuration reproducible and reviewable.

The Cloud Security Alliance Cloud Controls Matrix was reviewed as a vendor-neutral control framework that could be used to organise cloud-security requirements relating to identity, encryption, network security, monitoring, data protection and resilience.

The critical analysis showed that no single framework or vendor document provides the complete implemented solution required by this project. NIST defines Zero Trust principles, the Cloud Security Alliance provides control objectives, and GCP and Microsoft document their respective services. The practical task remains to integrate these materials into one secure, functioning and independently evaluated architecture.

The gap analysis therefore identified a shortage of documented and reproducible GCP- AWS implementations that combine native connectivity, segmentation, identity federation, centralised monitoring, Terraform and measurable security, performance and resilience evaluation within a Nigerian enterprise context.

6.1.3 Summary of Chapter Three

Chapter Three Presented The Methodology And Detailed System Design.

Design Science Research Methodology was selected because it supports the construction and evaluation of an artefact intended to solve an identified practical problem. The project was mapped to the six DSRM activities: problem identification, definition of solution objectives, design and development, demonstration, evaluation and communication.

Functional requirements were defined for:
provisioning segmented Google Cloud and AWS networks;
establishing encrypted inter-cloud connectivity;
exchanging approved routes using BGP;
deploying a representative inventory-management workload;
restricting communication through explicit policies;
supporting federated identity;
collecting security and operational logs;
conducting cloud-security assessment;
measuring latency and throughput;
performing failover tests; and
managing infrastructure through Terraform.

Non-functional requirements addressed security, availability, performance, maintainability, scalability and auditability. The principal performance target was an average inter-cloud round-trip latency below 100 milliseconds under the selected lab conditions.

The proposed architecture placed the primary web, application and database tiers in GCP, while AWS hosted a supporting service, monitoring capabilities and a recovery-oriented component. Google Cloud Network Connectivity Center provided central GCP routing, and AWS Site-to-Site VPN provided the AWS-side tunnel endpoint.

The design used distinct address spaces and separate subnets for public ingress, web, application, database, management, transit, AWS service, monitoring and gateway functions. Direct communication between the web tier and database tier was prohibited.

AWS workloads were not permitted direct access to the GCP database. The security model mapped NIST Zero Trust functions to actual platform controls. Microsoft Entra ID and Google Cloud Workforce Identity Federation supported federated workforce access.

GCP VPC firewall rules, AWS Network VPC Firewall Rules, route tables and host controls functioned as policy-enforcement mechanisms. Logging, threat detection and configuration scanning provided continuous diagnostics and assurance.

The chapter also presented architecture, network, use-case, sequence, deployment, entity-relationship and failover diagrams in Mermaid syntax. These models established a clear design basis for implementation.

6.1.4 Summary of Chapter Four

Chapter Four Described The Implementation Of The System.

The implementation began with the administrative workstation, Terraform, gcloud CLI, AWS CLI and Git. A protected AWS Storage Account was designed as the Terraform remote- state backend, with versioning, retention and restricted access.

The GCP implementation created:

an isolated VPC;
public, web, application, database, management and transit subnets;
an Internet Gateway and lab-scale NAT Gateway;
separate route tables;
Google Cloud Network Connectivity Center;
stateful VPC firewall rules;
KMS encryption;
IAM roles and instance profiles;
GCP Secrets Manager;
Compute Engine workloads;
an Application Load Balancer;
Cloud Audit Logs;
Cloud Monitoring;
VPC Flow Logs; and
Security Command Center.

The AWS implementation created:

a resource group;
an AWS VPC;
gateway, service, monitoring and management subnets;
Network VPC Firewall Rules;
an active-active AWS Site-to-Site VPN;
an AWS supporting-service VM;
a system-assigned managed identity;
AWS KMS and Secrets Manager;
Log Analytics;
Amazon CloudWatch diagnostic settings;
Amazon CloudWatch Agent; and
AWS Security Hub and GuardDuty plans.

The inter-cloud network was designed around route-based IPsec VPN connections and BGP. Current Microsoft guidance provides a supported architecture for connecting Google Cloud and AWS using an active-active AWS Site-to-Site VPN and multiple GCP site-to-site connections.

Terraform code was provided for the major infrastructure components. The implementation also documented the cross-provider dependency between AWS gateway addresses and GCP-generated tunnel parameters. A staged deployment procedure was therefore proposed.

Microsoft Entra ID groups and Google Cloud Workforce Identity Federation permission sets were designed to support federated access. SAML was selected for authentication and SCIM for automated user and group provisioning.

Chapter Four Also Documented Implementation Challenges Relating To Vpn Negotiation,

BGP peer configuration, state protection, cloud costs, federation prerequisites and service- provisioning time. Screenshot placeholders were used instead of fabricated screenshots. This preserves academic integrity because actual evidence must come from the implemented environment.

6.1.5 Summary of Chapter Five

Chapter Five Developed The Testing Methodology And Evaluation Framework.

Functional tests were designed to verify Terraform deployment, VPN tunnel status, BGP route exchange, application availability, database access, end-to-end AWS-GCP communication and monitoring.

Security tests covered:

public exposure;
web-to-database access;
AWS-to-database access;
identity federation;
role restriction;
multifactor authentication;
IPsec configuration;
encryption at rest;
Cloud Audit Logs;
VPC Flow Logs;
AWS CloudWatch and CloudTrail diagnostics;
Security Command Center;
AWS Security Hub and GuardDuty;
Prowler; and
ScoutSuite.

Performance testing was designed around repeated ping, traceroute, application-response and iperf3 measurements. The framework included TCP throughput, optional UDP throughput, jitter, packet loss and retransmission analysis.

Resilience testing involved disabling one selected VPN path while continuous probes measured service interruption and restoration. The measured Recovery Time Objective was defined as the di erence between the first failed request and stable restoration of communication.

Chapter Five Also Provided Tables, Formulas, Chart Specifications And Python Scripts For

processing the actual results.

However, the numerical result fields in Chapter Five remain placeholders pending actual execution. It would therefore be inaccurate to state conclusively that the security, latency, throughput and resilience thresholds have already been achieved. Those conclusions must be completed after real evidence is inserted.

6.2 Evaluation of the Project Objectives

6.2.1 Objective One

Objective One: To review multi-cloud network architectures, interconnection methods, Zero Trust frameworks and relevant Nigerian requirements and produce a documented gap analysis.

This objective is addressed by the design; final empirical confirmation depends on the completed lab evidence.

Chapter Two Reviewed More Than 25 Relevant Academic And Professional Sources Covering

cloud computing, multi-cloud adoption, security, identity federation, Zero Trust, VPN technology and Infrastructure as Code. NIST SP 800-207 and SP 800-207A were examined as the principal Zero Trust sources. The Cloud Security Alliance Cloud Controls Matrix was considered as a control framework, and o icial GCP and Microsoft documentation was used to explain platform-specific technologies.

The review also considered the Nigerian regulatory and policy context. The National Cloud Policy 2025 establishes an updated Cloud First framework and is accompanied by a National Cloud Technical Document containing technical and operational standards. The Nigeria Data Protection Act 2023 provides the statutory framework for personal-data processing, while the NDPC's General Application and Implementation Directive provides additional guidance on application of the Act.

The gap analysis established that existing materials do not provide a complete, independently evaluated AWS-GCP reference implementation combining networking, identity, security monitoring, Terraform and measurable testing in a Nigerian enterprise context.

Assessment: Achieved.

6.2.2 Objective Two

Objective Two: To design a secure hub-and-spoke AWS-GCP multi-cloud architecture, including segmentation, identity federation, encrypted inter-cloud connectivity and centralised observability, for a representative inventory-management enterprise workload.

This objective was achieved at the design level.

Chapter Three Presented A Complete Architecture Incorporating:
Google Cloud and AWS network hubs;
separate workload subnets;
controlled routing;
route-based IPsec VPN;
BGP;
web, application and database tiers;
AWS supporting services;
Entra-to-GCP federation;
least-privilege access;
KMS and AWS KMS and Secrets Manager;
Cloud Audit Logs and Amazon CloudWatch;
Security Command Center and AWS Security Hub and GuardDuty; and
centralised or correlated logging.
The architecture reflects Zero Trust by avoiding the assumption that an encrypted VPN creates automatic permission. NIST SP 800-207 describes Zero Trust as an approach focused on users, assets and resources rather than static network location, while NIST's implementation guidance specifically recognises resources distributed across on- premises and multiple cloud environments.
Assessment: Achieved at design level.

6.2.3 Objective Three

Objective Three: To implement the designed architecture using Terraform across Google Cloud and AWS.
This objective was substantially addressed through implementation design and complete reference code, but final achievement requires evidence from the deployed environment.
Chapter Four Provided Terraform Configurations For:
providers and remote state;
Google Cloud and AWS networks;
Network Connectivity Center;
AWS Site-to-Site VPN;
HA VPN;
BGP;
route tables;
VPC firewall rules and security groups;
Compute Engine and Amazon EC2 instances;
IAM and Entra groups;
KMS and AWS KMS and Secrets Manager;
Cloud Audit Logs and Amazon CloudWatch;
Security Command Center and AWS Security Hub and GuardDuty; and
workload deployment.
The code, directory structure, execution workflow and troubleshooting procedures are sufficient to support a reproducible lab implementation. Nevertheless, the final project should include actual Terraform plan, apply and final no-change outputs before the objective is declared fully achieved.
Assessment: Substantially achieved in the report; final confirmation pending deployment evidence.

6.2.4 Objective Four

Objective Four: To configure and enforce security controls, including encryption in transit, micro-segmentation, identity federation, centralised monitoring and policy-based access. This objective was achieved at design and implementation-specification level, but final operational achievement depends on test evidence.
The design includes:
IPsec encryption;
separate application tiers;
security-group and NSG restrictions;
federated identities;
MFA expectations;
temporary Google Cloud federated sessions;
KMS and AWS KMS and Secrets Manager;
Cloud Audit Logs and Amazon CloudWatch;
Security Command Center and AWS Security Hub and GuardDuty;
Prowler and ScoutSuite; and
documented deny-by-default traffic rules.

However, the actual effectiveness of these controls must be demonstrated through successful and unsuccessful connection tests, sign-in evidence, scanner reports and log records.

Assessment: Partially confirmed; final achievement pending security-test outputs.

6.2.5 Objective Five

Objective Five: To test and evaluate the implemented architecture against predefined security, performance and resilience metrics.

The testing methodology was fully developed, but the objective cannot yet be regarded as conclusively achieved because the actual numerical outputs have not been inserted.

Chapter Five Defined Comprehensive Tests For:
functionality;
segmentation;
identity;
encryption;
logging;
Prowler;
ScoutSuite;
ping;
traceroute;
iperf3;
application response;
VPN failure; and
recovery time.

The project's evaluation criteria require actual measurements. In particular, the architecture should not be described as meeting the sub-100-millisecond latency target, producing a particular throughput level, having zero high-risk findings or achieving a specific Recovery Time Objective until the real test results are available.

Assessment: Methodology achieved; empirical evaluation pending actual results.

6.2.6 Consolidated objective assessment

Objective	Assessment	Basis
Review literature and	Achieved	Completed critical review and gap identify gaps analysis
Design secure GCP-	Achieved	Complete architecture and design AWS architecture models
Implement through	Substantially	Full reference implementation; Terraform achieved deployment evidence required
Configure Zero Trust	Partially confirmed	Controls designed and coded; security controls operating evidence required Test security,
Pending empirical	Test methodology complete; actual performance and	completion results outstanding resilience

The overall project is therefore best described as a comprehensively designed and implementation-ready professional artefact whose final empirical validation must be completed through the execution and insertion of the Chapter Five results.

6.3 Conclusion

The project demonstrates that secure AWS-GCP multi-cloud architecture requires more than establishing network connectivity between two providers. The principal challenge is maintaining consistent security outcomes across different cloud technologies, administrative models and operational processes.

The proposed architecture addresses this challenge through six integrated design decisions.

First, the architecture separates workload tiers and limits communication to approved paths. The GCP web tier cannot communicate directly with the database, and AWS workloads are not granted unrestricted access to Google Cloud resources.

Second, the architecture treats the VPN as an encrypted transport rather than a trusted network extension. Route availability does not automatically create permission. Security groups, Network VPC Firewall Rules, workload configuration and identity policies remain responsible for authorising access.

Third, the architecture centralises connectivity through Google Cloud Network Connectivity Center and an AWS hub design. This is more scalable and governable than a growing collection of unmanaged point-to-point connections.

Fourth, the architecture integrates Microsoft Entra ID with GCP identity services. Federated access reduces the need for separate long-lived credentials and supports centralised onboarding, role assignment and o boarding.

Fifth, Terraform expresses the architecture as reviewable and repeatable code. This supports consistency, version control and controlled deployment. It also creates a basis for future policy-as-code and automated security scanning.

Sixth, the architecture includes monitoring, threat detection and testing as integral design components rather than treating them as post-implementation additions. The project also demonstrates that multi-cloud should not be adopted solely as a response to vendor lock-in concerns or as a fashionable architectural preference. It introduces additional cost, skills requirements, operational dependencies and troubleshooting complexity. The decision should be supported by clear workload, resilience, regulatory or service-differentiation requirements.

Where those requirements exist, a native AWS-GCP architecture can provide a practical and defensible solution. Microsoft's current guidance confirms that BGP-enabled active- active AWS Site-to-Site VPN connectivity can be established with GCP site-to-site connections, while NIST guidance supports Zero Trust access to resources distributed across multiple cloud environments.

For Nigerian enterprises, the design must also be connected to data governance. The National Cloud Policy 2025 promotes cloud adoption but also establishes expectations relating to sovereignty, classification, security and national digital development. The Nigeria Data Protection Act requires organisations to maintain appropriate safeguards and accountability for personal-data processing. Multi-cloud architecture must therefore be supported by a clear understanding of what data is processed, where it travels, who can access it and which party is responsible for each control.

Subject to the completion of the Chapter Five empirical results, the architecture provides a credible proof-of-concept reference for secure AWS-GCP enterprise integration.

6.4 Professional Contributions of the Project

6.4.1 Reproducible AWS-GCP reference architecture

The first professional contribution is a structured reference architecture that practitioners can adapt for AWS-GCP enterprise integration.

The artefact connects provider-native services without relying on a proprietary third-party multi-cloud networking platform. This makes the architecture easier to inspect and provides direct experience with Google Cloud and AWS networking behaviour.

The design includes addressing, routing, segmentation, gateway configuration, identity, encryption, monitoring and workload placement. It is therefore more useful than a high- level diagram that omits implementation relationships.

6.4.2 Provider-to-provider control mapping

The project maps comparable Google Cloud and AWS security functions without assuming that the technologies are identical. Examples include:

Security objective	GCP implementation	AWS implementation
Isolated network	Google Cloud VPC	AWS VPC
Stateful filtering	VPC Firewall Rules	Network VPC Firewall Rules
Central transit	Network Connectivity Center	Hub VNet or Virtual WAN
Encrypted connection	HA VPN	VPN Gateway
Workforce access	Workforce Identity Federation	Microsoft Entra ID
Key management	Cloud KMS	AWS KMS and Secrets Manager
Threat detection	Security Command Center	AWS Security Hub and GuardDuty
Operational logging	Cloud Audit Logs and Cloud Monitoring	Activity Log and Amazon CloudWatch

This mapping can assist enterprise architects and security teams that need to express one organisational policy through different provider controls.

6.4.3 Infrastructure-as-Code implementation

The Terraform implementation contributes a reusable approach to provisioning and managing multi-cloud infrastructure.

The code addresses:

provider configuration;
state protection;
reusable variables;
resource tagging;
network modules;
gateway dependencies;
workload deployment;
identity roles;
monitoring; and
controlled teardown.

It also highlights an important practical issue: successful automation does not necessarily mean secure automation. Terraform state can contain sensitive data, and Infrastructure as Code can reproduce incorrect configurations as e iciently as correct ones. The project therefore combines IaC with review, security scanning and post-deployment validation.

6.4.4 Integrated Zero Trust implementation

The project translates Zero Trust principles into concrete provider controls.
Rather than relying only on perimeter protection, it combines:
federated identity;
MFA;
short-lived sessions;
restricted routes;
security-group and NSG enforcement;
workload separation;
logging;
posture assessment; and
controlled negative testing.
This contribution is professionally relevant because Zero Trust is often discussed abstractly. The architecture demonstrates how the principle of removing implicit network trust can be implemented within an AWS-GCP environment. NIST's current implementation guidance confirms that Zero Trust is applicable to distributed enterprise resources across on-premises and multiple cloud environments.

6.4.5 Security, performance and resilience evaluation framework

The project contributes a practical evaluation method that goes beyond checking whether the VPN is connected.
It combines:
positive connectivity testing;
negative access testing;
Prowler;
ScoutSuite;
ping;
traceroute;
iperf3;
application-response measurement;
tunnel-failure testing; and
objective traceability.
This framework can be reused by practitioners conducting proof-of-concept assessments for cloud connectivity.

6.4.6 Nigerian-context architectural guidance

The project connects global cloud-security frameworks with Nigerian policy and legal considerations.
Its contribution is not the creation of a Nigeria-specific VPN protocol. Rather, it demonstrates how Nigerian data-protection and cloud-policy considerations influence technical decisions such as:
workload placement;
data classification;
cross-cloud flows;
audit logging;
access control;
encryption;
service-provider oversight;
incident evidence; and
retention.
This practical translation is important because compliance discussions often remain at policy level without being connected to network routes, VPC firewall rules, identity roles and cloud logs.

6.4.7 Professional documentation and knowledge transfer

The report includes:
architecture diagrams;
network diagrams;
sequence diagrams;
Terraform code;
CLI commands;
security matrices;

test cases;
result templates;
risk registers; and
implementation guidance.
These outputs can support architecture review, internal training, proof-of-concept delivery, system handover and future enhancement.

6.5 Practical Recommendations for Nigerian Enterprises

6.5.1 Begin with business and regulatory justification

A Nigerian enterprise should not begin multi-cloud adoption by connecting two cloud networks. It should first establish the business reason for using both Google Cloud and AWS.

Valid reasons may include:
differentiated provider services;
organisational identity integration;
disaster recovery;
concentration-risk reduction;
mergers or inherited technology;
customer or contractual requirements;
data-location considerations;
operational resilience; or
workload-specific performance requirements.
The organisation should document which workloads require multi-cloud integration and which can remain in one provider. Unnecessary cross-cloud coupling creates cost and security complexity without corresponding value.

6.5.2 Conduct data discovery and classification

Before deciding where to host workloads, the enterprise should identify:
categories of data processed;
data owners;
data subjects;
sensitivity;
processing purpose;
retention period;
location;
transfer pathways;
service providers; and
applicable legal restrictions.
This is particularly important under the Nigeria Data Protection Act 2023, which establishes accountability and safeguards for personal-data processing. The NDPC's implementation guidance should be incorporated into enterprise privacy and cloud- governance processes.

Data classification should directly influence architecture. Highly restricted data should not be exposed to both cloud environments merely because a VPN exists.

6.5.3 Adopt a formal cloud-governance framework

The enterprise should define a cloud-governance model before large-scale deployment. The model should address:
account and subscription structure;
naming and tagging;
approved regions;
identity ownership;
network standards;
data classification;
encryption;
logging;
incident response;
supplier management;
cost ownership;
architecture approval;
change control; and
resource decommissioning.
The National Cloud Policy 2025 and associated technical document should be reviewed as part of this governance framework, particularly where the enterprise operates in a regulated or public-sector environment.

6.5.4 Establish a multi-cloud landing zone

Google Cloud and AWS should not be opened as unrestricted administrative environments.

Each provider should have a governed landing zone covering:

organisational hierarchy;
accounts or subscriptions;
central logging;
identity federation;
baseline security policies;
network hubs;
approved regions;
KMS or AWS KMS and Secrets Manager;
security monitoring;
budget controls; and
mandatory tags.

The landing zones should be designed together so that equivalent organisational outcomes are achieved even where the provider technologies di er.

6.5.5 Use federated identity as the default

Nigerian enterprises using Microsoft 365 and Entra ID should consider using Microsoft Entra ID as the central workforce identity source and federating access into Google Cloud Workforce Identity Federation.

Federation should include:

SAML or OpenID Connect where appropriate;
automated user and group provisioning;
MFA;
conditional access;
role-based permissions;
short session durations for privileged access;
rapid deprovisioning;
emergency account governance; and
regular access reviews.

Long-lived Google Cloud IAM user credentials should not be the normal access model for administrators.

6.5.6 Apply least privilege to network routes

Enterprises should avoid advertising complete Google Cloud and AWS address spaces unless every advertised prefix is necessary.

A better approach is to:

advertise only required prefixes;
separate production, development and management routes;
use distinct Network Connectivity Center route tables;
restrict AWS route propagation;
place inspection controls at hubs where justified;
document all permitted flows;
test prohibited flows; and
review routes after every major deployment.

BGP simplifies route exchange, but incorrect BGP design can also spread unintended routes quickly. Microsoft provides current guidance for BGP-enabled AWS-GCP connectivity and troubleshooting peer or route-learning problems.

6.5.7 Do not treat the VPN as a trusted perimeter

The encrypted connection should not convert both clouds into one unrestricted internal network.

Enterprises should retain controls at several levels:

route tables;
cloud firewalls;
VPC firewall rules;
security groups;
workload firewalls;
API gateways;
application authentication;
database permissions;
workload identity; and

encryption.
This reflects NIST's principle that access should not be granted solely on the basis of network location.

6.5.8 Separate tightly coupled workload components

Latency-sensitive or highly chatty components should generally remain close together.
For example, an application that performs hundreds of sequential database calls should not place the application tier in GCP and the database in AWS without strong performance evidence. The round-trip latency of every cross-cloud call can compound.
Multi-cloud boundaries are better used for:
asynchronous processing;
analytics;
backup;
disaster recovery;
loosely coupled APIs;
identity;
monitoring;
batch processing; or
replicated services.

6.5.9 Use dedicated connectivity for critical production workloads

Internet-based IPsec VPN is suitable for proof-of-concept environments, moderate traffic and backup connectivity.
Enterprises with strict throughput, latency or availability requirements should assess:
GCP Cloud Interconnect;
AWS ExpressRoute;
carrier-based cloud exchange;
redundant provider circuits;
separate physical paths;
contractual service levels; and
encrypted overlay requirements.
Dedicated connectivity does not remove the need for application and identity security. It changes the transport characteristics.

6.5.10 Deploy all available VPN paths

Google Cloud HA VPN connections normally provide multiple tunnels. AWS redundant Site-to-Site VPN can also provide multiple gateway instances.
The enterprise should configure and monitor all intended paths rather than leaving secondary tunnels unused. Current Microsoft guidance for AWS-GCP active-active BGP connectivity describes multiple GCP tunnels and corresponding AWS connections.
Redundancy should be tested periodically. A tunnel that has never carried traffic should not be assumed to work during an incident.

6.5.11 Centralise logging and security operations

The enterprise should identify the authoritative sources of security evidence across both clouds.
At a minimum, it should collect and retain:
GCP Cloud Audit Logs;
Google Cloud VPC and Network Connectivity Center flow logs;
Security Command Center findings;
AWS CloudTrail;
Entra sign-in logs;
AWS Site-to-Site VPN diagnostics;
AWS Security Hub and GuardDuty findings;
AWS KMS and Secrets Manager audit logs;
application logs; and
identity-assignment changes.
Logs should be normalised or correlated through a security information and event management platform where operationally justified.
Retention should reflect business, incident-response, regulatory and evidential requirements. The enterprise should also test whether known events can be retrieved. Logging that is enabled but never reviewed provides limited operational benefit.

6.5.12 Use Infrastructure as Code with security gates

Terraform or an equivalent tool should be adopted as the normal method for provisioning cloud resources.
The delivery process should include:
1. feature branch;
2. peer review;
3. formatting and validation;
4. secret scanning;
5. Terraform plan;
6. IaC security scanning;
7. policy-as-code evaluation;
8. approved deployment;
9. cloud-posture scan;
10. drift monitoring.

Terraform state should be encrypted, versioned and accessible only to approved identities. State files should never be committed to public repositories.

6.5.13 Introduce policy as code

As maturity increases, the organisation should automate policy enforcement.
Examples include preventing:
public database endpoints;
open SSH or RDP access;
deployment outside approved regions;
unencrypted storage;
resources without required tags;
unrestricted security-group rules;
disabled audit logging;
unmanaged public IP addresses; and
excessive administrative roles.

Policy as code reduces dependence on manual review and prevents known high-risk configurations before deployment.

6.5.14 Perform continuous posture assessment

Prowler and ScoutSuite are useful for proof-of-concept evaluation, but enterprises should implement continuous cloud-security posture management rather than rely only on periodic manual scans.
Findings should be:
assigned to owners;
classified by business context;
remediated within defined timelines;
retested;
monitored for recurrence; and
reported to governance committees.

A scanner's numerical score should not replace risk assessment. Findings involving public exposure, identity privilege, encryption or disabled logging should generally receive priority.

6.5.15 Test resilience as an operational process

A multi-cloud diagram does not prove resilience.
Enterprises should test:
tunnel failure;
BGP withdrawal;
gateway failure;
region failure;
identity-provider unavailability;
DNS failure;
application dependency failure;
log-ingestion failure;
secret-store unavailability; and
recovery from incorrect infrastructure deployment.

Each test should define:
scenario;
expected outcome;

responsible team;
recovery procedure;
RTO;
Recovery Point Objective where data is involved;
evidence;
lessons learned; and
improvement action.

6.5.16 Maintain cost governance

Multi-cloud can create duplicated expenditure across gateways, logging, data transfer, security services and operational teams.

Nigerian enterprises should monitor:
inter-region charges;
cross-cloud data-egress charges;
VPN Gateway cost;
Network Connectivity Center cost;
AWS and Google Cloud security-service subscriptions;
Log Analytics and Cloud Monitoring ingestion;
idle virtual machines;
duplicated backup;
unused public IP addresses; and
non-production environments.

Cost allocation tags and budget alerts should be mandatory. Architecture teams should also consider exchange-rate volatility where cloud services are billed in foreign currency.

6.5.17 Build specialist skills and operating procedures

Multi-cloud architecture requires engineers who understand both provider models and the interactions between them.

Enterprises should develop skills in:
Google Cloud networking;
AWS networking;
BGP;
IPsec;
Entra ID;
Google Cloud IAM;
Terraform;
cloud logging;
incident response;
data protection;
FinOps; and
platform engineering.

Operational runbooks should be developed for common problems, including tunnel failure, BGP route loss, certificate expiry, SCIM failure, Terraform state locking and privileged- access incidents.

6.5.18 Maintain an exit and portability plan

A multi-cloud strategy should still include an exit plan.

The organisation should know:
which services are provider-specific;
which data formats are portable;
how data will be exported;
how long migration would take;
what dependencies exist;
what egress costs apply;
how encryption keys are handled; and
how backups can be restored elsewhere.

Terraform improves infrastructure reproducibility but does not automatically make applications portable. Portability must be designed at the application and data levels.

6.5.19 Obtain legal, regulatory and assurance review

For regulated organisations, cloud design should be reviewed by:
information security;

data-protection o icers;
legal advisers;
risk management;
internal audit;
compliance;
business continuity;
procurement; and
relevant regulators where required.
This project is a technical reference and not a substitute for a formal compliance assessment or legal opinion.

6.6 Recommended Implementation Roadmap

A Nigerian enterprise may adopt the architecture through the following phased approach.

Phase One: Discovery and governance

The enterprise should identify business drivers, data categories, regulatory obligations, workloads, owners and risk appetite. No inter-cloud connection should be created before the scope and governance model are approved.

Phase Two: Landing-zone establishment

Google Cloud and AWS account structures, identity federation, logging, key management, budgets and policy baselines should be deployed.

Phase Three: Network foundation

Distinct non-overlapping address spaces, hubs, route tables, VPN gateways and security boundaries should be established. A small test network should be connected before production routes are introduced.

Phase Four: Pilot workload

A low-risk, representative application should be deployed. Approved flows and prohibited flows should be documented and tested.

Phase Five: Security integration

Prowler, AWS Security Hub and GuardDuty, Security Command Center, SIEM integration, access reviews and incident workflows should be configured.

Phase Six: Performance and resilience validation

Latency, throughput and failover tests should be performed using expected workload patterns. The organisation should reject or redesign the architecture where performance or recovery results do not meet business requirements.

Phase Seven: Controlled production migration

Workloads should be migrated in phases with rollback plans, monitoring and formal approval.

Phase Eight: Continuous improvement

Architecture drift, cost, security findings, access assignments and resilience results should be reviewed regularly.

6.7 Recommendations for Future Enhancement of the Project

The following enhancements are recommended for future work.

6.7.1 Complete empirical testing

The immediate priority is to execute all Chapter Five tests and replace the placeholders with actual results. This includes:

Prowler;
ScoutSuite;
ping;
traceroute;
iperf3;
application-response measurement;
segmentation testing;
identity testing; and
failover testing.

6.7.2 Implement all four AWS-GCP tunnel paths

The reference implementation should be extended to represent all GCP-generated tunnels as AWS Customer Gateways and VPN connections. Current Microsoft guidance for active-active AWS-GCP connectivity describes four GCP tunnels and corresponding AWS connections.

6.7.3 Compare VPN with dedicated connectivity

A future study could compare:

Internet-based VPN;
Cloud Interconnect and ExpressRoute;

third-party cloud exchanges; and
software-defined multi-cloud networking platforms.
The comparison should consider latency, throughput, availability, cost and operational complexity.

6.7.4 Introduce containerised workloads and service mesh

The three-tier virtual-machine workload could be replaced with services hosted in Google Cloud EKS and AWS Kubernetes Service.

A service mesh could then provide:
mutual TLS;
workload identity;
service-level policy;
telemetry; and
finer-grained Zero Trust enforcement.
This would align more closely with the application-level controls described in NIST SP 800- 207A.

6.7.5 Implement cross-cloud workload identity

Future implementation should avoid static application credentials entirely and evaluate workload identity federation for machine-to-machine access.

6.7.6 Add a central SIEM and SOAR capability

Google Cloud and AWS security events could be ingested into a central SIEM such as Microsoft Sentinel or another suitable platform.

Automated response playbooks could address:
suspicious sign-in;
security-group expansion;
unexpected route changes;
disabled logging;
exposed storage; and
VPN failure.

6.7.7 Implement full policy as code

Terraform plans could be checked with tools such as Open Policy Agent, Checkov or equivalent controls before deployment.

6.7.8 Conduct longer-term performance observation

Performance should be measured over several days and at different times to capture variation in Internet routing and cloud activity.

6.7.9 Conduct formal compliance mapping

A future project could map implemented controls to:
the Nigeria Data Protection Act;
NDPC implementation guidance;
the National Cloud Policy;
sector-specific requirements;
NIST SP 800-53;
ISO/IEC 27001;
PCI DSS where payment data is involved; and
the Cloud Security Alliance Cloud Controls Matrix.

6.7.10 Extend to hybrid multi-cloud

The architecture could be expanded to include an on-premises data centre, private cloud or Nigerian local cloud provider. This would allow a more direct investigation of data- residency and hybrid integration requirements.

6.8 Final Project Position

This project has produced a detailed and professionally structured solution for secure AWS-GCP multi-cloud integration.

Its strongest completed elements are:
problem definition;
literature and technology review;
gap analysis;
Design Science Research methodology;
complete architecture;

network and security models;
Terraform implementation;
identity and monitoring design;
test methodology; and
Nigerian enterprise recommendations.
Its principal outstanding element is the insertion of actual empirical test evidence.
Accordingly, the defensible final position is:
The project has successfully designed and documented an implementation-ready secure AWS-GCP multi-cloud architecture based on Zero Trust, defence in depth, federated identity and Infrastructure as Code. The architecture provides a credible professional reference for Nigerian enterprise workloads. Final confirmation of its security, performance and resilience effectiveness depends on completing the planned implementation tests and inserting the actual Prowler, ScoutSuite, latency, throughput and failover results.
This conclusion maintains academic integrity and avoids presenting expected outcomes as completed experiments.

6.9 Chapter Summary

This chapter summarised the complete project and evaluated its objectives.
Chapter One Defined The Enterprise Multi-Cloud Problem, Aim, Scope And Significance.
Chapter Two Reviewed Relevant Concepts, Research, Frameworks And Technologies And
identified the implementation and evaluation gap. Chapter Three applied Design Science Research Methodology and produced the complete system design. Chapter Four provided the Terraform implementation and configuration guidance for GCP, AWS, VPN, identity, encryption, logging and monitoring. Chapter Five developed the testing and evaluation methodology for functionality, security, performance and resilience.
The first objective, concerning literature review and gap analysis, was achieved. The second objective, concerning architecture design, was also achieved. The third objective was substantially achieved through complete Terraform implementation guidance, although actual deployment evidence is required. The fourth and fifth objectives require the insertion of real security and performance outputs before they can be declared fully achieved.
The project's professional contributions include a reproducible AWS-GCP reference architecture, provider control mapping, Terraform code, an integrated Zero Trust model, a combined testing framework and practical guidance for Nigerian enterprises.
The recommendations emphasise that Nigerian organisations should begin with business justification, data classification and governance rather than immediate network connection. They should establish landing zones, federate identity, limit route advertisements, treat the VPN as an untrusted transport, centralise monitoring, use Infrastructure as Code, test failover, manage cost and incorporate Nigerian data-protection and cloud-policy requirements into technical decisions.
The completed project demonstrates that secure multi-cloud architecture is not achieved by the presence of two providers. It is achieved through deliberate design, explicit trust decisions, automated control, continuous visibility and evidence-based evaluation.

IMPLEMENTATION REPOSITORY AND REPRODUCIBILITY NOTE

The implementation code accompanying this report is structured for the GitHub repository https://github.com/lesileugwulebo/repo_miva.git. The repository is divided into three Terraform stages so that provider public addresses and BGP information can be resolved before the dependent cross-cloud tunnels are created. Secrets and state files are excluded from version control. The submitted repository should contain the exact revision used to collect Chapter Five evidence.

Representative Terraform pattern

```
resource "google_compute_ha_vpn_gateway" "ha_vpn" {
  name    = "verdad-gcp-ha-vpn"
  network = google_compute_network.hub.id
  region  = var.gcp_region
}

resource "awsrm_virtual_network_gateway" "vpn" {
  name          = "vng-verdad-aws"
  type          = "Vpn"
  vpn_type      = "RouteBased"
  active_active = true
  enable_bgp    = true
  sku           = "VpnGw1AZ"
  # two Standard public IP configurations are defined in the repository
}
```

The full repository also contains deployment, evidence-collection, ping, traceroute, curl and iperf3 helpers. The report intentionally avoids reproducing every source file because doing so would increase descriptive bulk without improving the architectural argument. Chapter Four instead explains implementation decisions and points to the version-controlled artefact.

REFERENCES

Alonso, J., Orue-Echevarria, L., Casola, V., Torre, A. I., Huarte, M., Osaba, E., & Lobo, J. L. (2023). Understanding the challenges and novel architectural models of multi-cloud native applications: A systematic literature review. Journal of Cloud Computing, 12, 6. https://doi.org/10.1186/s13677-022-00367-6

Chandramouli, R., & Butcher, Z. (2023). A zero trust architecture model for access control in cloud-native applications in multi-location environments (NIST SP 800-207A). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-207A

Cloud Security Alliance. (2026). Cloud Controls Matrix v4.1 and implementation guidance.

Diningrat, D. C., Suhardi, & Rahardjo, B. (2025). Security issues in multi-cloud: A systematic literature review. IEEE Access, 13, 70006-70017. https://doi.org/10.1109/ACCESS.2025.3561352

Federal Republic of Nigeria. (2023). Nigeria Data Protection Act, 2023. Nigeria Data Protection Commission.

Gambo, M. L., & Almulhem, A. (2026). Zero Trust Architecture: A systematic literature review. Journal of Network and Systems Management, 34, Article 25. https://doi.org/10.1007/s10922-025-09998-x

Google Cloud. (2026). Availability in Cloud SQL. Google Cloud Documentation.

Google Cloud. (2026). Cloud HA VPN and Cloud Router documentation. Google Cloud Documentation.

Google Cloud. (2026). Configure Workforce Identity Federation with Microsoft Entra ID. Google Cloud Documentation.

Google Cloud. (2026). Security Command Center overview. Google Cloud Documentation.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. MIS Quarterly, 28(1), 75-105.

Mell, P., & Grance, T. (2011). The NIST definition of cloud computing (NIST SP 800-145). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-145

Microsoft. (2025). What is hybrid identity with Microsoft Entra ID? Microsoft Learn.

Microsoft. (2026). Amazon CloudWatch Logs overview. Microsoft Learn.

Microsoft. (2026). AWS Site-to-Site VPN and BGP documentation. Microsoft Learn.

National Information Technology Development Agency. (2025). National Cloud Policy 2025. Federal Republic of Nigeria.

National Information Technology Development Agency. (2025). National Cloud Technical Document 2025. Federal Republic of Nigeria.

National Institute of Standards and Technology. (2020). Zero Trust Architecture (NIST SP 800-207).

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. Journal of Management Information Systems, 24(3), 45-77.

Reece, M., Lander, T. E., Stoffolano, M., Sampson, A., Dykstra, J., Mittal, S., & Rastogi, N. (2023). Systemic risk and vulnerability analysis of multi-cloud environments. arXiv:2306.01862.

Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2020). Zero Trust Architecture (NIST SP 800-207). National Institute of Standards and Technology.

Verdet, A., Hamdaqa, M., Da Silva, L., & Khomh, F. (2023). Exploring security practices in Infrastructure as Code: An empirical study. arXiv:2308.03952.

APPENDIX A: REPOSITORY STRUCTURE

```
repo_miva/
├── terraform/
│   ├── 01-core/        # AWS/GCP core networks and VPN gateways
│   ├── 02-vpn/         # GCP HA VPN ↔ AWS Site-to-Site VPN, IPsec and BGP
│   └── 03-workloads/   # inventory primary and AWS DR test hosts
├── scripts/             # deployment and evidence collection
```

```
├── tests/              # latency, route, HTTP and throughput tests
├── docs/               # implementation guide and evidence register
├── .gitignore
└── README.md
```

APPENDIX B: CHAPTER FIVE EVIDENCE COMPLETION CHECKLIST

1. Terraform version and successful plan/apply output
2. GCP VPC, subnet, firewall, Cloud Router and HA VPN screenshots/CLI output
3. AWS VPC, NSG, redundant Site-to-Site VPN and route evidence
4. Both IPsec tunnel states and BGP peer state
5. Effective/learned route evidence on both providers
6. Entra-to-GCP federated sign-in and least-privilege role evidence
7. Authorised application-flow test
8. Prohibited web-to-database or management-flow test
9. Cloud Audit Logs/VPC Flow Logs/Security Command Center evidence
10. Amazon CloudWatch/AWS Security Hub and GuardDuty evidence
11. Repeated latency and packet-loss measurements
12. iperf3 throughput measurements
13. Controlled tunnel failure, interruption time and measured recovery
14. Final Terraform plan demonstrating expected/no unapproved drift

All placeholders in Chapter Five should be replaced with the observed values before final academic submission. Screenshots and numerical values must come from the student-controlled implementation environment.